LECTURE NOTES

From the PCP Theorem to Unique Games

Proofs, algorithms and geometry
of hardness of approximation

Sixteen lectures from *proof checking* to the *Unique Games Conjecture*
Draft of October 10, 2026

About these notes

These notes retrace one of the most fruitful lines of research in theoretical computer science: how the question *"is there a proof of this statement?"* turned into the question *"how well can this optimization problem be approximated?"*, and how the second question, in turn, led to a single conjecture—the *Unique Games Conjecture* (UGC)—that seems to explain the answer for a startlingly large family of problems. The plan is one lecture per paper-sized idea: the language of games and the PCP theorem (Lectures 1–4), amplification by parallel repetition and by Dinur's gap amplification (5–6), Håstad's optimal inapproximability (7), the UGC and its consequences in analysis, geometry and semidefinite programming (8–12), the 2-to-2 Games theorem (13–15), and a critical reading of the 2026 claim of a full proof of the UGC (16).

Prerequisites. Undergraduate algorithms and the basics of NP-completeness. Everything else (approximation ratios, gap problems, games, ...) is defined when first needed.

How to read the boxes.

- **Blue boxes** summarize the big picture or a key idea; if you are short on time, read the blue boxes and the statements of the theorems.
- **Orange boxes** flag common misconceptions.
- **Wine-red boxes** state the *epistemic status* of a result. Whenever a claim has not been independently verified, we say so there and nowhere do we treat it as established.

How results are labelled. A result with an attribution in its heading, e.g. "Theorem (PCP theorem; Arora–Safra, Arora–Lund–Motwani–Sudan–Szegedy)", is quoted from the literature; we give a proof only if one follows the statement, and otherwise point to where the proof can be found. Statements that depend on an unproven conjecture carry that conjecture in their name, e.g. "Theorem (assuming the UGC)".

Exercises. Each lecture ends with exercises marked [$\star$] (routine), [$\star\star$] (needs an idea) or [$\star\star\star$] (substantial). Several exercises come with a hint.

Contents

About these notes **i**

1 Why PCP and Unique Games Matter **1**
1.1 A scoreboard of approximability . . . 2
1.2 Approximation algorithms in one page . . . 3
1.3 Gap problems: the language of hardness . . . 3
1.4 Why ordinary reductions cannot create a gap . . . 4
1.5 The PCP theorem: two faces . . . 6
1.6 Two provers, Label Cover, and parallel repetition . . . 8
1.7 Unique Games and the Unique Games Conjecture . . . 12
1.8 The map of the territory . . . 14
Exercises . . . 16
Notes and further reading . . . 18

2 Games, Label Cover, and the Language of PCPs **19**
2.1 Constraint graphs: one language for everything . . . 19
2.2 One oracle or two provers? . . . 22
2.3 From verifiers and CSPs to games . . . 24
2.4 The ladder of constraint types . . . 25
2.5 From PCPs to cliques: the FGLSS graph . . . 27
2.6 What Label Cover is for . . . 29
Exercises . . . 30
Notes and further reading . . . 31

3 The Arora–Safra Route: Polynomials, Sum-Check and Composition **32**
3.1 What was proved, and when . . . 33
3.2 Polynomials as codes . . . 34
3.3 Arithmetization: satisfiability as a vanishing condition . . . 35
3.4 The sum-check protocol, as a table . . . 37
3.5 The low-degree test . . . 39
3.6 The basic verifier . . . 41
3.7 Normal form verifiers and split-encoded assignments . . . 42
3.8 The composition lemma . . . 44
3.9 The parameter chase . . . 46
3.10 Consequence: Clique within $2^{\log^{1/2-\varepsilon} N}$ is NP-hard . . . 47
3.11 Why it stops at $\log^{1/2+\varepsilon} n$, and how to reach $O(1)$ . . . 48
Exercises . . . 49
Notes and further reading . . . 50

4 Constant Queries: Hadamard Codes, Parallelization and the ALMSS Proof **51**
4.1 Outer verifiers, inner verifiers, and composition again . . . 52
4.2 The Hadamard code and its local tests . . . 54

4.3 Verifying a circuit with Hadamard codes 56
4.4 Lines: low-degree testing and correction with $O(1)$ queries 58
4.5 Curves: collapsing l queries into one 60
4.6 The parallelized verifier 61
4.7 Assembling the proof: three steps, constant parameters 63
4.8 Consequences, and the link to games 65
4.9 What the proof does not do, and what replaces it 65
Exercises 66
Notes and further reading 67

LECTURE 1

Why PCP and Unique Games Matter

From checking proofs to approximating optima

Color every vertex of a graph red or blue, and call an edge *cut* if its endpoints get different colors. Flipping a fair coin for each vertex cuts every edge with probability $\frac{1}{2}$, so some coloring cuts at least half of all edges. In 1994 Goemans and Williamson (Goemans and Williamson, 1995) showed how to do much better: using semidefinite programming and a random hyperplane, one can find in polynomial time a cut that contains at least $0.8785\ldots$ times as many edges as a *maximum* cut. More than thirty years later that constant has not moved. Is it a limit of our imagination, or a limit of mathematics?

Here is a remarkable partial answer. The best *unconditional* hardness result says that, unless $\mathsf{P} = \mathsf{NP}$, no polynomial-time algorithm beats $16/17 \approx 0.941$. But if the *Unique Games Conjecture* (UGC) is true, then no polynomial-time algorithm can beat $0.8785\ldots + \varepsilon$ for any $\varepsilon > 0$ (Khot et al., 2007; Mossel et al., 2010): the strange constant defined by a calculus minimization is *exactly* the truth. The conjecture was formulated by Khot in 2002 (Khot, 2002). As we write this (October 2026) it is in an unusual position: a manuscript dated 23 September 2026 claims a proof (Section 1.8). We treat the conjecture as a conjecture throughout, and say so whenever a result depends on it.

Every step of this story—what "hard to approximate" means, why we can prove it at all, and why the conjecture takes such a peculiar form—traces back to a single idea:

> *To prove that an optimization problem is hard to approximate, we need proofs that can be checked by reading only a few bits.*

This lecture builds the vocabulary (gap problems, probabilistically checkable proofs, two-prover games, Label Cover, Unique Games), proves the handful of short results that explain *why* the vocabulary looks the way it does, and draws the map on which the remaining fifteen lectures sit.

After this lecture you should be able to…

(1) define a gap problem, and explain how NP-hardness of a gap problem rules out approximation algorithms (Lemma 1.4);
(2) explain why the classical reductions of the 1970s cannot produce a gap, and what the PCP theorem adds (Section 1.4);
(3) translate between PCP verifiers, CSPs, two-prover games and Label Cover, and prove the two forms of the PCP theorem equivalent (Proposition 1.9);
(4) state the Unique Games Conjecture with its quantifiers in the right order, and say why a Unique Game with perfect completeness is easy;
(5) locate each later lecture on a map of the field (Section 1.8).

Problem	Best algorithm	NP-hard beyond	If the UGC holds, hard beyond
Max-E3-Lin($\mathbb{F}_2$)	$\frac{1}{2}$ (random)	$\frac{1}{2}+\varepsilon$ (Håstad, 2001)	(already tight)
Max-E3-SAT	$\frac{7}{8}$ (random)	$\frac{7}{8}+\varepsilon$ (Håstad, 2001)	(already tight)
Max-Cut	$\alpha_{\mathrm{GW}} \approx 0.8786$ (Goemans and Williamson, 1995)	$\frac{16}{17}+\varepsilon \approx 0.941$ (Håstad, 2001; Trevisan et al., 2000)	$\alpha_{\mathrm{GW}}+\varepsilon$ (Khot et al., 2007; Mossel et al., 2010)
Vertex Cover	2 (maximal matching)	$\sqrt{2}-\varepsilon \approx 1.414$ (Dinur et al., 2018; Khot et al., 2018)	$2-\varepsilon$ (Khot and Regev, 2008)

Table 1.1: Approximability thresholds for four basic problems. Ratios are ≤ 1 for maximization and ≥ 1 for minimization problems (Definition 1.1). "NP-hard beyond α" means that no polynomial-time algorithm achieves ratio better than α unless P = NP. The first two rows are unconditionally tight; for the last two, the UGC would close the gap between algorithm and hardness.

1.1 A scoreboard of approximability

Let us begin with the answer key. Table 1.1 lists four problems, the best approximation ratio known for each, and what is known about the *limit* of approximability.

The first two rows are the great success of the PCP method, and they are astonishing. Max-E3-Lin($\mathbb{F}_2$) asks for an assignment satisfying as many as possible of a given list of linear equations $x_i + x_j + x_k = b \pmod 2$. A uniformly random assignment satisfies half of the equations, whatever the system is. Håstad proved that nothing polynomial-time does better, even if you are promised that a $(1-\varepsilon)$-fraction of the equations can be satisfied simultaneously: *guessing is optimal*. The same is true for Max-E3-SAT, where a random assignment satisfies a $\frac{7}{8}$-fraction of the clauses.

The last two rows are where the story stops being a triumph. For Max-Cut and Vertex Cover the best algorithms (a clever semidefinite program; a trivial maximal matching) leave a visible gap to the best hardness proofs. Assuming the UGC, both gaps close *simultaneously* and *exactly*. In Lecture 11 we shall see that this is not a coincidence: Raghavendra (Raghavendra, 2008) proved that, assuming the UGC, for *every* constraint satisfaction problem the optimal approximation ratio is the integrality gap of one generic semidefinite program. In the other direction, the constant in the Max-Cut row is

$$\alpha_{\mathrm{GW}} \;=\; \min_{0<\theta\leq\pi} \; \frac{2}{\pi}\cdot\frac{\theta}{1-\cos\theta} \;\approx\; 0.87856, \tag{1.1}$$

and the hardness proof of Khot, Kindler, Mossel and O'Donnell reproduces precisely this minimization, with θ the angle between two random-hyperplane vectors. We will see in Lecture 9 how the same expression emerges from a Fourier-analytic argument about Gaussian noise.

The question behind the whole course

For which problems can an efficient algorithm do better than the trivial one, by how much, and how do we *prove* that nothing does better? The PCP theorem answers this for problems like Max-E3-SAT; the Unique Games Conjecture (still unproven) would answer it for a much larger class.

1.2 Approximation algorithms in one page

All problems in this lecture are constraint satisfaction problems or close relatives. An *instance* of a Max-CSP consists of variables $x_1, \dots, x_n$ with values in a finite domain and a list of m *constraints*, each depending on a few variables. The *value* of an assignment is the *fraction* of constraints it satisfies, and $\mathrm{OPT}(I)$ is the maximum value over all assignments. Examples: Max-3SAT (clauses with at most three literals), Max-E3-Lin($\mathbb{F}_2$) (equations $x_i + x_j + x_k = b$), Max-Cut (constraints $x_u \neq x_v$, one per edge).

Definition 1.1 (Approximation ratio)**.** A polynomial-time algorithm $\mathcal{A}$ is an α-*approximation* for a maximization problem ($0 < \alpha \leq 1$) if on every instance I it outputs a solution of value at least $\alpha \cdot \mathrm{OPT}(I)$. For a minimization problem $\mathcal{A}$ is a ρ-*approximation* ($\rho \geq 1$) if it outputs a solution of value at most $\rho \cdot \mathrm{OPT}(I)$. The *approximation threshold* of a problem is the best constant α (or ρ) for which an approximation algorithm exists.

Example 1.2 (Four one-paragraph algorithms)**.**

- Max-Cut, *ratio* $\frac{1}{2}$. Put each vertex on a random side. Every edge is cut with probability $\frac{1}{2}$, so the expected number of cut edges is $m/2 \geq \mathrm{OPT}/2$.
- Max-E3-Lin($\mathbb{F}_2$), *ratio* $\frac{1}{2}$. For a random assignment and any fixed equation $x_i + x_j + x_k = b$, condition on x_i, x_j: exactly one value of x_k satisfies the equation, so it holds with probability $\frac{1}{2}$.
- Max-E3-SAT, *ratio* $\frac{7}{8}$ (every clause has three literals on three distinct variables). A clause is violated by exactly one of the eight settings of its variables, hence is satisfied with probability $\frac{7}{8}$.
- Vertex Cover, *ratio* 2. Let M be a maximal matching. Its $2|M|$ endpoints form a vertex cover (an uncovered edge could be added to M), while any cover must contain one endpoint of each of the $|M|$ disjoint edges of M, so $\mathrm{OPT} \geq |M|$.

The three randomized algorithms can be derandomized by the method of conditional expectations (Exercise 1.1).

So an algorithm that does *nothing intelligent* already achieves a constant ratio. The whole subject starts from the question whether intelligence can ever pay off—and for Max-E3-SAT and Max-E3-Lin it turns out that it cannot. The tool that proves this is the subject of the next three sections.

1.3 Gap problems: the language of hardness

How does one prove that an *approximation* problem is NP-hard? Decision problems are what the theory of NP-completeness knows how to handle, so we turn the approximation problem into a decision problem—a *promise* problem.

Definition 1.3 (Gap problem)**.** Let Π be a Max-CSP and $0 \le s < c \le 1$. The *gap problem* $\textsc{Gap}_{c,s}$-Π is the following promise problem. Given an instance I of Π such that either

$$\text{(YES)}\quad \mathrm{OPT}(I) \ge c \qquad \text{or} \qquad \text{(NO)}\quad \mathrm{OPT}(I) \le s,$$

decide which of the two cases holds. On instances outside both cases, any answer is acceptable. We call c the *completeness* and s the *soundness*; the gap problem has *perfect completeness* if $c = 1$. The problem is NP*-hard* if there is a polynomial-time map f from 3-CNF formulas to instances of Π with φ satisfiable $\Rightarrow$ $f(\varphi)$ is a YES instance, and φ unsatisfiable $\Rightarrow$ $f(\varphi)$ is a NO instance.

Lemma 1.4 (Gap hardness implies inapproximability)**.** *If* $\textsc{Gap}_{c,s}$-Π *is* NP*-hard, then for every* $\alpha > s/c$ *there is no polynomial-time* α*-approximation algorithm for* Π *unless* $\mathsf{P} = \mathsf{NP}$.

Proof. Suppose $\mathcal{A}$ is a polynomial-time α-approximation with $\alpha > s/c$. We use it to decide 3SAT. Given φ, compute $I = f(\varphi)$, run $\mathcal{A}$ on I, evaluate the value v of the solution it returns (this is easy), and accept iff $v > s$. If φ is satisfiable then $\mathrm{OPT}(I) \ge c$, so $v \ge \alpha c > s$. If φ is unsatisfiable then $v \le \mathrm{OPT}(I) \le s$. Hence this is a polynomial-time algorithm for 3SAT. □

Conversely, an approximation algorithm with ratio below s/c would solve the gap problem—so the gap problem *is* the decision core of approximation. In this language, the rows of Table 1.1 read as follows:

Håstad: $\textsc{Gap}_{1-\varepsilon,\frac{1}{2}+\varepsilon}$-Max-E3-Lin and $\textsc{Gap}_{1,\frac{7}{8}+\varepsilon}$-Max-E3-SAT are NP-hard;

UGC: $\textsc{Gap}_{c,\alpha_{\mathrm{GW}}c+\varepsilon}$-Max-Cut is NP-hard (for suitable c).

Remark (Why completeness $1-\varepsilon$ rather than 1). Perfect completeness is not always attainable. A system of linear equations over $\mathbb{F}_2$ whose equations can *all* be satisfied is solved by Gaussian elimination, so $\textsc{Gap}_{1,s}$-Max-E3-Lin $\in \mathsf{P}$ for every $s < 1$ (Exercise 1.3). Håstad's theorem has to take $c = 1-\varepsilon$ for exactly this reason, and we will see the same phenomenon, for a similar reason, in the Unique Games Conjecture (Section 1.7).

1.4 Why ordinary reductions cannot create a gap

The NP-completeness theory of the 1970s (Cook, 1971; Levin, 1973; Karp, 1972) is a theory of *exact* problems, and its reductions are, in a precise sense, too gentle for approximation. This section explains why, and in doing so isolates what the PCP theorem must supply.

1.4.1 Cook–Levin yields a gap of one clause

Recall how 3SAT is shown NP-hard. Let $L \in \mathsf{NP}$ with verifier V and let x be an input. The Cook–Levin reduction outputs a formula φ_x that encodes the computation tableau of V on (x, w), where the witness w is left as free variables. The clauses say that (a) the first row of the tableau contains x, (b) every 2×3 window of the tableau is consistent with the transition function of V, and (c) the last row contains the accepting state.

Observation. For *every* x—also for $x \notin L$—there is an assignment to φ_x that violates only the clause in (c). Hence $\mathrm{OPT}(\varphi_x) \geq 1 - 1/m$, where m is the number of clauses.

Proof sketch. Choose any witness w and write down the true computation of V on (x, w) as a tableau. Constraints (a) and (b) hold by construction; (c) fails if V rejects (x, w), but it is a single clause. □

So the classical reduction maps YES instances to formulas with $\mathrm{OPT} = 1$ and NO instances to formulas with $\mathrm{OPT} = 1 - \frac{1}{m}$ exactly. It proves that $\textsc{Gap}_{1,1-1/m}$-3SAT is NP-hard, which is nothing but the statement that maximizing the number of satisfied clauses *exactly* is NP-hard. The gap is a single clause out of m, and by Lemma 1.4 it rules out only approximation ratios above $1 - \frac{1}{m}$, which is no restriction at all on an algorithm.

1.4.2 Karp reductions transport gaps, they do not create them

Suppose now that someone *gives us* a gap, and we want to carry it to another problem. The standard reduction from 3SAT to Independent Set does this perfectly.

Proposition 1.5 (The triangle reduction)**.** *Let φ be a 3-CNF with m clauses, and define the graph G_φ as follows: there is a vertex for every pair (clause C_j, literal $\ell \in C_j$); the three vertices of each clause form a triangle; and two vertices (j, ℓ), (j', ℓ') are adjacent when $\ell' = \neg\ell$. Then the size of a maximum independent set of G_φ equals the maximum number of clauses of φ that can be simultaneously satisfied.*

Proof. ($\geq$) Given an assignment satisfying t clauses, pick for each satisfied clause one satisfied literal. The t chosen vertices lie in different triangles, and no two of them carry complementary literals, since both are true under the same assignment. So they are independent. ($\leq$) Let S be an independent set. It contains at most one vertex per triangle, and the literals it contains are pairwise non-complementary, so they can all be set to true by a single assignment, which then satisfies at least $|S|$ clauses. □

Consequently $\textsc{Gap}_{c,s}$-3SAT reduces to the problem of distinguishing independent sets of size $\geq cm$ from independent sets of size $\leq sm$ in a graph with $3m$ vertices: the gap is carried over *unchanged*. The same is true for almost all the textbook reductions among NP-complete problems. They are *local gadget constructions* that preserve the quantity "how many constraints can be satisfied"; so they preserve a gap if there is one at the start, and they have none to preserve if there is none.

Gap creation versus gap preservation

Hardness of approximation has two ingredients. **(i) Gap creation**: one reduction from NP (e.g. from 3SAT) to a constraint satisfaction problem that turns "satisfiable" vs. "unsatisfiable" into "everything satisfiable" vs. "at most a $(1 - \varepsilon_0)$ fraction satisfiable." This is the PCP theorem (Section 1.5), and it does not follow from anything in the NP-completeness toolbox. **(ii) Gap preservation or amplification**: a long chain of reductions that carry the gap to other problems, sharpening its parameters on the way. Lectures 3, 4 and 6 are about (i); Lectures 5 and 7 are about (ii); and Lectures 8–16 explore what happens when ordinary gap amplification runs out of steam, which is where the Unique Games Conjecture comes in.

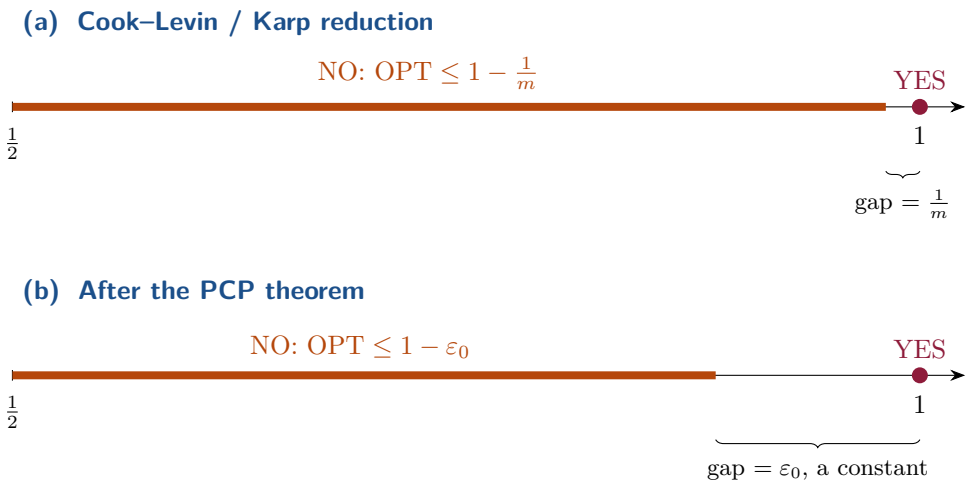


Figure 1.1: Where the optimum $\mathrm{OPT}(\varphi)$ lives for YES and NO instances of a reduction to Max-3SAT (schematic, not to scale). The classical reductions leave a single clause between the two cases; the PCP theorem moves the NO instances away by a constant fraction.

1.5 The PCP theorem: two faces

1.5.1 Proofs that can be spot-checked

A classical NP verifier reads its entire witness. Suppose instead we insist that the verifier be *randomized* and may read only a constant number of bits of a (very long) proof π. Of course it must then sometimes accept wrong proofs; the demand is that it reject every proof of a false statement with probability bounded away from 0.

Definition 1.6 (Probabilistically checkable proofs)**.** A *probabilistic verifier* V with randomness $r(n)$ and query complexity $q(n)$ is a polynomial-time algorithm that, on input x of length n and oracle access to a proof string π, draws a random string $\rho \in \{0,1\}^{r(n)}$, *non-adaptively* computes $q(n)$ positions $i_1(x,\rho), \dots, i_q(x,\rho)$, reads $\pi_{i_1}, \dots, \pi_{i_q}$, and accepts or rejects according to a predicate $P_{x,\rho}\colon \{0,1\}^q \to \{0,1\}$. Let $0 \le s < c \le 1$. A language L belongs to $\mathsf{PCP}_{c,s}[r,q]$ if there is such a verifier with

$$x \in L \implies \exists \pi : \Pr_{\rho}[V^{\pi}(x) \text{ accepts}] \ge c \qquad \text{(completeness)},$$
$$x \notin L \implies \forall \pi : \Pr_{\rho}[V^{\pi}(x) \text{ accepts}] \le s \qquad \text{(soundness)}.$$

We write $\mathsf{PCP}[r,q] = \mathsf{PCP}_{1,1/2}[r,q]$.

Two extreme cases show that the trade-off is not vacuous. With $r = 0$ and $q = n^{O(1)}$ the verifier is deterministic and reads the whole witness: $\mathsf{PCP}[0, \mathrm{poly}(n)] = \mathsf{NP}$. With $q = 0$ the proof is useless, and we get randomized polynomial time. The question is how much *randomness* (which buys coverage of the proof) and how many *queries* are needed. Note that $\mathsf{PCP}[O(\log n), O(1)] \subseteq \mathsf{NP}$ trivially: there are only $2^{O(\log n)} = \mathrm{poly}(n)$ random strings, and together they query only polynomially many positions of π, so a nondeterministic machine can guess these bits and check all random strings. The surprise is the other direction. (Requiring the queries to be non-adaptive costs nothing when q

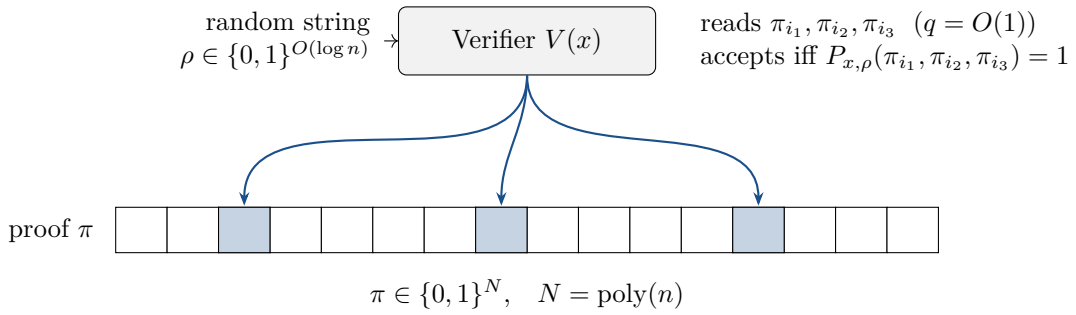


Figure 1.2: A PCP verifier. For each of polynomially many random strings ρ it inspects a constant number of proof bits and applies a local test. A correct proof passes every test; every proof of a false statement fails at least half of them.

is constant, since a verifier that makes q adaptive queries can be replaced by one that reads all $2^q - 1$ positions of its decision tree: Exercise 1.4.)

Theorem 1.7 (PCP theorem, proof-checking form; Arora–Safra, Arora–Lund–Motwani–Sudan–Szegedy)**.** $\mathsf{NP} = \mathsf{PCP}[O(\log n), O(1)]$.

Theorem 1.8 (PCP theorem, hardness form)**.** *There is a constant $\varepsilon_0 > 0$ such that* $\textsc{Gap}_{1,1-\varepsilon_0}$-3SAT *is* NP*-hard.*

Here $\textsc{Gap}_{1,1-\varepsilon_0}$-3SAT asks to distinguish satisfiable 3-CNF formulas from formulas in which every assignment violates an ε_0-fraction of the clauses. Compare with Figure 1.1: this is precisely the picture in panel (b). The two theorems look very different—one about proofs, one about optimization—but they say the same thing.

Proposition 1.9 (Equivalence of the two forms)**.** *Theorem 1.7 holds if and only if Theorem 1.8 holds.*

Proof. Hardness form ⇒ proof-checking form. Let $L \in \mathsf{NP}$. Compose the Cook–Levin reduction with the reduction witnessing the NP-hardness of $\textsc{Gap}_{1,1-\varepsilon_0}$-3SAT to obtain a polynomial-time map $x \mapsto \varphi_x$ such that $x \in L$ implies that φ_x is satisfiable, and $x \notin L$ implies that every assignment violates at least $\varepsilon_0 m$ clauses, where $m = |\varphi_x|$. The verifier expects as proof a satisfying assignment π of φ_x. It picks a clause uniformly at random ($\log m = O(\log n)$ random bits), reads the (at most) three variables occurring in it, and accepts iff the clause is satisfied. If $x \in L$, an honest proof is accepted with probability 1. If $x \notin L$ then *whatever* π is, it is an assignment and thus violates an ε_0-fraction of the clauses, so V accepts with probability at most $1 - \varepsilon_0$. Now repeat the test $k = \lceil \ln 2/\varepsilon_0 \rceil$ times with independent clauses, accepting iff all k clauses are satisfied. The proof π is fixed once and for all, so the k tests are independent given π and the acceptance probability is at most $(1-\varepsilon_0)^k \le e^{-\varepsilon_0 k} \le \frac{1}{2}$. The verifier uses $k \log m = O(\log n)$ random bits and $3k = O(1)$ queries.

Proof-checking form ⇒ hardness form. Let V be a verifier for 3SAT $\in \mathsf{NP}$ with $r = O(\log n)$ random bits, $q = O(1)$ queries, perfect completeness and soundness $\frac{1}{2}$. (We may assume $q \ge 3$.) Given a formula ψ, we build a 3-CNF φ as follows. Since $2^r = \mathrm{poly}(n)$, we can enumerate every random string ρ and compute the query positions $i_1(\rho), \dots, i_q(\rho)$ and the predicate P_ρ. Introduce one Boolean variable y_i for every position i of the proof. For each ρ we want to enforce $P_\rho(y_{i_1}, \dots, y_{i_q}) = 1$. This constraint is a

CNF with at most 2^q clauses of width q: one clause for each $a \in \{0,1\}^q$ with $P_\rho(a) = 0$, namely the clause that is false exactly on the input a. Finally, each clause $(\ell_1 \vee \cdots \vee \ell_q)$ of width q is converted into $q-2$ clauses of width 3 with fresh variables $z_1, \ldots, z_{q-3}$:

$$(\ell_1 \vee \ell_2 \vee z_1) \wedge (\neg z_1 \vee \ell_3 \vee z_2) \wedge \cdots \wedge (\neg z_{q-3} \vee \ell_{q-1} \vee \ell_q).$$

If some ℓ_j is true, one can choose the z's to satisfy all $q-2$ clauses; if all ℓ_j are false then z_1 must be true, hence z_2, …, hence z_{q-3}, and the last clause is violated, so *at least one* of the $q-2$ clauses fails for every choice of z. Let φ_ρ be the resulting 3-CNF; it has at most $K := (q-2)2^q$ clauses, and let $\varphi = \bigwedge_\rho \varphi_\rho$, which has $m \le K \cdot 2^r$ clauses.

Completeness. If ψ is satisfiable, there is a proof π accepted for every ρ. Setting $y = \pi$ and choosing the z's suitably satisfies all of φ.

Soundness. If ψ is unsatisfiable, every proof π is rejected for at least half of the random strings. Given any assignment to φ, read off $\pi = y$. For every ρ on which V^π rejects, the assignment violates *at least one* clause of φ_ρ. There are at least $2^r/2$ such ρ, so at least $2^r/2$ clauses are violated out of $m \le K2^r$, i.e. a fraction $\ge 1/(2K)$. Thus $\textsc{Gap}_{1,1-\varepsilon_0}$-3SAT is NP-hard with $\varepsilon_0 = 1/(2K)$, a constant. □

Remark (A dictionary). The proof is our first instance of a translation that we shall use constantly: *a verifier is a constraint satisfaction problem.* Random strings ρ correspond to constraints; positions of the proof correspond to variables; the predicate P_ρ is the constraint; the number q of queries is the arity; soundness s is the bound on the optimum in the NO case. We extend the dictionary to games in Section 1.6 (Table 1.2).

Pitfall: a proof is not a witness

In Definition 1.6 the soundness condition quantifies over *every* string π, including strings that have nothing to do with any computation. A PCP is useful precisely because its verifier cannot be fooled by an adversarially chosen π that is merely *locally plausible*. In the first half of the proof above this is why we can read off an assignment from π and apply the NO-case promise to it.

The PCP theorem is a deep result: the original proofs by Arora and Safra (1998) and Arora et al. (1998) are algebraic (low-degree polynomials and recursive proof composition), and Dinur (2007) later found a purely combinatorial proof by gap amplification on expander graphs. We take Theorem 1.7 for granted in this lecture, return to its proofs in Lectures 3, 4 and 6, and ask here what it gives us.

Remark (Two-sided history). Theorem 1.8 reads as a statement about *approximation*, yet the road to it was paved by results about *interactive proofs*: multi-prover interactive proofs (Ben-Or et al., 1988), NEXP = MIP (Babai et al., 1991b), and the discovery by Feige et al. (1996) that a PCP verifier can be turned into a graph whose largest clique encodes the maximum acceptance probability (Section 2.5), and hence that PCPs imply hardness of approximating Max-Clique. Section 1.6 follows the thread through the multi-prover systems.

1.6 Two provers, Label Cover, and parallel repetition

The gap in the PCP theorem is a small constant: soundness $1-\varepsilon_0$. To prove hardness results with sharp constants like $\frac{7}{8}$ and $\frac{1}{2}$ we need soundness $s \to 0$, and we need it

while keeping the *number of queries* under control. The cleanest setting for this is the following.

1.6.1 Games, labelings, projections

Definition 1.10 (Two-prover one-round game / Label Cover)**.** A *two-prover one-round game* G consists of a bipartite graph (U, W, E) with finite label sets Σ_U, Σ_W and, for each edge $e = (u, w)$, a *constraint* $C_e \subseteq \Sigma_U \times \Sigma_W$. The referee draws an edge $e = (u, w)$ uniformly at random, sends u to Alice and w to Bob, receives answers $a \in \Sigma_U$ and $b \in \Sigma_W$ from the provers (who cannot communicate), and accepts iff $(a, b) \in C_e$. The *value* of G,

$$\mathrm{val}(G) \;=\; \max_{\ell_U : U \to \Sigma_U,\ \ell_W : W \to \Sigma_W} \ \Pr_{e=(u,w)\sim E} \big[(\ell_U(u), \ell_W(w)) \in C_e\big],$$

is the maximum over *labelings*, i.e. over deterministic strategies of the two provers. Finding an optimal labeling is the optimization problem *Label Cover*. The game is a *projection game* if every C_e is the graph of a function $\pi_e \colon \Sigma_U \to \Sigma_W$: $C_e = \{(a, \pi_e(a))\}$. It is a *Unique Game* if moreover $|\Sigma_U| = |\Sigma_W| = k$ and every π_e is a bijection.

Restricting to deterministic strategies loses nothing.

Lemma 1.11 (Determinism suffices)**.** *The maximum acceptance probability over all (randomized, possibly correlated through shared randomness) strategies of Alice and Bob equals* $\mathrm{val}(G)$.

Proof. A randomized strategy with shared random string r is, for each fixed r, a deterministic strategy. Its acceptance probability is the average over r of the acceptance probabilities of the deterministic strategies, hence at most the maximum. □

Lemma 1.12 (Random labeling)**.** *In a projection game,* $\mathrm{val}(G) \geq 1/|\Sigma_W|$. *In particular, for every Unique Game with k labels* $\mathrm{val}(G) \geq 1/k$.

Proof. Fix any labels for the $u \in U$ and give every $w \in W$ an independent uniformly random label. For each edge $e = (u, w)$ the label $\pi_e(\ell_U(u))$ is fixed, so w matches it with probability exactly $1/|\Sigma_W|$. By linearity of expectation some labeling achieves at least that. □

Lemma 1.12 will matter later: no hardness theorem can have soundness δ with fewer than $1/\delta$ labels.

Example 1.13 (The clause–variable game)**.** Let φ be a 3-CNF in which every clause has three distinct variables. Alice plays the *clause prover*, Bob the *variable prover*. The referee picks a uniformly random clause C and a uniformly random variable x in it. Alice receives C and must answer with a satisfying assignment of its three variables (7 possibilities); Bob receives x and answers with a bit. They win iff Bob's bit equals Alice's value for x. This is a projection game with $|\Sigma_U| = 7$ and $|\Sigma_W| = 2$ (Figure 1.3); the projection $\pi_{(C,x)}$ sends an assignment of C to its restriction to x.

Proposition 1.14 (Soundness of the clause–variable game)**.** *If φ is satisfiable then* $\mathrm{val}(G_\varphi) = 1$. *If every assignment violates at least an ε-fraction of the clauses then* $\mathrm{val}(G_\varphi) \leq 1 - \varepsilon/3$.

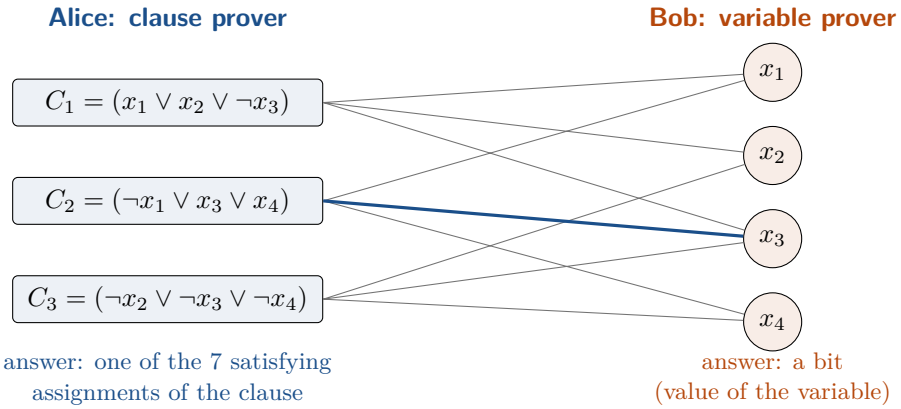


Figure 1.3: The clause–variable game of a 3-CNF. An edge is a pair (clause, variable occurring in it). The highlighted edge (C_2, x_3) is accepted iff Alice's assignment to C_2 and Bob's bit agree on x_3, i.e. iff the projection $\pi_{(C_2,x_3)}$ of Alice's label to the coordinate x_3 equals Bob's label.

PCP verifier	CSP	Two-prover game	Label Cover
random string ρ	constraint	question pair (u, w)	edge e
proof position	variable	(a prover's answer table)	vertex
q queries	arity q	2 provers	arity 2
predicate P_ρ	constraint predicate	referee's test	relation C_e
alphabet $\{0,1\}$	domain	answer set	label set Σ
soundness s	$\mathrm{OPT} \le s$	$\mathrm{val}(G) \le s$	$\mathrm{val} \le s$

Table 1.2: The dictionary. The same object is a verifier, a constraint satisfaction problem, a game, and a Label Cover instance.

Proof. If φ is satisfiable, let both provers answer according to a satisfying assignment. Conversely, fix deterministic strategies; Bob's answers define an assignment b to the variables. By assumption b violates at least an ε-fraction of the clauses. Consider a clause C violated by b. Alice's answer a_C is a satisfying assignment of C (otherwise she loses outright), so $a_C \neq b|_C$, and therefore a_C disagrees with b on at least one of the three variables of C. The referee asks exactly that variable with probability at least $\frac{1}{3}$, and rejects. Hence the rejection probability is at least $\varepsilon \cdot \frac{1}{3}$. □

Combined with Theorem 1.8, Proposition 1.14 shows that it is NP-hard to distinguish $\mathrm{val}(G) = 1$ from $\mathrm{val}(G) \le 1 - \varepsilon_0/3$ for projection games over alphabets of size 7 and 2. We have now met the "game" column of the dictionary.

1.6.2 Parallel repetition

To drive the soundness from $1 - \varepsilon_0/3$ to a small δ we want to repeat. For a one-oracle verifier, as we saw in the proof of Proposition 1.9, independent repetition multiplies the acceptance probabilities. For a game we are not free to repeat sequentially: that would require several rounds of communication, which we want to avoid (otherwise the provers could coordinate). Instead one repeats *in parallel*.

Definition 1.15 (Parallel repetition)**.** The *k-fold parallel repetition* $G^{\otimes k}$ of a game G: the referee draws k independent edges $e_i = (u_i, w_i)$, sends $(u_1, \dots, u_k)$ to Alice and $(w_1, \dots, w_k)$ to Bob, receives $a \in \Sigma_U^k$ and $b \in \Sigma_W^k$, and accepts iff $(a_i, b_i) \in C_{e_i}$ for every i.

Alice and Bob can play each coordinate independently with an optimal strategy of G, so $\mathrm{val}(G^{\otimes k}) \geq \mathrm{val}(G)^k$. Is this an equality? It is not, and the failure is dramatic enough to deserve a warning.

Pitfall: parallel repetition is not independent repetition

Let $\ell \geq 3$ be odd. In the *odd cycle game* the questions are the vertices of an ℓ-cycle. With probability $\frac{1}{2}$ the referee sends the *same* vertex to both provers and demands the same color (0 or 1); with probability $\frac{1}{2}$ it sends the two *endpoints of a random edge*, in random order, and demands different colors. The provers are trying to convince the referee that an odd cycle is 2-colorable. Its value is $1 - \frac{1}{2\ell}$ (Exercise 1.6); this is a Unique Game with two labels. Naively, k repetitions should yield value about $(1 - \frac{1}{2\ell})^k \approx e^{-k/2\ell}$, which for $k = \ell^2/100$ is exponentially small in ℓ. In fact Raz (2011) showed that for $k \leq \ell^2$ there is a strategy winning $G^{\otimes k}$ with probability $1 - O(\sqrt{k}/\ell)$, so that even for $k = c\,\ell^2$ with a small constant c the value of $G^{\otimes k}$ is still at least $\frac{1}{2}$ (this is essentially tight by Feige et al., 2007). Exercise 1.7 gives a much smaller example in which already the 2-fold game beats val^2. Examples of this kind go back to Fortnow and Feige around 1990, and for several years it was a famous open problem whether repetition drives the value to 0 at all.

The qualitative fact that repetition *does* drive the value to 0 is the theorem of Raz.

Theorem 1.16 (Parallel repetition; Raz 1998, Holenstein 2009, Rao 2011)**.** *There is an absolute constant $c > 0$ such that for every two-prover one-round game G with $\mathrm{val}(G) \leq 1 - \eta$ and answer sets of size at most σ,*

$$\mathrm{val}(G^{\otimes k}) \;\leq\; (1 - \eta^c)^{\Omega(k/\log \sigma)}.$$

For projection games, Rao showed $\mathrm{val}(G^{\otimes k}) \leq (1 - \Omega(\eta^2))^k$, independently of σ.

Raz's original paper has $c = 32$ (Raz, 1998); Holenstein simplified the proof and improved this to $c = 3$ (Holenstein, 2009). (We prove none of this until Lecture 5.) Putting the PCP theorem, the clause–variable game and parallel repetition together, we obtain the result that all of Lectures 7–16 take as their starting point.

Theorem 1.17 (Label Cover theorem)**.** *For every $\delta > 0$ there is a $\sigma = \mathrm{poly}(1/\delta)$ and a reduction, running in time $n^{O(\log(1/\delta))}$ on formulas of size n, from 3SAT to projection games with label sets of size at most σ, such that*

$$\varphi \text{ satisfiable} \;\Rightarrow\; \mathrm{val}(G_\varphi) = 1, \qquad \varphi \text{ unsatisfiable} \;\Rightarrow\; \mathrm{val}(G_\varphi) \leq \delta.$$

In other words $\textsc{Gap}_{1,\delta}$-$\textsc{LabelCover}_\sigma$ *is* NP*-hard.*

Proof. Apply Theorem 1.8 to obtain φ' with the ε_0-gap, then form the clause–variable game $G = G_{\varphi'}$; by Proposition 1.14 we have $\mathrm{val}(G) = 1$ or $\mathrm{val}(G) \leq 1 - \eta$ with $\eta = \varepsilon_0/3$.

Take $G^{\otimes k}$ with $k = O(\log(1/\delta)/\eta^2) = O(\log(1/\delta))$. Repetition of a projection game is again a projection game (the projection is $\pi_{e_1} \times \cdots \times \pi_{e_k}$). If $\mathrm{val}(G) = 1$ then $\mathrm{val}(G^{\otimes k}) = 1$. Otherwise Theorem 1.16 gives $\mathrm{val}(G^{\otimes k}) \le (1 - \Omega(\eta^2))^k \le \delta$. The new label sets have sizes 7^k and 2^k, which is $\mathrm{poly}(1/\delta)$, and $G^{\otimes k}$ has at most $(3m)^k$ edges. □

Two remarks about the price we paid. First, the alphabet size σ is polynomial in $1/\delta$, and by Lemma 1.12 it *must* be at least $1/\delta$. Second, the projection games we have produced are d-to-1 with large d: each label of Bob has about $\sigma/2^k$ preimages. Both facts will be the stage on which Unique Games enters.

1.7 Unique Games and the Unique Games Conjecture

1.7.1 Definition and a first observation

We now restrict the projection constraints as much as possible. It is convenient (and customary) to define Unique Games on a general graph.

Definition 1.18 (Unique Games)**.** A *Unique Games instance* $\mathcal{U} = (G, [k], \{\pi_e\})$ consists of a graph $G = (V, E)$, a label set $[k]$, and for every edge $e = (u, v)$ a *bijection* $\pi_e \colon [k] \to [k]$. A labeling $\ell \colon V \to [k]$ *satisfies* e if $\pi_e(\ell(u)) = \ell(v)$, and $\mathrm{val}(\mathcal{U}) = \max_\ell \Pr_{e \sim E}[\ell \text{ satisfies } e]$. Since π_e is a bijection, $\pi_e(\ell(u)) = \ell(v) \iff \ell(u) = \pi_e^{-1}(\ell(v))$, so the orientation of an edge is immaterial.

Example 1.19. Max-Cut is a Unique Game with $k = 2$ and $\pi_e =$ the transposition $0 \leftrightarrow 1$ for every edge. More generally, a system of equations $x_u - x_v = c_e \pmod k$ is a Unique Game with $\pi_e(i) = i + c_e$: two-variable linear equations over $\mathbb{Z}_k$. By contrast, Max-3-Cut ($x_u \neq x_v$ with $k = 3$) is *not* a Unique Game: given x_u, the constraint allows two values of x_v, not one (Exercise 1.8).

The word "unique" means that, given the label of one endpoint, the label of the other endpoint is forced. That makes perfect completeness trivial.

Proposition 1.20 (Unique Games with perfect completeness are easy)**.** *Given a Unique Games instance with k labels, one can decide in time $O(k\,(|V|+|E|))$ whether $\mathrm{val}(\mathcal{U}) = 1$, and find a perfect labeling if there is one.*

Proof. Treat each connected component separately. Pick a root r and try each of the k labels a for it. Propagate along a BFS tree: if u has label $\ell(u)$ and (u, v) is a tree edge then set $\ell(v) = \pi_e(\ell(u))$ (or $\pi_e^{-1}(\ell(u))$ if the edge is traversed against its orientation). Check all edges. If a perfect labeling ℓ^* exists, then with $a = \ell^*(r)$ the propagation reconstructs ℓ^* by induction along the tree, because bijectivity makes each step forced. So some a passes the check. □

Remark. The argument breaks down for projection games, where the label of u determines that of w but not vice versa: propagating from w to u requires choosing among the d preimages, and the branching is exponential. Indeed, deciding val $= 1$ for projection games is NP-hard (Proposition 1.14). Likewise, as in Section 1.3, if a hard Unique Games gap problem exists, it must have completeness $1 - \varepsilon < 1$: an approximately satisfiable instance may have a bad edge on every propagation path, and the instance has to be built so that no local repair works.

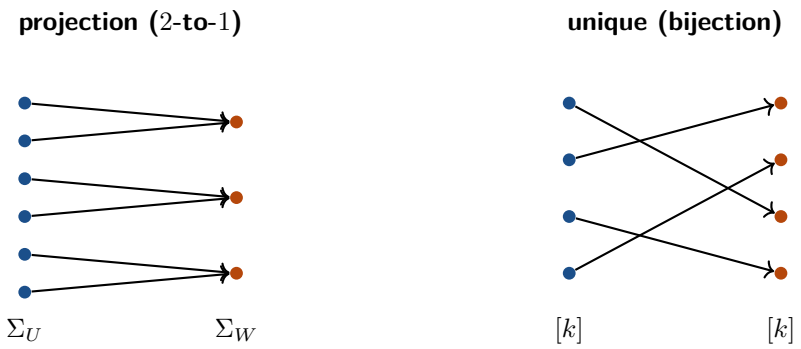


Figure 1.4: A constraint π_e between the label sets of two vertices. In a projection game (left) a label of w has several preimages; in a Unique Game (right) the constraint is a perfect matching.

1.7.2 The conjecture

Conjecture 1.21 (Unique Games Conjecture; Khot 2002)**.** *For every $\varepsilon, \delta > 0$ there is a $k = k(\varepsilon, \delta)$ such that* $\textsc{Gap}_{1-\varepsilon,\delta}$-$\textsc{UG}_k$ *is* NP*-hard: it is* NP*-hard to distinguish Unique Games instances over k labels with* $\mathrm{val} \geq 1 - \varepsilon$ *from those with* $\mathrm{val} \leq \delta$*.*

Every word of this statement carries weight.

(i) *Quantifiers.* First ε and δ are chosen, *then* the alphabet k. The alphabet is a constant but it grows without bound as $\varepsilon, \delta \to 0$. It must: by Lemma 1.12, $k \geq 1/\delta$.

(ii) *A family of problems.* The UGC is not a statement about one problem, but about a family of promise problems parametrised by (ε, δ). (Hence "UG is NP-hard to approximate" is a misleading summary.)

(iii) *Completeness* $1 - \varepsilon$. By Proposition 1.20 we cannot ask for completeness 1.

(iv) *Soundness* $\delta \to 0$. This is the opposite extreme to the PCP theorem, which only gets soundness $1-\varepsilon_0$: the conjecture says that even "almost satisfiable" and "almost completely unsatisfiable" are indistinguishable.

(v) *Reductions.* "NP-hard" means via polynomial-time Karp reductions from 3SAT. Some applications are known to go through under weaker hypotheses, e.g. randomized reductions.

Remark. There are many variants of the conjecture: with bipartite graphs, with linear constraints $\pi_e(i) = i + c_e$ (a special case, believed hard too), with "strong" versions in which soundness is measured on vertex sets rather than edges. Their relationships are surveyed by Khot (2010) and Trevisan (2012). The manuscript discussed in Section 1.8 addresses the usual edge-value form of Conjecture 1.21 with translation constraints on a bipartite graph.

1.7.3 Why Unique Games?

If we can already prove $\textsc{Gap}_{1,\delta}$-$\textsc{LabelCover}$ is NP-hard for every δ (Theorem 1.17), why would anyone want a new conjecture? Because Label Cover is the wrong tool for exactly the problems in the last two rows of Table 1.1.

The two barriers behind Khot's conjecture

(1) The two-query barrier. Håstad's optimal results for Max-E3-SAT and Max-E3-Lin come from PCPs that read *three* bits. A problem such as Max-Cut, Vertex Cover or Min-2SAT-Deletion has constraints on *two* variables, so a PCP proving its hardness must use only *two* queries—and two-query tests are intrinsically much weaker. Label Cover is the natural two-query starting point, with alphabet as long as we please. **(2) The many-to-one barrier.** Roughly speaking, a hardness reduction from Label Cover replaces every vertex by a code (the *long code*) and every constraint π_e by a local test between two codewords; the analysis of a cheating prover must *decode* the code tables back into a labeling of the Label Cover instance. When π_e is d-to-1 with large d, the decoding can only recover the label of w up to a list of d candidates for u, and the loss in soundness is too large to reach the thresholds of Table 1.1. When π_e is a bijection there is no such loss. Khot's insight (Khot, 2002) is that one can hope that a *unique* Label Cover is just as hard as general Label Cover, once completeness is relaxed to $1-\varepsilon$, and that this hypothesis—the UGC—would then imply tight hardness for two-variable problems. (We make all of this precise in Lectures 8 and 9.)

1.7.4 Evidence for and against

Nobody knows whether the UGC is true. The state of the evidence, up to this writing, is a mixture of three things.

- *Algorithms.* If val $\geq 1-\varepsilon$, one can find in polynomial time a labeling of value $1-O(\sqrt{\varepsilon \log k})$ (Charikar et al., 2006), and in time $\exp(n^{\varepsilon^{\Omega(1)}})$ a labeling of value $1-\varepsilon^{\Omega(1)}$ (Arora et al., 2015). No algorithm contradicts the conjecture, but the subexponential algorithm shows that Unique Games is not as hard as 3SAT is believed to be (Lecture 12).
- *Integrality gaps.* The semidefinite program that powers the algorithms above cannot refute the conjecture: it has integrality gaps (Khot and Vishnoi, 2015) (Lecture 10).
- *Partial hardness.* The 2-to-2 Games Conjecture, a weakened version of the UGC in which each constraint is 2-to-2 instead of a bijection, is a *theorem* (Khot et al., 2017; Dinur et al., 2018; Khot et al., 2018); see Khot (2019) for an exposition. It already implies NP-hardness of approximating Vertex Cover within $\sqrt{2}-\varepsilon$, the best-known (Lectures 13–15).

1.8 The map of the territory

Figure 1.5 places the main results of this course on a single page; Table 1.3 gives the chronology. Read an arrow as "is used to prove" (or "builds on"). One item on the map needs a health warning.

Status note: the 2026 claim

On 23 September 2026 OpenAI released a manuscript *The Unique Games Theorem* (OpenAI, 2026). Its abstract reads: "We prove the Unique Games Conjecture. For every fixed $\varepsilon, \delta \in (0, 1/2)$, we give a deterministic polynomial-time reduction from 3SAT to Unique Games over a fixed finite alphabet, with completeness at least $1-\varepsilon$ and soundness at most δ."

Status. At the time of writing we know of no independent expert verification of the proof, and no refutation; we have not verified it ourselves. Conjecture 1.21 is

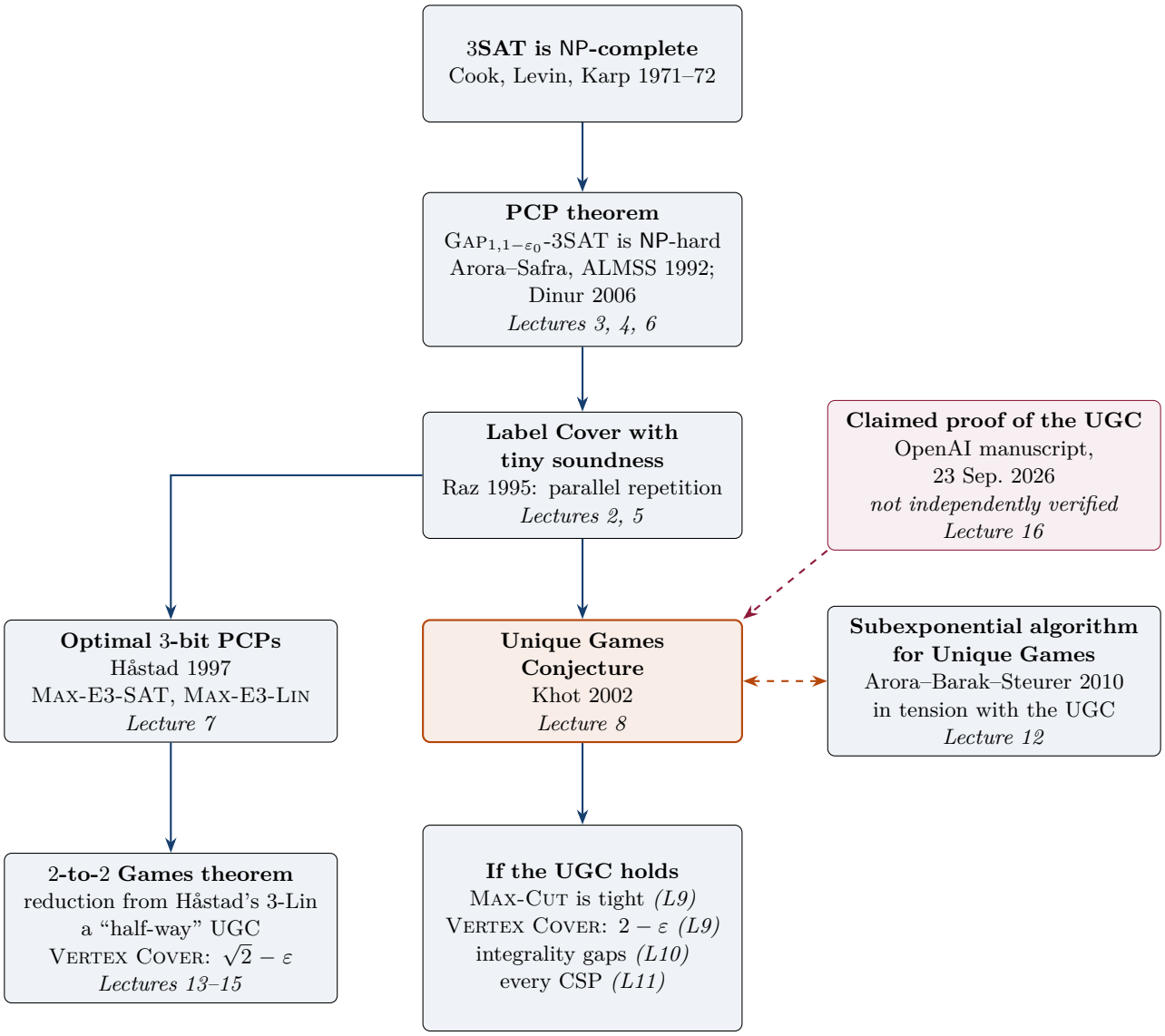


Figure 1.5: The route from NP-completeness to Unique Games. Solid blue arrows are established implications. The orange double arrow records a *tension*: a subexponential algorithm for Unique Games sits uneasily next to a conjecture asserting NP-hardness. The dashed wine-red arrow marks a claim whose status is explained in the box below. *Ln* abbreviates "Lecture n".

therefore treated as open, and every statement labelled "assuming the UGC" in these notes is *conditional*. Lecture 16 compares the manuscript's main theorem with the classical formulations and discusses what a careful verification would have to check. Until then we keep apart results established by the community, results in a complete but unrefereed paper, and claims published by their authors only.

Year	Event
1971–72	Cook, Levin, Karp: NP-completeness (Cook, 1971; Levin, 1973; Karp, 1972)
1974	Johnson: systematic study of approximation algorithms (Johnson, 1974)
1988	Ben-Or, Goldwasser, Kilian, Wigderson: multi-prover interactive proofs (Ben-Or et al., 1988)
1990	Babai, Fortnow, Lund: NEXP = MIP (Babai et al., 1991b)
1991	Feige, Goldwasser, Lovász, Safra, Szegedy: PCPs imply hardness of Clique (Feige et al., 1996)
1992	Arora–Safra and Arora–Lund–Motwani–Sudan–Szegedy: the PCP theorem (Arora and Safra, 1998; Arora et al., 1998); Feige–Lovász: two-prover one-round proof systems (Feige and Lovász, 1992)
1994	Goemans–Williamson: 0.878-approximation for Max-Cut (Goemans and Williamson, 1995)
1995	Raz: parallel repetition (Raz, 1998); Bellare–Goldreich–Sudan: the long code (Bellare et al., 1998)
1996	Håstad: Clique is hard within $n^{1-\varepsilon}$ (Håstad, 1999)
1997	Håstad: optimal inapproximability for Max-E3-SAT and Max-E3-Lin (Håstad, 2001)
2002	Khot: the Unique Games Conjecture (Khot, 2002)
2003	Khot–Regev: Vertex Cover is hard within $2-\varepsilon$ under the UGC (Khot and Regev, 2008)
2004–05	Khot–Kindler–Mossel–O'Donnell: Max-Cut (Khot et al., 2007); Khot–Vishnoi: integrality gaps (Khot and Vishnoi, 2015)
2006	Dinur: the PCP theorem by gap amplification (Dinur, 2007)
2008	Raghavendra: every CSP (Raghavendra, 2008)
2010	Arora–Barak–Steurer: subexponential algorithm for Unique Games (Arora et al., 2015)
2017–18	Khot–Minzer–Safra, Dinur–Khot–Kindler–Minzer–Safra: 2-to-2 Games (Khot et al., 2017; Dinur et al., 2018; Khot et al., 2018)
2026	OpenAI manuscript claiming the UGC (OpenAI, 2026)

Table 1.3: A timeline of the main results (conference years; journal versions are later). Many important contributions are missing.

Exercises

Exercise 1.1 ([⋆])**.** Derandomize the three random-assignment algorithms of Example 1.2 with the method of conditional expectations: fix the variables one at a time,

always choosing the value that does not decrease the conditional expected number of satisfied constraints. Why is this efficiently computable?

Exercise 1.2 ([⋆]). State and prove the analogue of Lemma 1.4 for minimization problems, where YES instances have OPT $\le c$ and NO instances have OPT $> s$ with $c < s$. What is the inapproximability ratio?

Exercise 1.3 ([⋆]). Show that $\textsc{Gap}_{1,s}$-Max-E3-Lin is in P for every $s < 1$. (Hence Håstad's theorem cannot have perfect completeness.) Why does the same argument not apply to $\textsc{Gap}_{1,\frac{7}{8}+\varepsilon}$-Max-E3-SAT?

Exercise 1.4 ([⋆⋆]). In Definition 1.6 we required the queries to be non-adaptive. Show that adaptivity buys nothing in the constant-query regime: a verifier that makes q *adaptive* queries (each position may depend on the previous answers) can be simulated by a non-adaptive verifier that makes $2^q - 1$ queries, with the same randomness, completeness and soundness. Conclude that $\mathsf{NP} = \mathsf{PCP}[O(\log n), O(1)]$ holds with or without adaptivity.

Exercise 1.5 ([⋆]). Let V be a one-oracle verifier with soundness s. Prove that its k-fold independent repetition has soundness s^k. Identify exactly where the argument uses that there is one fixed proof π, and explain why the same argument breaks for parallel repetition of two-prover games.

Exercise 1.6 ([⋆⋆]). Prove that the odd cycle game has value exactly $1 - \frac{1}{2\ell}$. *Hint: for the lower bound, let both provers color the cycle with a single monochromatic edge. For the upper bound, write the strategies as colorings f_A, f_B of the vertices and build a graph on the 2ℓ vertices $(v, A), (v, B)$ with "$\neq$" edges $(u, A)-(v, B)$ for every oriented edge (u, v) of the cycle, and "$=$" chords $(v, A)-(v, B)$. The "$\neq$" edges form a single even cycle of length 2ℓ, and the two endpoints of any chord are at odd distance ℓ along it. So each chord together with either arc of length ℓ forms a cycle (2ℓ of them in total) that no coloring can satisfy completely. Every "$\neq$" edge has probability $\frac{1}{4\ell}$ and lies in ℓ of these cycles, every chord has probability $\frac{1}{2\ell}$ and lies in 2 of them; give each cycle weight $\frac{1}{4\ell^2}$.*

Exercise 1.7 ([⋆⋆]). *Repetition can beat the product.* Consider the following game G. Alice and Bob receive independent uniform bits x and y and answer with bits a and b. They win iff $(x, y) = (0, 0)$ and $a \neq b$, or $(x, y) \in \{(0, 1), (1, 0)\}$ and $a = b = 0$. (For $(x, y) = (1, 1)$ they always lose.) Show that $\mathrm{val}(G) = \frac{1}{2}$ but $\mathrm{val}(G^{\otimes 2}) \ge \frac{3}{8} > \mathrm{val}(G)^2 = \frac{1}{4}$. *Hint: in $G^{\otimes 2}$ let both players use the same strategy, answering the pair (a_1, a_2) as a function of their input pair: $00 \mapsto 00$, $01 \mapsto 10$, $10 \mapsto 01$, $11 \mapsto 00$.*

Exercise 1.8 ([⋆]). (a) Prove that Max-3-Cut is not a Unique Game. (b) Prove that a Unique Game with val $= 1$ and a Label Cover instance with val $= 1$ can differ in complexity by explaining, using results of this lecture, why the first is in P and the second is NP-hard.

Exercise 1.9 ([⋆⋆]). Which of the following statements implies the UGC, which is implied by it, and which is neither? (a) For every $\varepsilon, \delta > 0$ and *every* $k \ge k_0(\varepsilon, \delta)$, $\textsc{Gap}_{1-\varepsilon,\delta}$-$\mathrm{UG}_k$ is NP-hard. (b) There exist $\varepsilon, \delta > 0$ such that for all k, $\textsc{Gap}_{1-\varepsilon,\delta}$-$\mathrm{UG}_k$ is NP-hard. (c) For every ε there is $\delta > 0$ and k such that $\textsc{Gap}_{1-\varepsilon,\delta}$-$\mathrm{UG}_k$ is NP-hard.

Exercise 1.10 ([⋆⋆])**.** *Reading.* Read the definition of Unique Games in Section 2 of Khot (2010) and the statement of Theorem 1.1 in OpenAI (2026). List all differences in the formulation (type of graph, type of constraint, alphabet, role of determinism, order of quantifiers). For each difference, decide whether it is a cosmetic restatement, a strengthening, or a weakening of the classical conjecture.

Notes and further reading

Where this lecture comes from. The equivalence of the two PCP formulations is standard; see Lecture 1 of Harsha (2007), or Lecture 1 and Lecture 3 of Trevisan (2006), and Chapter 11 of Arora and Barak (2009). That the formulas produced by Cook's theorem are "fragile"—they are almost satisfiable even for NO instances, so a gap cannot be created by such a reduction—is the starting point of Trevisan's first lecture; the gap-problem formalism of Section 1.3 follows Harsha (2007). The DIMACS tutorial notes (Harsha and Charikar, 2010) are an excellent companion for Lectures 1–9. For the Unique Games Conjecture, the survey of Khot (2010) and the Bulletin article of Trevisan (2012) are the primary references.

On attributions. The $n^{1-\varepsilon}$ hardness of Max-Clique is due to Håstad (1999), and was proved under the assumption $\mathsf{NP} \not\subseteq \mathsf{ZPP}$; the assumption was removed by Zuckerman (2007). It is a different paper from the one containing the $\frac{7}{8}$ and $\frac{1}{2}$ results (Håstad, 2001). The first examples of games whose parallel repetition does not behave like a product were due to Fortnow and to Feige; the odd cycle game and its tilings of space are studied by Feige et al. (2007) and Raz (2011).

What is not covered. We have not defined the *long code*, nor discussed the SDP relaxations of Unique Games, nor the connection between the UGC and the *Small-Set Expansion* problem. They enter in later lectures. Lecture 2 develops the formalism of games and Label Cover that we have only sketched.

LECTURE 2

Games, Label Cover, and the Language of PCPs

One object with four names, and the price of every translation

Lecture 1 introduced four characters: a PCP verifier, a constraint satisfaction problem, a two-prover game, and a Label Cover instance. Table 1.2 asserted that they are one and the same object seen from four sides. That is true, but only up to constants, and in hardness of approximation constants are exactly what we cannot afford to lose track of: the difference between soundness $1-\varepsilon_0$ and soundness $1-\varepsilon_0/3$ is invisible to the PCP theorem and decisive for a reduction that must end with soundness $\frac{1}{2}+\varepsilon$.

This lecture turns every arrow of the dictionary into a theorem and records what it costs. Three prices will appear: a factor 2 in value when we pass from one oracle to two provers (Section 2.2), a factor q in the gap when we pass from q-ary to binary constraints (Section 2.3), and a blow-up of the alphabet. Once the prices are known, statements such as "Label Cover with soundness δ over an alphabet of size $\mathrm{poly}(1/\delta)$" or "2-to-2 Games over $\mathbb{F}_2^{\ell}$" can be read at a glance. We end with the oldest customer of this language, the reduction of Feige et al. (1996) from PCPs to Max-Clique. Its proof fits on one page and contains the idea behind every later reduction in this course: *turn the consistency of local views of a proof into the structure of a graph.*

After this lecture you should be able to...

(1) write any PCP verifier, CSP, game or Label Cover instance as a constraint graph, and compute the value of a small one by hand (Sections 2.1, 2.3);

(2) say which constraint types—general, projection, 2-to-2, unique—a given instance has, and what each type buys and costs (Section 2.4);

(3) convert between a one-oracle and a two-prover view of the same instance, and state exactly how much value is lost (Proposition 2.8);

(4) prove that the clause–variable game of any q-ary CSP has soundness $1-\varepsilon/q$ (Theorem 2.12);

(5) build the FGLSS graph of a verifier and prove that its clique number is 2^r times the best acceptance probability (Theorem 2.19).

2.1 Constraint graphs: one language for everything

We begin with the most general object and specialize.

Definition 2.1 (Constraint satisfaction instance)**.** An instance Φ of a *constraint satisfaction problem* consists of a finite set X of *variables*, a finite alphabet Σ_x for every $x \in X$, and a finite multiset $\mathcal{C}$ of *constraints*. A constraint $c = (S_c, P_c)$ has a *scope* $S_c \subseteq X$ and a *predicate* $P_c \colon \prod_{x\in S_c} \Sigma_x \to \{0,1\}$. The *arity* of Φ is $q = \max_c |S_c|$. An

Problem	Alphabet	Arity	Predicate	Random assignment
Max-E3-SAT	$\{0,1\}$	3	OR of three literals	$\frac{7}{8}$
Max-E3-Lin($\mathbb{F}_2$)	$\{0,1\}$	3	$x+y+z=b$	$\frac{1}{2}$
Max-Cut	$\{0,1\}$	2	$x \neq y$	$\frac{1}{2}$
Max-3-Cut	$\{0,1,2\}$	2	$x \neq y$	$\frac{2}{3}$
Linear Unique Games	$\mathbb{Z}_k$	2	$x-y=c$	$\frac{1}{k}$
Label Cover (projection)	Σ_U, Σ_W	2	$b=\pi(a)$	$\frac{1}{\lvert\Sigma_W\rvert}$
2-to-2 Games	$[k]$	2	$\pi(a)=\pi'(b)$	$\frac{2}{k}$

Table 2.1: A zoo of constraint satisfaction problems. The last column is the fraction of constraints satisfied by a *uniformly random assignment*; it is the "trivial" threshold that approximation algorithms must beat.

assignment σ gives every variable x a value $\sigma(x) \in \Sigma_x$, and

$$\mathrm{val}(\Phi) \;=\; \max_{\sigma} \Pr_{c\sim\mathcal{C}}\big[P_c(\sigma|_{S_c}) = 1\big].$$

(Lecture 1 wrote OPT for the optimum of a Max-CSP; from now on we write val, which is also the name for the value of a game.) The problems of Lecture 1 all fit.

The last column of Table 2.1 is not a coincidence of the examples. It is an expectation.

Lemma 2.2 (Random assignment)**.** *Let the* density *of a constraint* c *be* $\delta(c) = |P_c^{-1}(1)|/\prod_{x\in S_c}|\Sigma_x|$. *Then* $\mathrm{val}(\Phi) \geq \mathbb{E}_{c\sim\mathcal{C}}\,\delta(c)$.

Proof. For a uniformly random assignment σ and a fixed constraint c, the restriction $\sigma|_{S_c}$ is uniform on $\prod_{x\in S_c}\Sigma_x$, so c is satisfied with probability $\delta(c)$. By linearity of expectation the expected fraction of satisfied constraints is $\mathbb{E}_c\,\delta(c)$, and some assignment does at least as well. □

This is the baseline of the scoreboard in Table 1.1: the first two rows say that for Max-E3-SAT and Max-E3-Lin nothing efficient beats the density of the predicate, unless $\mathsf{P} = \mathsf{NP}$.

2.1.1 Binary constraints: constraint graphs

From here on the arity is almost always 2.

Definition 2.3 (Constraint graph)**.** A *constraint graph* is a CSP instance of arity 2. Concretely it is a finite multigraph $G = (V, E)$, alphabets Σ_v for $v \in V$, and, for every edge, oriented arbitrarily as $e = (u, v)$, a *relation* $C_e \subseteq \Sigma_u \times \Sigma_v$. A *labeling* ℓ gives each vertex v a label $\ell(v) \in \Sigma_v$; it *satisfies* $e = (u, v)$ if $(\ell(u), \ell(v)) \in C_e$, and

$$\mathrm{val}(G) \;=\; \max_{\ell} \Pr_{e=(u,v)\sim E}\big[(\ell(u),\ell(v)) \in C_e\big].$$

If G is bipartite, $G = (U, W, E)$ with every edge oriented from U to W, we call it a *game*; this is Definition 1.10, and $\mathrm{val}(G)$ is the value of the game.

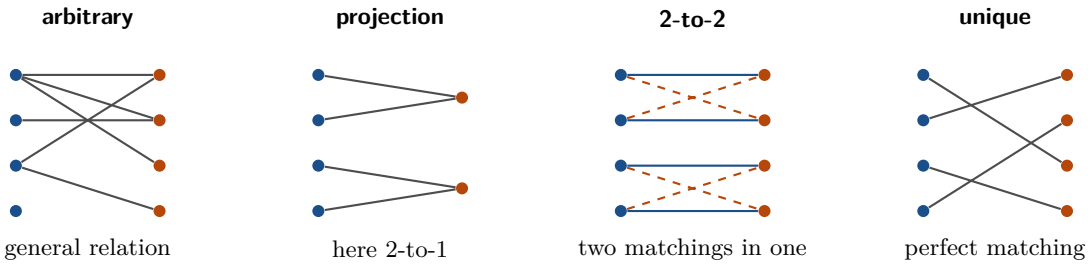


Figure 2.1: The four constraint types of Definition 2.4 as bipartite graphs between Σ_u (left, blue) and Σ_v (right, orange). In a unique constraint each label determines its partner *and* vice versa; in a projection, only one direction is determined; in a 2-to-2 constraint each label leaves two candidates. The 2-to-2 relation of panel (c) is the union of the solid and the dashed perfect matching (Lemma 2.15).

When an edge $e = (u, v)$ carries a projection we write $\pi_{uv} \colon \Sigma_u \to \Sigma_v$ so that $C_e = \{(a, \pi_{uv}(a))\}$, and the value reads $\mathrm{val}(G) = \max_\ell \Pr_{(u,v)\sim E}[\pi_{uv}(\ell(u)) = \ell(v)]$, the formula we shall use throughout. Unique Games (Definition 1.18) are the case where every π_{uv} is a bijection of $[k]$.

The word "Label Cover" is attached to this optimization problem—find a labeling of maximum value—and usually, though not always, refers to projection constraints. We shall say *Label Cover* for the general problem and specify the constraint type when it matters.

Definition 2.4 (Types of constraints)**.** Let $C \subseteq \Sigma \times \Sigma'$ be the relation on an edge (u, v), with $\Sigma = \Sigma_u$ and $\Sigma' = \Sigma_v$.

- C is a *projection* if $C = \{(a, \pi(a))\}$ for a function $\pi \colon \Sigma \to \Sigma'$: every left label has exactly one partner. It is *d-to-1* if moreover every fibre $\pi^{-1}(b)$ has exactly d elements.
- C is *unique* if it is the graph of a bijection, i.e. a perfect matching between Σ and Σ'.
- C is *2-to-2* if there are a set Γ and two 2-to-1 maps $\pi \colon \Sigma \to \Gamma$, $\pi' \colon \Sigma' \to \Gamma$ with $C = \{(a, b) : \pi(a) = \pi'(b)\}$. Every label then has exactly two partners, and the bipartite graph of C is a disjoint union of copies of $K_{2,2}$.

An instance *is* of a given type if all its constraints are.

Lemma 2.2 immediately quantifies how many labels a type forces on us.

Corollary 2.5 (Alphabets cannot be small)**.** *For a projection game,* $\mathrm{val}(G) \geq 1/|\Sigma_W|$*; for a Unique Game with k labels,* $\mathrm{val}(G) \geq 1/k$*; for a 2-to-2 game with k labels on both sides,* $\mathrm{val}(G) \geq 2/k$*. Hence a hardness result with soundness δ needs alphabets of size at least $1/\delta$, $1/\delta$ and $2/\delta$, respectively.*

Proof. A projection has $|C| = |\Sigma|$ pairs, hence density $|\Sigma|/(|\Sigma||\Sigma'|) = 1/|\Sigma'|$; a 2-to-2 constraint has $|C| = 2|\Sigma|$ pairs and, when $|\Sigma| = |\Sigma'| = k$, density $2/k$. Apply Lemma 2.2. □

Example 2.6 (Unique Games on a cycle)**.** Let G be a cycle on vertices $0, 1, \ldots, \ell-1$ with the linear constraints $x_{i+1} - x_i = c_i \pmod k$ on the edges $(i, i+1)$ (indices modulo ℓ).

If $\sum_i c_i \equiv 0$, set $x_0 = 0$ and propagate: all ℓ equations hold. Otherwise summing all the equations would give $0 \equiv \sum_i c_i \not\equiv 0$, so at least one equation fails; but deleting one edge leaves a path, which can always be satisfied. Hence $\mathrm{val}(G) = 1 - \frac{1}{\ell}$. *A cycle is satisfiable iff the constraints compose to the identity around it*; this is the whole obstruction to perfect completeness in a Unique Game, and checking it along a spanning tree is what the algorithm of Proposition 1.20 does.

2.2 One oracle or two provers?

A PCP verifier that makes two queries reads two positions of *one* proof. In a game the referee sends two questions to *two* provers who answer independently. A constraint graph on a general graph is the first, a game the second. The two differ in one respect: a single proof is automatically consistent with itself, whereas two provers may give contradictory answers to the same question.

Definition 2.7 (Double cover)**.** Let $G = (V, E)$ be a constraint graph with relations C_e. Its *double cover* $G^{\times 2}$ is the game with left vertices v_A (Alice's questions) and right vertices v_B (Bob's questions), one pair (v_A, v_B) for every $v \in V$, both with alphabet Σ_v, and for every edge $e = (u, v)$ of G the two edges (u_A, v_B) with relation C_e and (v_A, u_B) with the transposed relation $C_e^{\top} = \{(b, a) : (a, b) \in C_e\}$. In words: the referee picks an edge of G and a random orientation of it, and asks the two endpoints to the two provers.

Proposition 2.8 (Value of the double cover)**.** *For every constraint graph G,*

$$\mathrm{val}(G) \;\le\; \mathrm{val}(G^{\times 2}) \;\le\; 2\,\mathrm{val}(G),$$

and both inequalities can be tight.

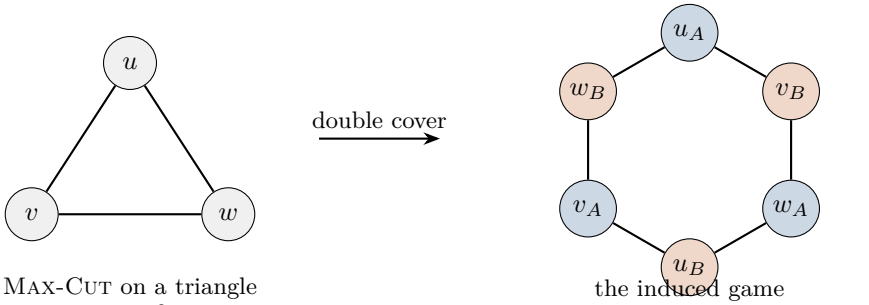
Proof. For the first inequality let ℓ be an optimal labeling of G and let Alice and Bob both use it: $\ell_A(v_A) = \ell_B(v_B) = \ell(v)$. Whatever orientation the referee picks, the two answers are the labels of the two endpoints, so the acceptance probability is $\mathrm{val}(G)$.

For the second, let (ℓ_A, ℓ_B) be an optimal strategy in $G^{\times 2}$ with value p. Define a labeling of G at random: for every vertex v, independently and uniformly pick a side $s(v) \in \{A, B\}$ and set $\ell(v) = \ell_{s(v)}(v)$. Fix an edge $e = (u, v)$. With probability $\frac{1}{4}$ we have $s(u) = A$ and $s(v) = B$; then ℓ satisfies e iff the edge (u_A, v_B) of $G^{\times 2}$ is satisfied by (ℓ_A, ℓ_B). With probability $\frac{1}{4}$ we have $s(v) = A$ and $s(u) = B$, and ℓ satisfies e iff the edge (v_A, u_B) is satisfied. These two events are disjoint, so

$$\Pr[\ell \text{ satisfies } e] \;\ge\; \tfrac{1}{4}\big(\mathbf{1}[(u_A, v_B) \text{ sat.}] + \mathbf{1}[(v_A, u_B) \text{ sat.}]\big).$$

Averaging over a uniform edge e of G, the right-hand side has expectation $\frac{1}{4} \cdot 2p = p/2$, since the $2|E|$ edges of $G^{\times 2}$ are the two orientations of the edges of G. Hence some labeling has value at least $p/2$, i.e. $\mathrm{val}(G) \ge \mathrm{val}(G^{\times 2})/2$.

For tightness of the first inequality take any satisfiable G. For the second take Max-Cut on the complete graph K_n: the best cut contains a $\frac{\lceil n/2 \rceil \lfloor n/2 \rfloor}{\binom{n}{2}} \to \frac{1}{2}$ fraction of the edges, but in the double cover Alice answers 0 everywhere and Bob 1 everywhere, and all constraints $a \neq b$ hold (Figure 2.2). □



MAX-CUT on a triangle
val $= \frac{2}{3}$

the induced game
val $= 1$: Alice says 0, Bob says 1

Figure 2.2: The double cover of a triangle is a hexagon, which is bipartite. Self-consistency of a single labeling is what the triangle tests; the two provers of the hexagon need not be consistent with each other, and they win with probability 1.

Remark. The factor 2 is harmless when the soundness we care about is small: a hardness result for constraint graphs with completeness c and soundness s gives one for games with completeness c and soundness $2s$. It is fatal when $s \geq \frac{1}{2}$. This is why the PCP theorem is converted into a game by the clause–variable construction of Section 2.3 (soundness $1 - \varepsilon/q$) and *not* by the double cover (soundness $2(1 - \varepsilon)$, which exceeds 1 and says nothing as soon as $\varepsilon < \frac{1}{2}$). For fixed orientations the loss is a factor 4 instead of 2, which is the bound quoted in Khot (2010).

A game is conversely a perfectly good PCP.

Lemma 2.9 (Answer tables)**.** *Let $G = (U, W, E)$ be a game with alphabets Σ_U, Σ_W of size at most σ. Consider the verifier V_G whose proof is a pair of tables $A\colon U \to \Sigma_U$ and $B\colon W \to \Sigma_W$, that picks a uniformly random edge (u, w), reads $A(u)$ and $B(w)$, and accepts iff $(A(u), B(w)) \in C_{(u,w)}$. Then $\max_{A,B} \Pr[V_G \text{ accepts}] = \mathrm{val}(G)$. If proofs must be binary strings, each label is written with $\lceil \log_2 \sigma \rceil$ bits, and the verifier makes $2\lceil \log_2 \sigma \rceil$ binary queries, uses $\log_2 |E|$ random bits and has the same completeness and soundness.*

Proof. A proof (A, B) is exactly a labeling of G, and the acceptance probability is the fraction of satisfied edges. □

So $\text{GAP}_{1,\delta}$-LABELCOVER_σ is a PCP with $O(\log \sigma)$ queries and soundness δ. Is that better than what we could already do? By Lemma 2.9 and Theorem 1.17 the parameters are $O(\log(1/\delta))$ queries and soundness δ with $O(\log n)$ random bits—and independent repetition of the plain verifier of Proposition 1.9 achieves the very same trade-off. The point of games is not their parameters but their *structure*: the 2 queries come in two groups, and the answer in one group is a function of the answer in the other. This is the structure that the long-code tests of Lectures 7–11 exploit.

Proposition 2.10 (Two binary queries are not enough)**.** *If a language L has a PCP verifier with $r = O(\log n)$ random bits,* two *binary queries, perfect completeness and soundness $s < 1$, then $L \in \mathsf{P}$. Hence, unless $\mathsf{P} = \mathsf{NP}$, a two-query PCP for NP with perfect completeness needs alphabets of growing size.*

Proof. On input x enumerate the $2^r = \mathrm{poly}(n)$ random strings ρ. Each defines a predicate P_ρ on two Boolean proof bits, and every Boolean predicate on two variables is a 2-CNF (forbid each rejected pair of values by one clause of width 2). So the verifier defines a

2-CNF ψ_x of polynomial size. If $x \in L$ the honest proof satisfies ψ_x; if $x \notin L$ every proof is rejected for some ρ, so ψ_x is unsatisfiable. 2SAT is in P. □

2.3 From verifiers and CSPs to games

Lecture 1 turned the clauses of a 3-CNF into a game by asking one prover about a clause and the other about a variable of it. The construction works for every CSP, and its analysis is the same one-paragraph argument.

Definition 2.11 (Constraint–variable game)**.** Let Φ be a CSP instance of arity q. The *constraint–variable game* G_Φ is the following game. The referee draws a uniformly random constraint c and a uniformly random variable $x \in S_c$, and sends c to Alice and x to Bob. Alice answers an assignment $a \in \prod_{y \in S_c} \Sigma_y$ to the scope of c, Bob answers a symbol $b \in \Sigma_x$, and they win iff $P_c(a) = 1$ and $a_x = b$.

(The edge (c, x) has weight $\frac{1}{|\mathcal{C}||S_c|}$; weights are harmless since we may repeat edges. If every predicate is satisfiable we restrict Alice's alphabet to the satisfying assignments of c, which changes nothing since a non-satisfying answer loses outright. Then G_Φ is a *projection game*: $\pi_{(c,x)}(a) = a_x$, with $|\Sigma_U| \le \max_x |\Sigma_x|^q$ and $\Sigma_W = \Sigma_x$.)

Theorem 2.12 (Soundness of the constraint–variable game)**.** *For every CSP instance Φ of arity q,*

$$\mathrm{val}(\Phi) \;\le\; \mathrm{val}(G_\Phi) \;\le\; 1 - \frac{1 - \mathrm{val}(\Phi)}{q}.$$

In particular $\mathrm{val}(\Phi) = 1$ *iff* $\mathrm{val}(G_\Phi) = 1$*, and if every assignment violates an ε-fraction of the constraints, then* $\mathrm{val}(G_\Phi) \le 1 - \varepsilon/q$.

Proof. Lower bound. Let σ be an optimal assignment of Φ. Bob answers $\sigma(x)$; Alice, for a constraint c that σ satisfies, answers $\sigma|_{S_c}$, and for a constraint it violates, anything. On every constraint satisfied by σ the provers win for every $x \in S_c$, hence the winning probability is at least $\mathrm{val}(\Phi)$.

Upper bound. Fix deterministic strategies. Bob's answers define an assignment b of Φ with $b(x) = \ell_W(x)$. Let c be a constraint violated by b, and let $a = \ell_U(c)$ be Alice's answer. If $P_c(a) = 0$ the provers lose on every edge at c. Otherwise $a \neq b|_{S_c}$ (because $P_c(b|_{S_c}) = 0$), so there is some $x \in S_c$ with $a_x \neq b(x)$, and the referee asks that x with probability at least $1/|S_c| \ge 1/q$. So the provers lose at c with probability at least $1/q$. Since b violates at least a $1 - \mathrm{val}(\Phi)$ fraction of the constraints, the total losing probability is at least $(1 - \mathrm{val}(\Phi))/q$. □

This is Proposition 1.14 for $q = 3$ and a CNF. The bound is attained, e.g. for the formula below.

Example 2.13 (A formula and its game)**.** Let φ_0 have the four clauses $C_1 = (x_1 \vee x_2)$, $C_2 = (\neg x_1 \vee x_2)$, $C_3 = (\neg x_2 \vee x_3)$, $C_4 = (\neg x_2 \vee \neg x_3)$. If $x_2 = 1$ then C_1, C_2 hold and exactly one of C_3, C_4 does; if $x_2 = 0$ then C_3, C_4 hold and exactly one of C_1, C_2 does. So *every* assignment satisfies exactly three of the four clauses: $\mathrm{val}(\varphi_0) = \frac{3}{4}$. Theorem 2.12 with $q = 2$ gives $\mathrm{val}(G_{\varphi_0}) \le 1 - \frac{1/4}{2} = \frac{7}{8}$. This is tight: let Bob answer any assignment b, say $b = (0, 0, 0)$, which violates C_1 only; Alice answers $b|_{S_c}$ on C_2, C_3, C_4 and $(x_1, x_2) = (1, 0)$ on C_1, a satisfying assignment that still agrees with b on x_2. The provers lose only if the referee picks C_1 and then the variable x_1, an event of probability $\frac{1}{4} \cdot \frac{1}{2} = \frac{1}{8}$.

Translation	Effect on the value	Price
verifier $\to$ CSP	$\mathrm{val}(\Phi_V) = \max_\pi \Pr[V^\pi \text{ acc.}]$ exactly	none; 2^r constraints
CSP $\to$ verifier	the same, exactly	none (pick a random constraint)
constraint graph $\to$ game (double cover, Prop. 2.8)	$\mathrm{val}(G) \le \mathrm{val}(G^{\times 2}) \le 2\,\mathrm{val}(G)$	soundness at most doubles
game $\to$ verifier (answer tables, Lemma 2.9)	exactly the value of the game	$2\lceil \log \sigma \rceil$ binary queries
q-ary CSP $\to$ projection game (Thm. 2.12)	$1 - \frac{1-\mathrm{val}(\Phi)}{q} \ge \mathrm{val}(G_\Phi) \ge \mathrm{val}(\Phi)$	gap shrinks by a factor q; alphabet $\lvert\Sigma\rvert^q$
verifier $\to$ clique (Thm. 2.19)	$\omega(H_x) = 2^r \max_\pi \Pr[V^\pi \text{ acc.}]$	2^{r+q} vertices

Table 2.2: The price of every translation in the dictionary of Table 1.2.

Corollary 2.14 (Every verifier is a game)**.** *Let V be a PCP verifier with r random bits and q queries, perfect completeness and soundness s on a language L. Let $\Phi_V(x)$ be the CSP with one constraint per random string ρ (scope: the positions queried at ρ, predicate: $P_{x,\rho}$). Then the game $G_{\Phi_V(x)}$, in which Alice is asked a random string ρ and Bob a position queried at ρ, is a projection game with $\lvert\Sigma_U\rvert \le 2^q$, $\lvert\Sigma_W\rvert = 2$, and*

$$x \in L \;\Rightarrow\; \mathrm{val} = 1, \qquad x \notin L \;\Rightarrow\; \mathrm{val} \le 1 - \frac{1-s}{q}.$$

Proof. $\mathrm{val}(\Phi_V(x))$ is the best acceptance probability over proofs, which is 1 or at most s; apply Theorem 2.12. □

Combined with $\mathsf{NP} = \mathsf{PCP}[O(\log n), O(1)]$, Corollary 2.14 says that it is NP-hard to distinguish projection games of value 1 from games of value at most $1-\varepsilon_1$, for a constant $\varepsilon_1 > 0$: the PCP theorem in the language of games, stated without mentioning 3SAT.

Table 2.2 collects the translations between the four columns of the dictionary and what each costs.

2.4 The ladder of constraint types

Why are we so careful about the type of a constraint? Because it determines both what an algorithm can do and what a hardness reduction needs. We saw in Proposition 1.20 that the *unique* type makes perfect completeness easy: a label on one vertex forces the label on its neighbours in both directions, so a satisfying labeling is found by propagation along a spanning tree. For a projection the forcing goes in one direction only, and for a general relation in neither. A 2-to-2 constraint sits exactly between: after one step of propagation *two* candidates remain, after t steps up to 2^t.

This is the right way to read the hierarchy of conjectures and theorems that organizes the rest of the course. Every row of Table 2.3 is a statement about the complexity of a gap problem for constraint graphs of this type.

Constraints	Gap problem	Status	Main use
projection (large alphabet)	$\textsc{Gap}_{1,\delta}$-LabelCover, every $\delta > 0$	theorem (Thm. 1.17)	Håstad's tight 3-bit results (Lecture 7)
2-to-2	$\textsc{Gap}_{1-\varepsilon,\varepsilon}$-2-to-$2_k$ for some $k = k(\varepsilon)$	theorem (Khot et al., 2017; Dinur et al., 2018; Khot et al., 2018)	Vertex Cover within $\sqrt{2} - \varepsilon$ (Lectures 13–15)
unique	$\textsc{Gap}_{1-\varepsilon,\delta}$-UG$_k$ for some $k = k(\varepsilon, \delta)$	conjecture (Khot, 2002); claimed in (OpenAI, 2026), not verified	tight bounds for Max-Cut, Vertex Cover, every CSP (Lectures 8–11)
unique, completeness 1	$\textsc{Gap}_{1,\delta}$-UG	in P (Prop. 1.20)	—

Table 2.3: The ladder of constraint types. Moving down the table the constraints become more rigid and the hardness results become stronger *per label*, but harder to prove.

The 2-to-2 theorem is called "half-way" to the UGC for a reason that can be proved in a few lines, and the proof is a good test of the language we have built.

Lemma 2.15 (A 2-to-2 relation is two bijections)**.** *Let $C \subseteq \Sigma \times \Sigma'$ be a relation in which every label of Σ and every label of Σ' has exactly two partners; in particular C may be a 2-to-2 constraint in the sense of Definition 2.4. Then C is the disjoint union $M \sqcup M'$ of two perfect matchings, i.e. of the graphs of two bijections.*

Proof. The bipartite graph $(\Sigma \sqcup \Sigma', C)$ is simple and 2-regular, hence a disjoint union of cycles, each of even length at least 4. Colour the edges of each cycle alternately red and blue. Every vertex lies on exactly one red and one blue edge, so the red edges form a perfect matching M and the blue edges another one, M'. □

Corollary 2.16 (Splitting a 2-to-2 game)**.** *Let G be a constraint graph whose relations are all as in Lemma 2.15. Let $\mathcal{U}_G$ be the Unique Games instance on the same vertices in which every edge e of G is replaced by two parallel edges, one carrying each of the two bijections of C_e. Then for every labeling ℓ, the fraction of edges of $\mathcal{U}_G$ satisfied by ℓ is exactly* half *the fraction of edges of G satisfied by ℓ. Consequently* $\mathrm{val}(\mathcal{U}_G) = \frac{1}{2}\,\mathrm{val}(G)$.

Proof. A labeling ℓ satisfies $e = (u, v)$ in G iff $(\ell(u), \ell(v)) \in C_e = M_e \sqcup M'_e$, i.e. iff it satisfies exactly one of the two parallel edges of $\mathcal{U}_G$ (the matchings are disjoint, so never both); otherwise it satisfies neither. So ℓ satisfies as many edges of $\mathcal{U}_G$ as of G, while $\mathcal{U}_G$ has twice as many edges. □

Parallel edges are harmless: they only change the weights. Hence the 2-to-2 theorem gives the following unconditional statement, which differs from the Unique Games Conjecture only in the *completeness*.

Corollary 2.17 (A half-way Unique Games Conjecture, from Table 2.3)**.** *For every $\varepsilon > 0$ there is a $k = k(\varepsilon)$ such that it is* NP*-hard to distinguish Unique Games instances with k labels and* $\mathrm{val} \geq \frac{1-\varepsilon}{2}$ *from those with* $\mathrm{val} \leq \frac{\varepsilon}{2}$.

This is the sense in which the 2-to-2 theorem proves the UGC "half-way" (Khot, 2019): completeness $\approx \frac{1}{2}$ instead of $1-\varepsilon$. Khot remarks that the arguments usually cited as evidence against the UGC apply to this version too. The claim of OpenAI (2026) is a claim to supply the missing half; we treat it as unverified (Lecture 16).

Remark (Max-3-Cut is 2-regular). The relation $x \neq y$ on three labels has exactly two partners for every label, so Lemma 2.15 applies: it is the union of the two translations $y = x+1$ and $y = x+2$ modulo 3. (It is not a union of $K_{2,2}$'s, since the bipartite graph is a 6-cycle, so it is not 2-to-2 in the sense of Definition 2.4; see Exercise 2.5.)

2.5 From PCPs to cliques: the FGLSS graph

All of the above treated the proof as an oracle to be queried. The reduction of Feige et al. (1996) does something different: it turns the *set of everything the verifier might see* into a graph, and lets consistency do the work. We fix a verifier V as in Definition 1.6, and an input x. For $\rho \in \{0,1\}^r$ write $I_\rho = (i_1(\rho), \dots, i_q(\rho))$ for the positions it queries (possibly with repetitions).

Definition 2.18 (FGLSS graph)**.** A *local view* is a pair (ρ, a) with $\rho \in \{0,1\}^r$ and $a \in \{0,1\}^q$ such that $P_{x,\rho}(a) = 1$ and $a_j = a_{j'}$ whenever $i_j(\rho) = i_{j'}(\rho)$. Two local views (ρ, a) and (ρ', a') are *compatible* if $\rho \neq \rho'$ and $a_j = a'_{j'}$ whenever $i_j(\rho) = i_{j'}(\rho')$. The *FGLSS graph* H_x has the local views as vertices and an edge between any two compatible views. It has at most 2^{r+q} vertices.

In words: a vertex says "on random string ρ, the proof contained the accepting answers a"; two vertices are adjacent when they could both occur in one and the same proof.

Theorem 2.19 (FGLSS)**.** $\omega(H_x) = 2^r \cdot \max_{\pi \in \{0,1\}^N} \Pr_\rho[V^\pi(x) \text{ accepts}]$.

Proof. ($\geq$) Fix a proof π and let $R = \{\rho : V^\pi(x) \text{ accepts on } \rho\}$. For $\rho \in R$ the pair $(\rho, \pi|_{I_\rho})$ is a local view (accepting, and consistent since it is read from the single string π). Any two of these views agree with π and have different ρ, so they are compatible: a clique of size $|R| = 2^r \Pr_\rho[V^\pi \text{ accepts}]$.

($\leq$) Let K be a clique. Adjacent vertices have different ρ, so K contains at most one view per ρ. Define a proof π by setting $\pi_i = a_j$ whenever $(\rho, a) \in K$ and $i_j(\rho) = i$, and $\pi_i = 0$ for unconstrained positions. This is well defined: two occurrences of position i within one view agree by the consistency condition of a local view, and two occurrences in different views agree by compatibility. Now for every $(\rho, a) \in K$ we have $\pi|_{I_\rho} = a$, so $P_{x,\rho}(\pi|_{I_\rho}) = P_{x,\rho}(a) = 1$: V^π accepts on ρ. Hence V^π accepts on at least $|K|$ random strings. □

Example 2.20 (The FGLSS graph of φ_0)**.** Take V to be the verifier that expects an assignment $\pi \in \{0,1\}^3$ of φ_0 (Example 2.13), picks one of the four clauses ($r = 2$) and reads its two variables ($q = 2$). Each clause has three satisfying answers, so H_x has $4 \cdot 3 = 12$ vertices and (by direct count) 20 edges; Figure 2.3. Every assignment satisfies exactly three of the four clauses, so $\max_\pi \Pr[\text{accept}] = \frac{3}{4}$ and Theorem 2.19 predicts $\omega(H_x) = 4 \cdot \frac{3}{4} = 3$, as one finds by inspection (there are 8 maximum cliques, one for each assignment).

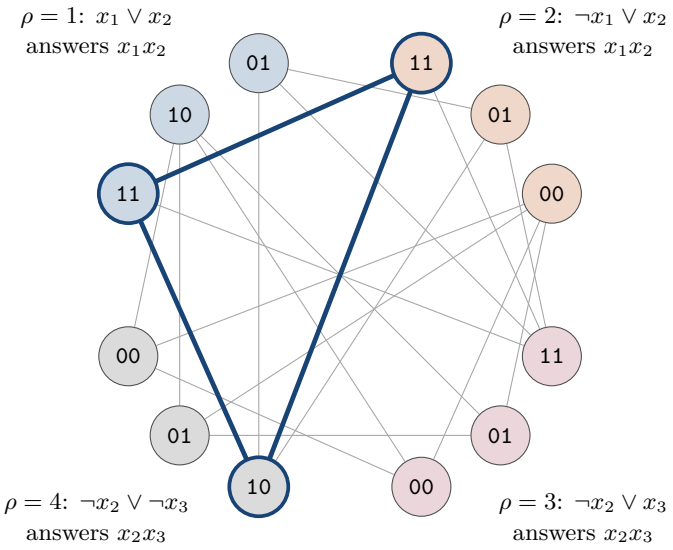


Figure 2.3: The FGLSS graph of the verifier for φ_0 (Example 2.13) that picks one of the four clauses and reads its two variables. Each group of three vertices contains the three satisfying answers of its clause; edges join compatible views. The heavy triangle is a maximum clique: it consists of the views of $\rho = 1, 2, 4$ that agree with the assignment $x = (1, 1, 0)$, which satisfies C_1, C_2, C_4 and violates C_3. No view of $\rho = 3$ is compatible with all three.

Pitfall: a clique is a consistent set, not just an accepting one

Two vertices of H_x may both be accepting views and still be non-adjacent: they disagree about one position of the proof. A clique does not say that many random strings accept *somewhere*; it says that they accept *the same proof*. This is the same idea as the difference between the two provers of a game, who need not be consistent, and a single proof, which is: Theorem 2.19 is the one-oracle statement, and it is exact; no factor 2 appears.

Corollary 2.21 (Constant-factor hardness for Clique)**.** *Unless* $\mathsf{P} = \mathsf{NP}$*, no polynomial-time algorithm outputs, for every graph, a clique of more than half the maximum size.*

Proof. By Theorem 1.7 there is a verifier for 3SAT with $r = O(\log n)$, $q = O(1)$, perfect completeness and soundness $\frac{1}{2}$. Build H_x in time polynomial in n ($2^{r+q} = \mathrm{poly}(n)$ vertices). If $x \in$ 3SAT then $\omega(H_x) = 2^r$; otherwise $\omega(H_x) \le 2^{r-1}$. By Lemma 1.4 no polynomial-time algorithm achieves ratio $\alpha > \frac{1}{2}$ unless $\mathsf{P} = \mathsf{NP}$. □

To pass from the constant 2 to larger factors we amplify the gap. Here graph products do exactly what independent repetition does for proofs. The *strong product* $G \boxtimes H$ has vertex set $V(G) \times V(H)$, and two distinct vertices (g, h), (g', h') are adjacent iff ($g = g'$ or $g \sim g'$) and ($h = h'$ or $h \sim h'$).

Proposition 2.22 (Clique number is multiplicative)**.** $\omega(G \boxtimes H) = \omega(G)\,\omega(H)$.

Proof. If $A \subseteq V(G)$ and $B \subseteq V(H)$ are cliques then $A \times B$ is a clique, giving $\ge$. Conversely let K be a clique of $G \boxtimes H$ and let A and B be its projections to $V(G)$ and $V(H)$. Two distinct elements of A come from two vertices of K that are adjacent,

and since their first coordinates differ they must be adjacent in G; so A is a clique, and likewise B. Since $K \subseteq A \times B$ we get $|K| \le |A||B| \le \omega(G)\omega(H)$. □

Corollary 2.23 (Hardness of Clique at larger factors)**.** *(a) For every constant c, it is* NP*-hard to find a clique of size at least $1/c$ times the maximum. (b) For every $\varepsilon > 0$, if on N-vertex graphs one can find in polynomial time a clique of size at least $2^{-(\log N)^{1-\varepsilon}}$ times the maximum, then* $\mathsf{NP} \subseteq \mathsf{DTIME}(2^{\mathrm{polylog}\, n})$.

Proof. Let $H = H_x$ be the graph of Corollary 2.21, with $M = \mathrm{poly}(n)$ vertices, say $M \le n^a$. The k-fold power $H^{\boxtimes k}$ has M^k vertices and clique number 2^{rk} (YES) or at most $2^{(r-1)k}$ (NO), by Proposition 2.22: a gap of 2^k. For (a) take $k = \lceil \log_2 c \rceil + 1$, a constant. For (b) take $k = \lceil (\log n)^{1/\varepsilon} \rceil$: the graph has $N = n^{O(k)}$ vertices, is built in time $\mathrm{poly}(N) = 2^{O((\log n)^{1+1/\varepsilon})}$, and since $\log N = \Theta(k \log n)$ the gap is $2^k = 2^{\Theta((\log N)^{1/(1+\varepsilon)})}$, which exceeds $2^{(\log N)^{1-\varepsilon}}$ for large n because $\frac{1}{1+\varepsilon} > 1 - \varepsilon$. An algorithm as in (b) would therefore decide 3SAT in quasi-polynomial time. □

This is the shape of the argument of Feige et al. (1996), who had a PCP for NP with randomness and queries $O(\log n \log\log n)$ rather than $O(\log n)$ and $O(1)$; Theorem 2.19 applied to their proof system gives graphs with $n^{O(\log\log n)}$ vertices, and hence hardness under the assumption $\mathsf{NP} \not\subseteq \mathsf{DTIME}(n^{O(\log\log n)})$. The PCP theorem removes the loss.

Where the polynomial factors come from

Corollary 2.23 reaches $2^{(\log N)^{1-\varepsilon}}$ but not $N^{\Omega(1)}$, because k independent repetitions cost rk random bits, and 2^{rk} vertices is quasi-polynomial once k grows with n. The remedy is *randomness-efficient repetition*: replace the k independent random strings by a k-step random walk on a constant-degree expander, which needs only $r + O(k)$ random bits and still drives the soundness down exponentially in k (Exercise 2.6). With $k = \Theta(\log n)$ one gets graphs of polynomial size and gap $N^{\Omega(1)}$, which is the conclusion of Theorem 6 of Arora et al. (1998) (we do not claim that their proof proceeds this way). The exponent was improved by the *free bit* analysis of Bellare et al. (1998) and finally to $N^{1-\varepsilon}$ by Håstad (1999); the latter proof uses Label Cover, parallel repetition and the long code, the toolkit of Lectures 5 and 7.

2.6 What Label Cover is for

The FGLSS reduction reads a PCP and builds a graph. For problems with a rigid local structure—Max-E3-SAT, Max-E3-Lin, and later Max-Cut—a different recipe is needed, and Label Cover is its first ingredient.

The recipe of the next lectures

Step 1 (Lectures 1–2, 5). Prove that a Label Cover instance (projection constraints, soundness δ, alphabet $\mathrm{poly}(1/\delta)$) is NP-hard to approximate. This is the *outer verifier*. **Step 2 (Lecture 7).** Replace every vertex v of the instance by a table f_v that is supposed to be the *long-code* encoding of the label of v, and every edge $e = (u, v)$ by a *local test* between f_u and f_v that reads very few entries; this is the *inner verifier*. The tests have the shape of the target problem: three bits for Max-E3-Lin, two bits for Max-Cut. **Step 3 (analysis).** If the Label Cover instance has a labeling of value 1, encoding it passes all tests (completeness). If a family of tables passes the tests with probability above the claimed soundness, *decode* each table into a short list of candidate labels; a random choice from the lists is a labeling that satisfies many edges, contradicting the soundness of Label

Cover. The decoding is where the *type* of the constraints matters: a projection lets the table at v be read from the table at u, and the *unique* type of the UGC lets decoding lose nothing when the two tables are asked only two bits.

Label Cover is also the source of hardness for problems that look nothing like constraint satisfaction. The same two-step recipe gives the logarithmic hardness of Set Cover (Lund and Yannakakis, 1994; Feige, 1998; Dinur and Steurer, 2014); and Label Cover under its original name was introduced in the context of lattice and code problems (Arora et al., 1997).

Exercises

Exercise 2.1 ([⋆]). Let G be a connected graph with linear constraints $x_v - x_u = c_e \pmod k$ on the edges $e = (u, v)$. Prove that $\mathrm{val}(G) = 1$ iff around every cycle the signed sum of the c_e is $0 \bmod k$ (an edge traversed against its orientation contributes $-c_e$). *Hint: spanning tree.* Use this to give another proof of Proposition 1.20 for linear constraints.

Exercise 2.2 ([⋆⋆]). Compute $\mathrm{val}(G)$ and $\mathrm{val}(G^{\times 2})$ for Max-Cut on K_n and show $\mathrm{val}(G^{\times 2})/\mathrm{val}(G) = 2 - \frac{2}{n}$ for even n. Show that for Max-Cut on an odd cycle C_ℓ the ratio is $\frac{\ell}{\ell-1}$. Conclude that the factor 2 in Proposition 2.8 cannot be replaced by any constant smaller than 2.

Exercise 2.3 ([⋆⋆]). (a) Let φ be the 3-CNF consisting of all eight clauses on three variables (every sign pattern). Show that $\mathrm{val}(\varphi) = \frac{7}{8}$ and $\mathrm{val}(G_\varphi) = \frac{23}{24} = 1 - \frac{1/8}{3}$, so the bound of Theorem 2.12 is tight. (b) Show that the bound is also tight for Max-Cut on a triangle, viewed as a CSP of arity 2: $\mathrm{val}(\Phi) = \frac{2}{3}$ and $\mathrm{val}(G_\Phi) = \frac{5}{6}$.

Exercise 2.4 ([⋆⋆]). Show that a relation $C \subseteq \Sigma \times \Sigma'$ in which every label of both sides has exactly d partners is a disjoint union of d perfect matchings. *Hint: Hall's theorem; remove one perfect matching and induct.* Deduce the analogue of Corollary 2.16: from a d-to-d game with $\mathrm{val} = 1 - \varepsilon$ one gets a Unique Game with $\mathrm{val} = (1 - \varepsilon)/d$ on the same graph, with every edge repeated d times.

Exercise 2.5 ([⋆]). (a) Verify that $x \neq y$ on $\mathbb{Z}_3$ is 2-regular but not a disjoint union of copies of $K_{2,2}$. (b) Write it as the union of the Unique constraints $y = x + 1$ and $y = x + 2$. (c) For Max-3-Cut on K_4 compute $\mathrm{val}(G)$ and the value of the split Unique Games instance of Corollary 2.16.

Exercise 2.6 ([⋆ ⋆ ⋆]). *Randomness-efficient repetition.* Let A be the normalized adjacency matrix of a d-regular graph on the vertex set $\{0,1\}^r$ with second largest eigenvalue in absolute value at most λ, and let $B \subseteq \{0,1\}^r$ have density μ. (a) Prove that $\|P_B A P_B\| \le \mu + \lambda$, where P_B is the coordinate projection onto B. *Hint: write a vector supported on B as $\alpha \mathbf{1}/\sqrt{2^r} + y$ with $y \perp \mathbf{1}$ and bound $\alpha^2 \le \mu \|u\|^2$ by Cauchy–Schwarz.* (b) Deduce that a uniformly started random walk $\rho_1, \dots, \rho_k$ stays inside B throughout with probability at most $\mu(\mu + \lambda)^{k-1}$. (c) Let V be a verifier with perfect completeness and soundness $\frac{1}{2}$ and suppose an explicit family of such expanders with $\lambda \le \frac{1}{4}$ and constant degree d exists. Define V_k that reads the walk's k random strings, makes all kq queries and accepts iff all accept. Show that V_k uses $r + (k-1)\log d$ random bits and has soundness $(3/4)^{k-1}$, and conclude that with $k = \Theta(\log n)$ the FGLSS graph has $N = \mathrm{poly}(n)$ vertices and a gap $N^{\Omega(1)}$.

Exercise 2.7 ([⋆⋆])**.** *Reading.* Section 2.3 of Harsha and Charikar (2010) reduces the PCP theorem to Label Cover by putting the random strings of the verifier on the left and the proof positions on the right. Compare this reduction with Corollary 2.14. Apply both to the verifier that picks a random clause of a 3-CNF and queries its three variables, and identify the games.

Notes and further reading

Where this lecture comes from. The dictionary between verifiers, CSPs and games is standard; we followed the treatment of the PCP theorem as a statement about Label Cover in Section 2.3 of the DIMACS notes (Harsha and Charikar, 2010). The name *Label Cover* comes from Arora et al. (1997); two-prover one-round proof systems are due to Feige and Lovász (1992); the clause–variable game is the canonical example. The d-to-1 constraints are from Khot (2002), and the 2-to-2 constraints and the "half-way" remark are from Khot (2019); the passage from a Unique Game to the bipartite two-prover game, with a loss of a factor of at most 4, is noted in Khot (2010); we sharpened the factor to 2 for the symmetric double cover (Proposition 2.8).

On attributions. The Clique reduction and the exact formula $\omega(H_x) = 2^r \cdot \max_\pi \Pr[\text{accept}]$ are due to Feige et al. (1996). The hardness of approximating Clique within N^ε for some $\varepsilon > 0$, assuming only $\mathsf{P} \neq \mathsf{NP}$, is Theorem 6 of Arora et al. (1998), improving the $2^{\sqrt{\log N}}$ of Arora and Safra (1998); the factor $N^{1-\varepsilon}$ is Håstad (1999), as mentioned in Lecture 1. The multiplicativity of the clique number under the strong product is folklore.

What is not covered. We have not proved any hardness result for Label Cover itself beyond what Lecture 1 did, and we have not discussed the *size* of Label Cover instances (the Projection Games Conjecture asks for a nearly linear size). Raz's theorem, which supplies the soundness, is proved in Lecture 5.

LECTURE 3

The Arora–Safra Route: Polynomials, Sum-Check and Composition

How a global algebraic statement becomes a handful of local queries

Lectures 1 and 2 took the PCP theorem as a black box and asked what it gives us. This lecture opens the box—or rather, opens the first one, the proof of Arora and Safra (1998). The second proof, the one that reaches $O(1)$ queries (Arora et al., 1998), is Lecture 4; the third, combinatorial one is Lecture 6.

Be warned about what Arora and Safra actually proved. Their theorem is *not* $\mathsf{NP} = \mathsf{PCP}[O(\log n), O(1)]$. It is

$$\mathsf{NP} = \mathsf{PCP}[\log n, \log n] \qquad \text{and} \qquad \mathsf{NP} = \mathsf{PCP}\left[\log n,\ \log^{1/2+\varepsilon} n\right] \ \text{ for every fixed } \varepsilon > 0,$$

a verifier that reads a *sublogarithmic* number of proof bits. The step from $\log^{1/2+\varepsilon} n$ to $O(1)$ was taken soon afterwards by Arora et al. (1998) (both papers appeared at FOCS 1992), using the same composition lemma and a sharper low-degree test (Section 3.11). We therefore state precisely the theorem proved here, and we resist the common shorthand "Arora–Safra proved the PCP theorem".

What makes the proof worth a full lecture is that it is built from *modules* with clean interfaces, and we can state and prove most of them. The cast:

- a *code*: low-degree polynomials, which turn "f is almost right everywhere" into "f is exactly right at a random point";
- an *arithmetization*: satisfiability of a 3-CNF becomes "a certain polynomial sums to 0 over a cube";
- the *sum-check*: a table that lets a verifier check such a sum by reading $O(m)$ entries;
- the *low-degree test*: a table that lets a verifier check that a function is close to a polynomial by reading $O(m)$ entries;
- *composition*: the lemma that lets a verifier with many queries to long symbols be rewritten as one that reads fewer, shorter symbols.

Each module has an input, an output and a parameter change. The point of the lecture is to track them.

After this lecture you should be able to...

(1) state exactly what Arora and Safra (1998) proved and which constants separate it from the PCP theorem (Section 3.1);

(2) prove the Schwartz–Zippel lemma, and use it to decode a function that is close to a low-degree polynomial (Section 3.2);

(3) arithmetize a 3-CNF as a vanishing condition on a cube, and compute the single nonzero point of g_A on a small example (Section 3.3);

Result	Randomness and queries	Consequence for Max-Clique
$\mathsf{NP} = \mathsf{PCP}[0, \mathrm{poly}(n)]$ (definition)	none, all of the witness	none
Babai et al. (1991a)	$\mathsf{NP} \subseteq \mathsf{PCP}[\mathrm{polylog}\, n, \mathrm{polylog}\, n]$ (implicit)	—
Feige et al. (1996)	$\mathsf{PCP}[\log n \log\log n, \log n \log\log n]$	hard within any constant unless $\mathsf{NP} \subseteq \mathsf{DTIME}(n^{O(\log\log n)})$
Arora and Safra (1998)	$\mathsf{PCP}[\log n, \log n]$ and $\mathsf{PCP}[\log n, \log^{1/2+\varepsilon} n]$	within $2^{c \log^{1/2-\varepsilon} N}$ is NP-hard
Arora et al. (1998)	$\mathsf{PCP}[\log n, 1]$	within N^{ε_1} is NP-hard

Table 3.1: The race for the number of queries (all verifiers non-adaptive; N is the number of vertices of the clique instance). The row of Arora and Safra is the subject of this lecture. Each row implies the ones above it, since $\mathsf{PCP}[r, q] \subseteq \mathsf{PCP}[r', q']$ whenever $r \le r'$ and $q \le q'$.

(4) prove the soundness $1 - dl/q$ of the sum-check by induction and compute the cheating probability of a given table (Section 3.4);

(5) prove that a function all of whose axis-parallel lines are low-degree is a polynomial, and explain why the real test needs a *robust* version of this fact (Section 3.5);

(6) prove the composition lemma, and carry out the parameter chase that ends at $\log^{1/2+\varepsilon} n$ queried bits (Sections 3.8–3.9).

3.1 What was proved, and when

Recall the notation of Definition 1.6: $\mathsf{PCP}[r, q]$ is the class of languages with a non-adaptive verifier using $O(r)$ random bits and $O(q)$ queries, perfect completeness and soundness $\frac{1}{2}$. Trivially $\mathsf{PCP}[r, q] \subseteq \mathsf{NTIME}(2^{O(r)} q + \mathrm{poly}(n))$, so $\mathsf{PCP}[O(\log n), q] \subseteq \mathsf{NP}$ for every $q = \mathrm{poly}(n)$ (we used this in Section 1.5). The whole question is how *small* q can be.

Two further statements from Arora and Safra (1998) will be used.

Theorem 3.1 (Arora–Safra)**.** *For every fixed $\varepsilon > 0$, $\mathsf{NP} = \mathsf{PCP}\big[\log n,\ \log^{1/2+\varepsilon} n\big]$.*

Corollary 3.2 (Arora–Safra)**.** *For all constants $c, \varepsilon > 0$, if Max-Clique on N-vertex graphs can be approximated within $2^{c \log^{1/2-\varepsilon} N}$ in polynomial time, then $\mathsf{P} = \mathsf{NP}$.*

Corollary 3.2 is the *reason* the paper was written: it removes the "almost" from the title of Feige et al. (1996), who had shown that approximating Clique within any constant factor implies $\mathsf{NP} \subseteq \mathsf{DTIME}(n^{O(\log\log n)})$. We derive it in Section 3.10.

Pitfall: three different statements share the name "PCP theorem"

(1) $\mathsf{NP} = \mathsf{PCP}[\log n, \log n]$ and $\mathsf{NP} = \mathsf{PCP}[\log n, \log^{1/2+\varepsilon} n]$: Arora–Safra, FOCS 1992. (2) $\mathsf{NP} = \mathsf{PCP}[\log n, 1]$: Arora–Lund–Motwani–Sudan–Szegedy, FOCS 1992. This is what Lecture 1 called the PCP theorem (Theorem 1.7); the *approximation* consequences for Max-3SAT need (2), not (1): through the reduction of Proposition 1.9 the constant ε_0 is $2^{-O(q)}$, so with $\log^{1/2+\varepsilon} n$ queried bits one only gets a gap that shrinks with n, not a constant (Arora and Safra, Section 6). (3) The combinatorial proof of the same statement as (2) by Dinur (2007). Arora and Safra's own Section 6 is explicit about this division of labour.

3.2 Polynomials as codes

Fix a finite field $\mathbb{F}_q$. For integers $h, m \geq 1$ let

$$\mathcal{P}_h^m \;=\; \{\, p \in \mathbb{F}_q[x_1, \ldots, x_m] \;:\; \deg_{x_i} p \leq h \text{ for every } i \,\}$$

be the polynomials of degree at most h *in each variable*, viewed as functions $\mathbb{F}_q^m \to \mathbb{F}_q$. (Each $p \in \mathcal{P}_h^m$ has total degree at most hm.) For $f, g \colon \mathbb{F}_q^m \to \mathbb{F}_q$ write $\Delta(f,g) = \Pr_{x \in \mathbb{F}_q^m}[f(x) \neq g(x)]$ for their (normalized) distance, and say that f is δ-*close* to $\mathcal{P}_h^m$ if $\Delta(f,p) \leq \delta$ for some $p \in \mathcal{P}_h^m$.

Lemma 3.3 (Schwartz–Zippel)**.** *A nonzero $p \in \mathcal{P}_d^m$ vanishes on at most a dm/q fraction of the points of $\mathbb{F}_q^m$.*

Proof. Induction on m. For $m = 1$ a nonzero univariate polynomial of degree $\leq d$ has at most d roots. For $m > 1$ write $p = \sum_{i=0}^{k} x_1^i \, p_i(x_2, \ldots, x_m)$ with $k \leq d$ and $p_k \neq 0$. By induction p_k vanishes on at most a $d(m-1)/q$ fraction of $\mathbb{F}_q^{m-1}$. For every point $(x_2, \ldots, x_m)$ where $p_k \neq 0$, the polynomial $p(\cdot, x_2, \ldots, x_m)$ is a nonzero univariate polynomial of degree $k \leq d$, hence vanishes for at most k values of x_1. So the fraction of zeros is at most $\frac{d(m-1)}{q} + \frac{k}{q} \leq \frac{dm}{q}$. □

Corollary 3.4 (Codes of large distance)**.** *Distinct $p, p' \in \mathcal{P}_h^m$ satisfy $\Delta(p,p') \geq 1 - hm/q$. If $hm < q/2$ and f is δ-close to $\mathcal{P}_h^m$ for some $\delta < \frac{1}{4}$, then f is δ-close to a* unique *$\tilde{f} \in \mathcal{P}_h^m$, and no other polynomial of $\mathcal{P}_h^m$ is within $\frac{1}{2} - \delta$ of f.*

Proof. The first claim is Lemma 3.3 applied to $p - p'$. If p, p' were both within δ of f, the triangle inequality would give $\Delta(p,p') \leq 2\delta < \frac{1}{2} < 1 - hm/q$, a contradiction. The last claim is the same computation with $\frac{1}{2} - \delta$. □

Corollary 3.4 is the reason polynomials are used: a function that is 1% wrong still *determines* the polynomial it is trying to be. Next, how to turn a string into a polynomial.

Lemma 3.5 (Polynomial extension)**.** *Let $h < q$, identify $\{0, 1, \ldots, h\}$ with the first $h+1$ elements of $\mathbb{F}_q$, and let $s \colon \{0, \ldots, h\}^m \to \mathbb{F}_q$ be any function. Then there is a* unique *$\hat{s} \in \mathcal{P}_h^m$ with $\hat{s} = s$ on $\{0, \ldots, h\}^m$, namely*

$$\hat{s}(x) \;=\; \sum_{u \in \{0,\ldots,h\}^m} s(u)\, \Lambda_u(x), \qquad \Lambda_u(x) = \prod_{i=1}^{m} \lambda_{u_i}(x_i), \qquad \lambda_a(x) = \prod_{b \in \{0..h\},\, b \neq a} \frac{x-b}{a-b}.$$

Proof. Each λ_a has degree h, equals 1 at a and 0 at the other points of $\{0..h\}$; hence Λ_u is 1 at u and 0 elsewhere on the cube, and $\hat{s} \in \mathcal{P}_h^m$ agrees with s on the cube. For uniqueness it suffices to show that $p \in \mathcal{P}_h^m$ vanishing on the cube is zero. Write $p = \sum_{j \le h} x_m^j p_j(x_1, \dots, x_{m-1})$. For every fixed $x' \in \{0..h\}^{m-1}$ the univariate polynomial $p(x', \cdot)$ has degree $\le h$ and $h+1$ roots, so it is zero, i.e. $p_j(x') = 0$ for all j. Hence each p_j vanishes on $\{0..h\}^{m-1}$ and by induction $p_j = 0$. □

Example 3.6 (The multilinear extension of XOR)**.** For $m = 2$, $h = 1$ and $s(0,0) = s(1,1) = 0$, $s(1,0) = s(0,1) = 1$, Lemma 3.5 gives

$$\hat{s}(x_1, x_2) = x_1(1 - x_2) + (1 - x_1)x_2 = x_1 + x_2 - 2x_1x_2.$$

Over $\mathbb{F}_7$ it takes the value $3 + 5 - 30 \equiv 6$ at $(3, 5)$. The function s is determined by 4 values; its extension is a table with $q^2 = 49$ entries, and two distinct such extensions differ on at least $1 - 2/7 = 5/7$ of the 49 points (Corollary 3.4), so even a corrupted table pins down s.

Encoding strings. Let $n = (h+1)^m$. Identify $\{1, \dots, n\}$ with the cube $\{0, \dots, h\}^m$ (base-$(h+1)$ digits) and encode $a \in \{0,1\}^n \subset \mathbb{F}_q^n$ by $\hat{a} \in \mathcal{P}_h^m$. The encoding is injective, and the codeword is a table of q^m field elements; if $q = \text{poly}(h)$ then $q^m = h^{O(m)} = \text{poly}(n)$ since $(h+1)^m = n$. This is the sense in which the encoding is "efficient".

Pitfall: a table that is close to a polynomial is not a polynomial

The verifier will not hold a polynomial; it holds a *table* $f \colon \mathbb{F}_q^m \to \mathbb{F}_q$ supplied by an adversary. The low-degree test (Section 3.5) can only certify that f is δ-close to some $\tilde{f} \in \mathcal{P}_h^m$ for a small constant δ. Any later step that needs values of $\tilde{f}$ at a point x reads $f(x)$ and is correct unless x lies in the δ-fraction where $f \neq \tilde{f}$. The design principle of the whole verifier is therefore: *never evaluate $\tilde{f}$ at a point the adversary can predict.* Every point at which f is read must be uniformly distributed, so that the error probability is $\le \delta$ per read.

3.3 Arithmetization: satisfiability as a vanishing condition

Let φ be a 3-CNF with $n = (h+1)^m$ variables and n clauses (pad with dummy variables and clauses). Let $\mathbb{K} = \{0, 1, \dots, h\} \subset \mathbb{F}_q$. Identify variables and clauses with points of the cube $\mathbb{K}^m$. Every clause $c \in \mathbb{K}^m$ has an ordered triple of variables $v_1(c), v_2(c), v_3(c) \in \mathbb{K}^m$ and signs $\varsigma_j(c) \in \{0,1\}$ ($\varsigma_j(c) = 1$ if the jth literal is unnegated). Define

$$\chi_j(c, v) = \begin{cases} 1 & \text{if } v = v_j(c) \\ 0 & \text{otherwise} \end{cases} \qquad (\mathbb{K}^m \times \mathbb{K}^m \to \{0,1\}), \qquad \varsigma_j \colon \mathbb{K}^m \to \{0,1\},$$

and let $\hat{\chi}_j \in \mathcal{P}_h^{2m}$, $\hat{\varsigma}_j \in \mathcal{P}_h^m$ be their polynomial extensions (Lemma 3.5); they depend only on φ and are computable in time $\text{poly}(n, q^m)$.

For $A \in \mathcal{P}_h^m$, which we think of as the extension of a Boolean assignment, define $g_A \colon \mathbb{F}_q^{4m} \to \mathbb{F}_q$ by

$$g_A(z, w_1, w_2, w_3) \;=\; \prod_{j=1}^{3} \hat{\chi}_j(z, w_j) \cdot (\hat{\varsigma}_j(z) - A(w_j)), \qquad z, w_1, w_2, w_3 \in \mathbb{F}_q^m.$$

Lemma 3.7 (Algebraic view of 3SAT)**.** *Let $A \in \mathcal{P}_h^m$.*

(a) If $a \in \{0,1\}^n$ satisfies φ and $A = \hat{a}$, then $g_A(y) = 0$ for every $y \in \mathbb{K}^{4m} = \{0,\dots,h\}^{4m}$. If $a \in \{0,1\}^n$ does not satisfy φ, then $g_{\hat{a}}$ is nonzero at some point of $\mathbb{K}^{4m}$.

(b) Conversely, for every *A: if g_A vanishes on $\mathbb{K}^{4m}$ then φ is satisfiable. (The assignment $a'_v = \mathbf{1}[A(v) = 1]$ satisfies it.)*

(c) $g_A \in \mathcal{P}_{6h}^{4m}$, and evaluating g_A at a point (z, w_1, w_2, w_3) requires only the values of A at the three points $w_1, w_2, w_3 \in \mathbb{F}_q^m$ (plus $\mathrm{poly}(n)$ work independent of A). If the point is uniform in $\mathbb{F}_q^{4m}$ then each w_j is uniform in $\mathbb{F}_q^m$.

Proof. Fix $(c, v_1, v_2, v_3) \in \mathbb{K}^{4m}$. On the cube the extensions $\hat{\chi}_j, \hat{\varsigma}_j$ equal χ_j, ς_j. If $(v_1, v_2, v_3) \neq (v_1(c), v_2(c), v_3(c))$ then some $\chi_j(c, v_j) = 0$ and $g_A = 0$ at this point. Otherwise $g_A(c, v_1, v_2, v_3) = \prod_{j=1}^{3}(\varsigma_j(c) - A(v_j))$, which vanishes iff $A(v_j) = \varsigma_j(c)$ for some j.

(a) For Boolean a and $A = \hat{a}$ we have $A(v_j) = a_{v_j}$, and $a_{v_j(c)} = \varsigma_j(c)$ says exactly that the jth literal of c is true. So g_A vanishes at $(c, v_1(c), v_2(c), v_3(c))$ iff a satisfies c.

(b) If $A(v_j(c)) = \varsigma_j(c) \in \{0,1\}$ then $a'_{v_j(c)} = \varsigma_j(c)$ (for $\varsigma_j = 1$ because $A = 1$; for $\varsigma_j = 0$ because $A = 0 \neq 1$). So every clause has a literal made true by a'.

(c) Each factor is a product of two polynomials of degree $\leq h$ in every variable; the three factors share z, so the degree is $\leq 6h$ in each z_i, and $\leq 2h$ in each $w_{j,i}$. The rest is read off the formula; the w_j are disjoint blocks of coordinates of a uniform point, hence uniform. □

Pitfall: the arithmetization certifies the *rounded* assignment

Part (b) says that a table A with $g_A \equiv 0$ on the cube is satisfiable-by-rounding, not that its values on the cube are bits. Indeed take any satisfying a and change a_v to 5 for a variable v that occurs in clauses whose other literals are true: g_A still vanishes on the cube. This does no harm for the *soundness* of a verifier for 3SAT (an unsatisfiable φ is rejected by every A, by (b)), but it matters in Section 3.7, where the verifier will be asked about the *decoded string* of A. There one needs the further vanishing condition $A(A-1) \equiv 0$ on the cube, and one more sum-check. Arora and Safra (1998) leave this implicit; the cost is a constant factor.

Example 3.8 (A formula, an assignment, one nonzero point)**.** Take $h = 1$, $m = 2$ (so $n = 4$ variables $x_0, \dots, x_3$, named by their binary index, and 4 clauses) and

$$C_0 = x_3 \vee \neg x_2 \vee \neg x_1, \quad C_1 = \neg x_3 \vee x_2 \vee x_0, \quad C_2 = x_1 \vee x_0 \vee \neg x_2, \quad C_3 = \neg x_0 \vee \neg x_2 \vee \neg x_3.$$

The assignment $a = (x_0, x_1, x_2, x_3) = (0,0,0,1)$ falsifies exactly C_1 (here $x_3 = 1$, $x_2 = 0$, $x_0 = 0$). Computing $g_{\hat{a}}$ at all $2^8 = 256$ points of $\{0,1\}^8$ over $\mathbb{F}_{31}$ gives a single nonzero value:

$$g_{\hat{a}}(\underbrace{(0,1)}_{C_1}, \underbrace{(1,1)}_{x_3}, \underbrace{(1,0)}_{x_2}, \underbrace{(0,0)}_{x_0}) = (0-1)(1-0)(1-0) = -1,$$

the product of $\varsigma_j - a_{v_j}$ for the three false literals. So a single violated clause becomes a single nonzero point of a 256-point cube: a needle in a haystack. We cannot hope to find it by sampling, and this is exactly why we multiply by the random *zero-testers* of the next lemma, which turn "somewhere nonzero" into "the weighted sum is nonzero".

Lemma 3.9 (Zero-testers)**.** *For $t = (t_1, \dots, t_k) \in \mathbb{F}_q^k$ let $R_t \in \mathcal{P}_h^k$ be the polynomial $R_t(y) = \prod_{i=1}^k I_{t_i}(y_i)$, where $I_s \in \mathcal{P}_h^1$ is the polynomial with $I_s(a) = s^a$ for $a = 0, \dots, h$ (Lemma 3.5). Then for every $f\colon \{0..h\}^k \to \mathbb{F}_q$ that is not identically zero,*

$$\Pr_{t \in \mathbb{F}_q^k}\Big[\sum_{y \in \{0..h\}^k} R_t(y) f(y) = 0\Big] \;\le\; \frac{hk}{q}.$$

In particular this is less than $\frac{1}{8}$ when $q > 8hk$ ($q > 32hm$ for $k = 4m$).

Proof. For $y \in \{0..h\}^k$ we have $R_t(y) = \prod_i t_i^{y_i}$. Hence $F(t) := \sum_y R_t(y) f(y) = \sum_y f(y) \prod_i t_i^{y_i}$ is a polynomial in t of degree $\le h$ in every variable, and it is nonzero because distinct y give distinct monomials and some $f(y) \neq 0$. By Lemma 3.3, $\Pr_t[F(t) = 0] \le hk/q$. □

Example 3.10 (Zero-testers on Example 3.8)**.** Here $k = 4m = 8$ and $g_{\hat{a}}|_{\mathbb{K}^{4m}}$ has a single nonzero point, $y = (0,1\,|\,1,1\,|\,1,0\,|\,0,0)$ with value -1. So $F(t) = -t_2 t_3 t_4 t_5$ (the coordinates where y has a 1), which vanishes iff one of these four coordinates of t is 0:

$$\Pr_t[F(t) = 0] \;=\; 1 - \left(\tfrac{30}{31}\right)^4 \;\approx\; 0.123 \;\;\le\;\; \frac{hk}{q} = \frac{8}{31} \approx 0.258.$$

(A simulation with $2 \cdot 10^5$ samples gives 0.123.) The field is too small for the hypothesis $q > 8hk = 64$ of the lemma, but the bound hk/q itself holds. Note how sparse F is: a single violated clause is a one-monomial polynomial, and the zero-tester detects it unless t is unlucky in one of four coordinates.

Corollary 3.11 (The summed polynomials P_t)**.** *For $t \in \mathbb{F}_q^{4m}$ and $A \in \mathcal{P}_h^m$ put $P_t^A = R_t \cdot g_A \in \mathcal{P}_{7h}^{4m}$ (R_t the zero-testers with $k = 4m$, $q > 32hm$). Then*

- *if $A = \hat{a}$ for a satisfying assignment a, then $\sum_{y \in \mathbb{K}^{4m}} P_t^A(y) = 0$ for* every *t;*
- *if φ is* unsatisfiable*, then for* every *$A \in \mathcal{P}_h^m$ we have $\sum_{y \in \mathbb{K}^{4m}} P_t^A(y) \neq 0$ for all but at most an $\frac{1}{8}$ fraction of the t.*

Moreover $P_t^A(y)$ at a uniformly random $y \in \mathbb{F}_q^{4m}$ is computed from three uniformly distributed values of A.

Proof. First bullet: Lemma 3.7(a). Second bullet: by Lemma 3.7(b), $g_A \not\equiv 0$ on $\mathbb{K}^{4m}$ when φ is unsatisfiable; apply Lemma 3.9 to $f = g_A|_{\mathbb{K}^{4m}}$. □

What arithmetization buys

A 3-CNF is a list of n constraints, and a single violated clause is a needle in a haystack that a verifier reading $O(1)$ clauses cannot find. After arithmetization the same fact is one equation, "$\sum_{y \in \mathbb{K}^{4m}} P_t^A(y) = 0$", about a single low-degree polynomial for a random t. The sum has $(h+1)^{4m} = n^4$ terms, but the polynomial has structure—low degree in every variable—and the *sum-check* of the next section exploits that structure to verify the sum by evaluating the polynomial at *one* random point.

3.4 The sum-check protocol, as a table

The sum-check is due to Lund et al. (1992); we give the non-adaptive, table-based form of Arora and Safra (1998).

Let $B \in \mathcal{P}_d^\ell$ and $H \subseteq \mathbb{F}_q$. For $1 \le i \le \ell$ and $a_1, \dots, a_{i-1} \in \mathbb{F}_q$ put

$$B_{a_1,\dots,a_{i-1}}(x_i) \;=\; \sum_{x_{i+1},\dots,x_\ell \in H} B(a_1, \dots, a_{i-1}, x_i, x_{i+1}, \dots, x_\ell),$$

a univariate polynomial of degree $\le d$ in x_i (for $i = 1$ we write $B_\varnothing(x_1)$). A *table of partial sums* is any assignment g of a univariate polynomial $g_{a_1,\dots,a_{i-1}}$ of degree $\le d$ to every prefix $(a_1, \dots, a_{i-1}) \in \mathbb{F}_q^{i-1}$, $1 \le i \le \ell$; it has $\sum_i q^{i-1} \le q^\ell$ entries, each of $O(d \log q)$ bits. The *honest* table is $g_{a_1..a_{i-1}} = B_{a_1..a_{i-1}}$.

Definition 3.12 (Sum-check test)**.** Input: B (available through oracle access), H, a claimed value $c \in \mathbb{F}_q$ and a table g. Choose $a_1, \dots, a_\ell \in \mathbb{F}_q$ independently and uniformly; then accept iff

$$\begin{aligned}
&\sum_{x \in H} g_\varnothing(x) = c, \\
&\sum_{x \in H} g_{a_1,\dots,a_{i-1}}(x) = g_{a_1,\dots,a_{i-2}}(a_{i-1}) \quad (2 \le i \le \ell), \\
&g_{a_1,\dots,a_{\ell-1}}(a_\ell) = B(a_1, \dots, a_\ell).
\end{aligned}$$

The test reads ℓ table entries and one value of B at the uniform point $a = (a_1, \dots, a_\ell)$. The entries to be read depend only on a, not on earlier answers: the test is non-adaptive. It uses $\ell \log q = \log |\mathbb{F}_q^\ell|$ random bits.

Lemma 3.13 (Sum-check)**.** *Let $B \in \mathcal{P}_d^\ell$, $c \in \mathbb{F}_q$.*

(a) If $\sum_{x \in H^\ell} B(x) = c$, the honest table is accepted with probability 1*.*

(b) If $\sum_{x \in H^\ell} B(x) \ne c$, then every *table is accepted with probability at most $d\ell/q$.*

Proof. (a) Each check is an identity between partial sums.

(b) Induction on ℓ. For $\ell = 1$ the test accepts iff $\sum_H g_\varnothing = c$ and $g_\varnothing(a_1) = B(a_1)$. Since $\sum_H B \ne c$ we have $g_\varnothing \ne B$, and two distinct polynomials of degree $\le d$ agree in at most d points, so the acceptance probability is at most d/q.

For $\ell > 1$ let $B_\varnothing(x_1) = \sum_{x_2..x_\ell \in H} B(x_1, x_2, \dots, x_\ell)$. Then $\sum_{x \in H} B_\varnothing(x) = \sum_{H^\ell} B \ne c$. *Case 1:* $g_\varnothing = B_\varnothing$. Then $\sum_H g_\varnothing \ne c$ and the first check fails with probability 1. *Case 2:* $g_\varnothing \ne B_\varnothing$. They agree in at most d points a_1. For every other a_1, the claim "$\sum_{x_2..x_\ell \in H} B(a_1, x_2, \dots, x_\ell) = g_\varnothing(a_1)$" is false. Fixing a_1, the remaining checks form exactly the sum-check test for the polynomial $B(a_1, \cdot) \in \mathcal{P}_d^{\ell-1}$, the claim $c' = g_\varnothing(a_1)$ and the sub-table $g_{a_1,*}$; by induction it accepts with probability $\le d(\ell-1)/q$. Hence the overall acceptance probability is at most

$$\frac{d}{q} \cdot 1 + \Big(1 - \frac{d}{q}\Big) \frac{d(\ell-1)}{q} \;\le\; \frac{d\ell}{q}. \qquad \square$$

(The recursion $p_\ell \le d/q + (1 - d/q)p_{\ell-1}$, $p_1 \le d/q$, solves to the slightly sharper $p_\ell \le 1 - (1 - d/q)^\ell \le d\ell/q$.)

Example 3.14 (A cheating table, exactly)**.** Let $q = 7$, $d = 2$, $\ell = 2$, $H = \{0, 1\}$ and $B(x_1, x_2) = x_1x_2 + x_2^2 + 1$. Then $B_\varnothing(x_1) = B(x_1, 0) + B(x_1, 1) = 1 + (x_1 + 2) = x_1 + 3$, and the true sum is $B_\varnothing(0) + B_\varnothing(1) = 3 + 4 = 7 \equiv 0$. Suppose the prover wants to convince

us of the false claim $c = 1$. An exhaustive computation over all tables (every polynomial of degree ≤ 2 at every node of the tree) gives

$$\max_g \Pr[\text{accept}] \;=\; \frac{24}{49} \;=\; 1 - \Big(\frac{5}{7}\Big)^2 \;\approx\; 0.490 \quad < \quad \frac{d\ell}{q} = \frac{4}{7} \approx 0.571.$$

The maximum is the same for every false claim $c \neq 0$, and equals exactly the sharper bound $1-(1-d/q)^\ell$ of the remark after the proof: at the first level the cheating polynomial can have $d = 2$ points a_1 at which it agrees with $B_\varnothing$, and at each of the other points the second level leaves a further d/q. The bound $d\ell/q$ of the lemma is its linearization.

Pitfall: a table of partial sums is a prover strategy written out

In the interactive sum-check of Lund et al. (1992) the prover answers round by round, and its ith message may depend on the challenges $a_1, \dots, a_{i-1}$. A deterministic strategy of such a prover is *exactly* a table of partial sums: the entry $g_{a_1,\dots,a_{i-1}}$ is its ith message. So the lemma says the same thing in both models. What the table form adds is that the verifier may choose all of a first and then read the ℓ entries on the path to a in one batch: the test is non-adaptive. This is needed for the composition lemma (Section 3.8).

3.4.1 Putting arithmetization and sum-check together

Combine Corollary 3.11 and Lemma 3.13 with $B = P_t^A$, $d = 7h$, $\ell = 4m$, $H = \{0..h\}$, $c = 0$. A verifier that holds $f\colon \mathbb{F}_q^m \to \mathbb{F}_q$ (in the role of A) and a collection of sum-check tables $\{g^{(t)}\}_{t\in\mathbb{F}_q^{4m}}$ does the following:

1. pick $t \in \mathbb{F}_q^{4m}$ uniformly ($4m \log q$ random bits) and compute from φ and t everything about P_t^f except the values of f;
2. run the sum-check on P_t^f with claim $c = 0$ and table $g^{(t)}$.

If f extends a satisfying assignment, there are tables making this accept with probability 1. If φ is unsatisfiable and f is *any* polynomial in $\mathcal{P}_h^m$, then by Corollary 3.11 and Lemma 3.13 the test accepts with probability at most $\frac{1}{8}+28hm/q$ (here $d\ell/q = 7h\cdot 4m/q$). In the next section we remove the assumption that f is a polynomial.

3.5 The low-degree test

The sum-check of the previous section was proved for polynomials. The verifier holds a table f. We need a *local* test that accepts every $f \in \mathcal{P}_h^m$ and rejects every f far from $\mathcal{P}_h^m$.

3.5.1 The exact statement, and why it is not enough

An *axis-parallel line* of direction i in $\mathbb{F}_q^m$ is $\ell = \{(a_1, \dots, a_{i-1}, x, a_{i+1}, \dots, a_m) : x \in \mathbb{F}_q\}$. There are $m\,q^{m-1}$ of them. If $p \in \mathcal{P}_h^m$ then its restriction to every axis-parallel line is a univariate polynomial of degree $\le h$. The converse is true, and its proof is a good warm-up.

Lemma 3.15 (Axis-parallel lines suffice, exact version)**.** *Let $f\colon \mathbb{F}_q^m \to \mathbb{F}_q$ be such that the restriction of f to every axis-parallel line is a polynomial of degree $\le h$ (with $h < q$). Then $f \in \mathcal{P}_h^m$.*

Proof. Induction on m; $m = 1$ is the hypothesis. Fix distinct $c_0, \dots, c_h \in \mathbb{F}_q$. For each j the function $f(\cdot, c_j)$ on $\mathbb{F}_q^{m-1}$ satisfies the hypothesis in $m-1$ variables, so it is a polynomial $p_j \in \mathcal{P}_h^{m-1}$. Let λ_j be the univariate Lagrange polynomials of degree h for the nodes $c_0, \dots, c_h$ and define $G(x', x_m) = \sum_{j=0}^{h} \lambda_j(x_m)\, p_j(x') \in \mathcal{P}_h^m$. For each fixed x', the functions $f(x', \cdot)$ and $G(x', \cdot)$ are both univariate polynomials of degree $\le h$ and they agree at the $h+1$ points $c_0, \dots, c_h$; hence they are equal. So $f = G$ everywhere. □

As a sanity check, take $q = 5$, $h = 1$, $m = 2$. A pruned exhaustive search through the 5^{25} functions $\mathbb{F}_5^2 \to \mathbb{F}_5$ finds exactly $625 = 5^{(h+1)^2}$ with all rows and all columns of degree ≤ 1: precisely the 5^4 polynomials of $\mathcal{P}_1^2$, as the lemma predicts.

Exact versus robust

Lemma 3.15 is a statement about *perfect* functions. A verifier can only look at a few lines, and can at best find that *most* of the lines it looked at are low-degree and *mostly* agree where they meet. The theorem the test needs is the *robust* form: if the restrictions to most axis-parallel lines are consistent with each other and with f, then f is *close* to a polynomial. All the difficulty of the Arora–Safra analysis sits here, and it is the reason the field size is cubic in the degree.

3.5.2 The test of Arora and Safra

A *decomposition* is a table g that assigns to every axis-parallel line ℓ a univariate polynomial $g[\ell]$ of degree $\le h$ (an entry of $O(h \log q)$ bits). For $1 \le i \le m$ and $a \in \mathbb{F}_q^m$ let $g_i(a) = g[\ell_i(a)](a_i)$, where $\ell_i(a)$ is the line of direction i through a. A *checkpoint* is a pair (i, a) with $1 \le i < m$; the decomposition is *consistent* at it if $g_i(a) = g_{i+1}(a)$. Note that a genuine decomposition of a polynomial p ($g[\ell] = p|_\ell$) is consistent everywhere and $g_i = p$ for all i.

Definition 3.16 (Low-degree test)**.** Given $f\colon \mathbb{F}_q^m \to \mathbb{F}_q$, the closeness parameter $\delta < 0.1$ and a decomposition g:

(i) choose $\lceil 4/\delta \rceil$ points $a \in \mathbb{F}_q^m$ uniformly and independently, and check $f(a) = g_1(a)$ at each;

(ii) choose $\lceil 8m/\delta \rceil$ checkpoints (i, a) uniformly and independently, and check $g_i(a) = g_{i+1}(a)$ at each.

Accept iff all checks pass.

The test reads $O(1/\delta)$ values of f and $O(m/\delta)$ entries of g: $O(m)$ queries for constant δ, non-adaptively. (The randomness is $O(m \log q)$ bits per sample, which is reduced to $O(\log q^m)$ in total by pairwise-independent sampling; we omit it.) It accepts every $f \in \mathcal{P}_h^m$ together with its true decomposition.

Lemma 3.17 (Robust consistency; Arora–Safra, Lemma 4.6.4)**.** *Let $m \ge 2$ and $q \ge 100\, h^3 m^2$. If g_1 is not ε-close to $\mathcal{P}_h^m$, then the fraction of checkpoints at which g is inconsistent is at least $\gamma^{2(m-1)} \min\{0.1, \varepsilon\}/(m-1)$, where $\gamma = 1 - \sqrt{h/q}$. Since $\sqrt{h/q} \le 1/(10m)$ we have $\gamma^{2(m-1)} \ge 1 - 2(m-1)/(10m) \ge 0.8$, so this fraction is at least $\min\{0.1, \varepsilon\}/(2m)$.*

We do not prove Lemma 3.17. The proof is an induction on m. Its base case $m = 2$ is the *bivariate* lemma that Arora et al. (1998) quote as Theorem 69 (due to Arora–Safra): if the row polynomial and the column polynomial of a function $F\colon \mathbb{F}_q^2 \to \mathbb{F}_q$ each agree with F

on all but an η fraction of points ($\eta \le \eta_0$) and $q \ge c\,h^3$, then F is 4η-close to a polynomial of degree $\le h$ in each variable. This is the robust analogue of the exact Lemma 3.15 with $m = 2$, and it is where the *cubic* dependence on h comes from. Polishchuk and Spielman (1994) later gave a proof that needs only $q = \Omega(h)$.

Corollary 3.18 (Soundness of the test)**.** *Let $q \ge 100\,h^3 m^2$ and $m \ge 2$. If f is not δ-close to $\mathcal{P}_h^m$ ($\delta < 0.1$), then for* every *decomposition g the test of Definition 3.16 rejects with probability at least $3/4$.*

Proof. If $\Delta(f, g_1) \ge \delta/2$, part (i) fails to reject with probability at most $(1 - \delta/2)^{4/\delta} \le e^{-2} \approx 0.135 < \frac{1}{4}$. Otherwise $\Delta(f, g_1) < \delta/2$, so g_1 is not $\delta/2$-close to $\mathcal{P}_h^m$ (else f would be δ-close by the triangle inequality). By Lemma 3.17 with $\varepsilon = \delta/2 \le 0.05$ a fraction at least $\delta/(4m)$ of the checkpoints is inconsistent, and part (ii) misses all of them with probability at most $(1 - \delta/4m)^{8m/\delta} \le e^{-2} < \frac{1}{4}$. □

Remark. With $\delta = 0.01$ the two failure probabilities are ≤ 0.1347 and ≤ 0.1353 respectively (computed from the formulas above). The older analysis of this test (for $h = 1$ in Feige et al., 1996) required $O(mh)$ queries; Arora and Safra's algebraic argument gets $O(m)$, at the price of $q = \Omega(h^3 m^2)$. The number of queries to the low-degree table is exactly what is later reduced to a constant by Arora et al. (1998); see Section 3.11.

3.6 The basic verifier

We now assemble the three modules. Let $n = (h+1)^m$ with $h \ge \log n$ and $q = \Theta(h^3 m^2)$ a prime power (note that $m < h$ forces $q = O(h^5)$). The verifier V_{poly} for a 3-CNF φ with n variables expects the proof to consist of

component	content	size
f	a function $\mathbb{F}_q^m \to \mathbb{F}_q$ (the claimed extension of a satisfying assignment)	$q^m = n^{O(1)}$ field elements
g	a decomposition for the low-degree test of f	mq^{m-1} polynomials of degree h
$\{\tau^{(t)}\}_{t \in \mathbb{F}_q^{4m}}$	a table of partial sums for the sum-check of P_t^f, one per t	$\le 2q^{8m} = n^{O(1)}$ polynomials of degree $7h$

and works as follows.

Definition 3.19 (The verifier V_{poly})**.** Given the proof components above, V_{poly} does the following.

1. Run the low-degree test (Definition 3.16) on f and g with $\delta = 0.01$; reject if it rejects.
2. Choose $t \in \mathbb{F}_q^{4m}$ uniformly and run the sum-check (Definition 3.12) for $B = P_t^f$ (that is, with the values of f in place of A), $H = \{0..h\}$, $\ell = 4m$, $d = 7h$, $c = 0$, and table $\tau^{(t)}$; reject if it rejects. Whenever the sum-check needs f at a point w, it reads $f(w)$.
3. Otherwise accept.

Theorem 3.20 (The basic verifier; Arora–Safra, Theorem 3.5(a))**.** *V_{poly} is a non-adaptive verifier for* 3SAT *with perfect completeness, soundness $< \frac{1}{4}$, using $O(\log n)$ random bits and $O(m)$ queries, each to an entry of $O(h \log h)$ bits, and with decision time* $\mathrm{poly}(h)$.

Proof. Completeness. If φ is satisfiable, let f be the extension of a satisfying assignment, g its true decomposition, and $\tau^{(t)}$ the honest table for P_t^f. The low-degree test accepts by completeness, and the sum-check accepts by Lemma 3.13(a), because $\sum_{y \in \mathbb{K}^{4m}} P_t^f(y) = 0$ by Corollary 3.11.

Soundness. Let φ be unsatisfiable and fix any proof. If f is not 0.01-close to $\mathcal{P}_h^m$, step 1 rejects with probability $\geq 3/4$ (Corollary 3.18). So assume f is 0.01-close, and let $\tilde{f} \in \mathcal{P}_h^m$ be its unique nearest polynomial (Corollary 3.4). Step 2 accepts only if one of three events happens:

(i) $\sum_{\mathbb{K}^{4m}} P_t^{\tilde{f}} = 0$: by Corollary 3.11 this has probability $\leq \frac{1}{8}$ over t (this is where unsatisfiability is used, for the arbitrary polynomial $\tilde{f}$);

(ii) the sum is nonzero but the sum-check does not detect it: probability $\leq d\ell/q = 28hm/q$ by Lemma 3.13(b), which is applied to the genuine polynomial $B = P_t^{\tilde{f}}$;

(iii) the sum-check reads $f(w_j)$ at one of three points w_1, w_2, w_3 with $f(w_j) \neq \tilde{f}(w_j)$: each w_j is uniform (Lemma 3.7(c)), so by the union bound this has probability $\leq 3 \cdot 0.01$.

Altogether the acceptance probability is at most $\frac{1}{8} + 0.03 + 28hm/q < \frac{1}{4}$ (for $q \geq h^3 m^2$ and $h \geq \log n$ the last term is $o(1)$).

Parameters. The random bits are those of t, the point $a \in \mathbb{F}_q^{4m}$ of the sum-check, and the samples of the low-degree test: $O(m \log q) = O(m \log h) = O(\log n)$ in total, since $(h+1)^m = n$. The low-degree test reads $O(m)$ entries and the sum-check $4m$ entries of $\tau^{(t)}$, plus three values of f. Entries have $O(h \log q) = O(h \log h)$ bits. Before reading the proof, the verifier computes from φ and (t, a) everything about $P_t^f(a)$ that does not involve f: $\hat{\chi}_j, \hat{\zeta}_j, R_t$ at the relevant points, which takes $\mathrm{poly}(n)$ time; after reading, it does $\mathrm{poly}(m, h, \log q) = \mathrm{poly}(h)$ field operations. All positions queried are determined by the random string alone: the verifier draws all its randomness first, so it can read the entries of the low-degree test and of the sum-check in one batch. This is the point of using sum-check *tables* rather than the interactive protocol. □

Remark. Note how the verifier avoids a circularity. The sum-check needs $\tilde{f}$ at three points, but only f is available. The three points are *uniform*, hence outside the 1% of disagreement with probability ≥ 0.97. This is the "never evaluate at a predictable point" principle of Section 3.2, and it is the reason the arithmetization of Lemma 3.7(c) was designed so that each w_j is uniform.

Theorem 3.20 is already a PCP: choosing $h + 1 = n^{1/m}$ for a constant m it has $O(1)$ queries. The catch is the *size* of the queried symbols: $O(h \log h) = n^{\Omega(1/m)}$ bits each. To get a verifier that reads *bits* one must re-encode every queried symbol and check it without reading it. That is the job of composition.

3.7 Normal form verifiers and split-encoded assignments

Suppose an *outer* verifier V_1 has a decision circuit C_w (on the symbols it queried under random string w), of size T much smaller than the proof. Instead of reading the queried

symbols $P[i_1(w)], \ldots, P[i_Q(w)]$ in full, we would like to check "C_w accepts them" by running a second, *inner* verifier V_2 on the formula of C_w. Two requirements:

- the inner verifier must work when the *input assignment is spread over several separate pieces*, each of which is held in some encoded form in the proof (the Q queried symbols and a scratch string);
- the inner verifier must read only a few *bits* of each piece, since reading the pieces in full is what we want to avoid.

This is formalized in the following definition (a simplified form of Definition 3.2 of Arora and Safra (1998)).

Definition 3.21 (Normal form verifier)**.** An (r, s, q, t)-*constrained normal form verifier* V_2 has an associated encoding scheme σ (bit strings to codewords of minimum distance $\delta_{\min}$ over an alphabet of $2^{O(s)}$ symbols). Given a 3-CNF ψ with N variables and an integer p dividing N, it is given access to a proof $z_1 \sharp \cdots \sharp z_p \sharp \pi$, where each z_i is a word over the alphabet and π is auxiliary, and must satisfy:

- *(completeness)* if each $z_i = \sigma(a_i)$ for $a_i \in \{0,1\}^{N/p}$ and $a_1 \circ \cdots \circ a_p$ satisfies ψ, there is a π with acceptance probability 1;
- *(soundness)* if some z_i is not $\delta_{\min}/3$-close to a codeword, or if each z_i is close to a codeword $\sigma(a_i)$ but $a_1 \circ \cdots \circ a_p$ does not satisfy ψ, then every π is accepted with probability $< \frac{1}{2}$;
- *(resources)* it uses $O(r(N))$ random bits and alphabet symbols of $O(s(N))$ bits, makes $O(p + q(N))$ non-adaptive queries, and has decision time $O(p\, t(N))$.

The unusual feature is the query count $O(p + q(N))$ rather than $O(p \cdot q(N))$: the inner verifier reads $O(1)$ symbols of each piece plus $q(N)$ symbols of the *shared* auxiliary table. Proposition 3.3 of Arora and Safra (1998) records the trivial case $p = 1$: a normal form verifier gives a verifier for 3SAT with the same resources.

This notion is the ancestor of the *PCP of proximity* (Ben-Sasson et al., 2006) and of the *assignment tester* (Dinur and Reingold, 2006), which isolate exactly the property used here—"a tester that looks at a few places of an encoded input and a proof"—and which are what Dinur (2007) composes with in Lecture 6.

3.7.1 How V_{poly} becomes a normal form verifier

By Theorem 3.20, V_{poly} is a verifier with parameters $(\log n,\ h \log h,\ m,\ \text{poly}(h))$ for r, s, q, t. Two additions turn it into a normal form verifier, and both are instances of the exact-versus-robust pattern.

The encoding. Take σ to be the polynomial extension: $\sigma \colon \{0,1\}^N \to \mathcal{P}_h^m$ with $N = (h+1)^m$, a code of minimum distance $\geq 1 - hm/q > \frac{1}{2}$.

Split encodings. Suppose $N = (h+1)^m$ and $p = (h+1)^{m-\ell}$, so that each of the p pieces has $N/p = (h+1)^\ell$ bits. The proof contains $f \colon \mathbb{F}_q^m \to \mathbb{F}_q$ (the encoding of the whole assignment, for the sum-check as before) and $f_1, \ldots, f_p \colon \mathbb{F}_q^\ell \to \mathbb{F}_q$ (the encodings of the pieces), and must check the *concatenation property*: for $i \in \mathbb{K}^{m-\ell}$ and $u \in \mathbb{K}^\ell$,

$$f(i, u) = f_i(u). \tag{3.1}$$

Lemma 3.22 (Concatenation)**.** *Let $f \in \mathcal{P}_h^m$ and $f_1, \dots, f_p \in \mathcal{P}_h^\ell$ (indexed by $i \in \mathbb{K}^{m-\ell}$). Then (3.1) holds if and only if, as polynomials,*

$$f(x_1, \dots, x_m) \;=\; \sum_{i \in \mathbb{K}^{m-\ell}} \Lambda_i(x_1, \dots, x_{m-\ell})\, f_i(x_{m-\ell+1}, \dots, x_m),$$

where Λ_i is the Lagrange basis polynomial of Lemma 3.5 for the cube $\mathbb{K}^{m-\ell}$.

Proof. Let G denote the right-hand side; $G \in \mathcal{P}_h^m$. On a cube point (i', u) we have $\Lambda_i(i') = \mathbf{1}[i = i']$, so $G(i', u) = f_{i'}(u)$. If (3.1) holds, f and G agree on $\mathbb{K}^m$ and are equal by the uniqueness in Lemma 3.5. Conversely, if $f = G$ then $f(i', u) = G(i', u) = f_{i'}(u)$. □

So the verifier tests the identity $f(x) = \sum_i \Lambda_i(x') f_i(x'')$ at $O(1)$ random points $x = (x', x'') \in \mathbb{F}_q^m$. Each test reads $f(x)$ and one value $f_i(x'')$ for each of the p pieces: $O(1)$ values per piece, $O(p)$ in all, which is the source of the $O(p + q)$ in Definition 3.21. If the functions are polynomials but the identity fails, then $f - G$ is a nonzero polynomial and the test catches it with probability $\geq 1 - hm/q$ per sample (Lemma 3.3). Arora and Safra prove the *robust* version (their Theorem 4.4.2): if f is 0.01-close and the test accepts with probability $> \frac{1}{4}$, then each f_i is 0.03-close and the concatenation property holds for the nearest polynomials. The proof rests on a second robust lemma (their Lemma 5.1.1, proved with the same field-size hypothesis as Lemma 3.17): a function G that is 0.02-close to a polynomial and all of whose restrictions $x' \mapsto G(x', x'')$, for fixed x'', are polynomials of degree $\leq h$, has the property that every restriction $x'' \mapsto G(x', x'')$ is 0.03-close to a polynomial—namely to the corresponding restriction of the nearest polynomial $\tilde{G}$. Applied to $G = \sum_i \Lambda_i f_i$ it yields that each f_i is 0.03-close, with $\tilde{f}_i = \tilde{f}(i, \cdot)$.

Pitfall: decoding must produce bits

In the definition of a normal form verifier the decoded pieces $a_i = \sigma^{-1}(\text{nearest codeword to } z_i)$ must be *bit strings*, and ψ is evaluated on their concatenation. By the pitfall after Lemma 3.7, the arithmetization alone only certifies satisfiability of the rounded assignment; to meet the definition the verifier adds a Booleanity check ($A(A-1)$ vanishing on the cube) to the sum-check, which changes only the constants.

3.8 The composition lemma

Let V_1 be a verifier with random bits R, alphabet of 2^S symbols, Q queries, decision time T, perfect completeness and soundness $\frac{1}{4}$. For a random string $w \in \{0,1\}^R$ it queries positions $i_1(w), \dots, i_Q(w)$ of its proof P, and accepts iff a circuit C_w of size $O(T^2)$ accepts $P[i_1(w)] \circ \cdots \circ P[i_Q(w)]$. By the Cook–Levin theorem for circuits this is equivalent to the satisfiability of a 3-CNF c_w of size $\tau = O(T^2)$ over the bits of the Q symbols and of a witness string y_w. We append the strings y_w ($w \in \{0,1\}^R$) to P as extra symbols and let V_1 read y_w as a $(Q{+}1)$st query. After this harmless change, writing $i_{Q+1}(w)$ for the position of y_w,

$$V_1 \text{ accepts } P \text{ on } w \iff P[i_1(w)] \circ \cdots \circ P[i_{Q+1}(w)] \text{ satisfies } c_w. \tag{3.2}$$

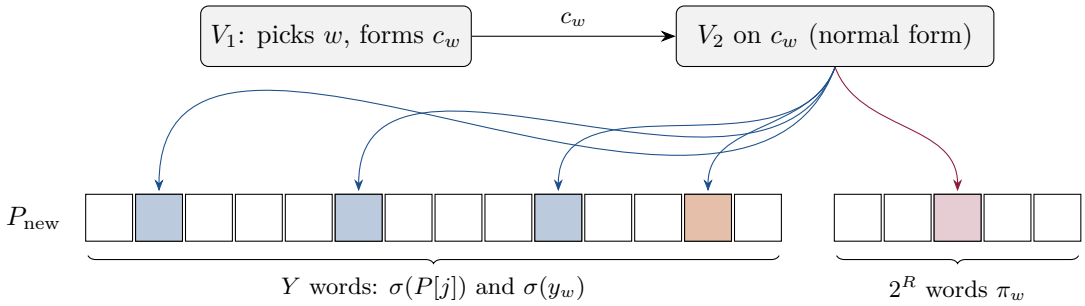


Figure 3.1: The composed verifier V_{new} (Lemma 3.23). The proof consists of encodings $\sigma(P[j])$ of the symbols of V_1's proof (including the witness strings y_w) and one auxiliary word π_w per random string w of V_1. On random string w the verifier V_1 would read the Q blue words and the orange word in full; V_{new} instead hands them to V_2 as a split-encoded assignment to c_w and reads only a few symbols of each.

Lemma 3.23 (Composition; Arora–Safra, Lemma 3.4)**.** *Let V_1 be as above, with parameters (R, S, Q, T), and let V_2 be an (r, s, q, t)-constrained normal form verifier (Definition 3.21). Then the same language has a verifier V_{new} with perfect completeness and soundness $< \frac{1}{2}$, and with*

$$\begin{aligned} &\textit{random bits } R + O(r(\tau)), && \textit{alphabet bits } O(s(\tau)), \\ &\textit{queries } O(Q + q(\tau)), && \textit{decision time } O(Q\,t(\tau)), \end{aligned} \qquad \tau = O(T^2).$$

Proof. Replacing V_1 and V_2 by two independent repetitions of themselves we may assume that both have soundness $\frac{1}{4}$ (this affects only constants). Let σ be the encoding of V_2 and write Y for the number of symbols of V_1's (extended) proof.

The verifier. V_{new} expects a proof P_{new} of $Y + 2^R$ words; the word $P_{\text{new}}[j]$ for $j \le Y$ is supposed to be $\sigma(P[j])$, and $P_{\text{new}}[Y + w]$ is an auxiliary proof π_w. On input x it (1) chooses $w \in \{0,1\}^R$ and computes c_w and $i_1(w), \dots, i_{Q+1}(w)$ without reading the proof; (2) chooses a random string w' for V_2 and runs V_2 on c_w with the $(Q{+}1)$-part split-encoded assignment $z_j = P_{\text{new}}[i_j(w)]$ $(1 \le j \le Q+1)$ and auxiliary proof $\pi_w = P_{\text{new}}[Y + w]$. It reads only those symbols of the z_j and of π_w that V_2 queries under w', and accepts iff V_2 does.

Resources. The random bits are those of w and w'. Everything else is that of V_2 run with $p = Q + 1$ parts on a formula of size τ: alphabet $O(s(\tau))$, queries $O(p + q(\tau))$, decision time $O(p\,t(\tau))$. The queries depend only on (w, w'), so V_{new} is non-adaptive; this is why non-adaptivity was built into every definition.

Completeness. If $x \in L$ take a proof P with V_1 accepting on every w, and put $P_{\text{new}}[j] = \sigma(P[j])$. By (3.2) the concatenation of the $P[i_j(w)]$ satisfies c_w, so by completeness of V_2 there is π_w accepted with probability 1; place it at $Y + w$.

Soundness. Let $x \notin L$ and suppose P_{new} is accepted with probability $\ge \frac{1}{2}$. Define $P[j] = \sigma^{-1}(N(P_{\text{new}}[j]))$, where N is the nearest codeword (an arbitrary string if it is undefined). Let $p_1 = \Pr_w[V_1 \text{ accepts } P \text{ on } w] < \frac{1}{4}$. If V_1 rejects P on w then by (3.2) the decoded concatenation $P[i_1(w)] \circ \cdots \circ P[i_{Q+1}(w)]$ does not satisfy c_w, so either some z_j

is not close to a codeword or the decoded assignment is unsatisfying. By the soundness of V_2, V_{new} accepts on such w with probability $< \frac{1}{4}$, *whatever* π_w *is*. Hence

$$\Pr[V_{\text{new}} \text{ accepts}] \;<\; p_1 + (1-p_1)\cdot\tfrac{1}{4} \;<\; \tfrac{1}{4} + \tfrac{3}{4}\cdot\tfrac{1}{4} \;=\; \tfrac{7}{16} \;<\; \tfrac{1}{2},$$

a contradiction. □

Pitfall: what composition does *not* do

Composition does not make a *hard* instance easier; it makes the *queries* cheaper by moving the cost into the proof length and the verifier's computation. Also, the lemma needs V_2 to *check an encoded input*. A verifier with the same parameters that must read its input in the clear would give nothing: the whole point is that V_{new} never reads a z_j in full. This is the reason normal form (Section 3.7), and not just a verifier with small parameters, is the right interface.

3.9 The parameter chase

We now put the pieces together. Throughout, n is the formula size, $L = \log n$, and all logarithms are base 2.

Step 1. *Outer verifier.* Apply Theorem 3.20 with $h+1 = 2^{\sqrt{L}}$ and $m = \sqrt{L}$ ($h \geq \log n$ holds for large n). Call it $V^{(0)}$:

$$(R, S, Q, T) \;=\; (O(L),\; 2^{O(\sqrt{L})},\; \sqrt{L},\; 2^{O(\sqrt{L})}).$$

Step 2. *Inner verifier.* By Section 3.7 (the concatenation test and the Booleanity check added), the same verifier with the same parameter choice is, at input size N, an (r, s, q, t)-constrained normal form verifier with $(r, s, q, t)(N) = (\log N,\; 2^{O(\sqrt{\log N})},\; \sqrt{\log N},\; 2^{O(\sqrt{\log N})})$.

Step 3. *Compose with $V^{(0)}$ as inner verifier.* Apply Lemma 3.23. Since $T = 2^{O(\sqrt{L})}$, $\tau = O(T^2)$ has $\log \tau = O(\sqrt{L})$, and

$$\begin{aligned}
\text{random bits} &: L + O(\log \tau) = O(L),\\
\text{alphabet bits} &: 2^{O(\sqrt{\log \tau})} = 2^{O(L^{1/4})},\\
\text{queries} &: O(\sqrt{L} + \sqrt{\log \tau}) = O(\sqrt{L}),\\
\text{decision time} &: \sqrt{L}\cdot 2^{O(\sqrt{\log \tau})} = 2^{O(L^{1/4})}.
\end{aligned}$$

Step 4. *Iterate.* Each further composition with $V^{(0)}$ replaces the exponent e in $2^{O(L^e)}$ by $e/2$ and leaves queries and randomness unchanged up to constant factors. After k compositions the verifier has alphabet bits and decision time $2^{O(L^{2^{-(k+1)}})}$, $O(\sqrt{L})$ queries, $O(L)$ random bits.

Step 5. *Final composition with V_{savior}.* Apply Theorem 3.20 with $h = \Theta(\log N)$ and $m = \log N / \log\log N$ at input size N: it is a normal form verifier with $(r, s, q, t)(N) = (\log N,\; \log N \log\log N,\; \log N/\log\log N,\; \text{poly}\log N)$. Composing with it, where $\log \tau = O(L^{2^{-(k+1)}})$, gives

$$\begin{aligned}
\text{alphabet bits} &= \log \tau \log\log \tau = L^{2^{-(k+1)}+o(1)},\\
\text{queries} &= O(\sqrt{L}),\\
\text{decision time} &= \sqrt{L}\cdot \text{poly}(\log \tau) = \text{poly}(L).
\end{aligned}$$

verifier	random bits	queries (symbols)	bits per symbol	decision time
$V^{(0)}$ (Step 1)	$O(L)$	$\sqrt{L}$	$2^{O(L^{1/2})}$	$2^{O(L^{1/2})}$
$V^{(1)} = V^{(0)} \circ V^{(0)}$	$O(L)$	$O(\sqrt{L})$	$2^{O(L^{1/4})}$	$2^{O(L^{1/4})}$
$V^{(2)} = V^{(1)} \circ V^{(0)}$	$O(L)$	$O(\sqrt{L})$	$2^{O(L^{1/8})}$	$2^{O(L^{1/8})}$
$V^{(k)}$	$O(L)$	$O(\sqrt{L})$	$2^{O(L^{2^{-(k+1)}})}$	$2^{O(L^{2^{-(k+1)}})}$
$V^{(k)} \circ V_{\text{savior}}$	$O(L)$	$O(\sqrt{L})$	$L^{2^{-(k+1)}+o(1)}$	$\text{poly}(L)$

Table 3.2: The parameter chase of Section 3.9 ($L = \log n$). Each composition square-roots the exponent of the alphabet size; the number of queries is stuck at $\sqrt{L}$ because it is inherited from the outer verifier.

Proof of Theorem 3.1. Given $\varepsilon > 0$ choose k with $2^{-(k+1)} \le \varepsilon/2$, a constant. The verifier of Table 3.2 after k compositions and the final one with V_{savior} uses $O(L)$ random bits, makes $O(\sqrt{L})$ queries to symbols of $L^{\varepsilon/2+o(1)}$ bits each, so reads $L^{1/2+\varepsilon/2+o(1)} \le L^{1/2+\varepsilon}$ bits of the proof, has perfect completeness and soundness $\frac{1}{2}$. Hence 3SAT $\in \mathsf{PCP}[\log n, \log^{1/2+\varepsilon} n]$ and so $\mathsf{NP} \subseteq \mathsf{PCP}[\log n, \log^{1/2+\varepsilon} n]$. The reverse inclusion was noted at the beginning of the lecture. □

Each step of the chase hides a constant in $\tau = O(T^2)$ and in $O(Q + q(\tau))$; the next section is about why that matters.

What is proved here and what is quoted

Proved in this lecture, from scratch: Schwartz–Zippel, polynomial extension, the arithmetization (Lemma 3.7) and zero-testers (Lemma 3.9), the sum-check (Lemma 3.13), the exact low-degree statement (Lemma 3.15), the concatenation identity (Lemma 3.22), the composition lemma (Lemma 3.23), and the whole parameter chase. **Quoted:** the two *robust* algebraic lemmas, Lemma 3.17 (consistency of a decomposition) and the robust concatenation theorem (Theorem 4.4.2 of Arora and Safra, 1998), both of which rely on the bivariate lemma of Arora and Safra and on the cubic field size. Everything else in the text is bookkeeping around them.

3.10 Consequence: Clique within $2^{\log^{1/2-\varepsilon} N}$ is NP-hard

Proof of Corollary 3.2. Let $0 < \varepsilon_1 < \varepsilon$. By Theorem 3.1 there is a verifier V for 3SAT with $O(\log n)$ random bits and $q = \log^{1/2+\varepsilon_1} n$ queries, perfect completeness and soundness $\frac{1}{2}$. Amplify it by the random walk on an explicit constant-degree expander on the set of random strings (Exercise 2.6; Ajtai et al., 1987): $k = \lceil \log n / q \rceil$ steps use $O(\log n) + O(k) = O(\log n)$ random bits, make $kq = O(\log n)$ queries, and give soundness $2^{-\Omega(k)} = 2^{-\Omega(\log^{1/2-\varepsilon_1} n)}$. The FGLSS graph H_x of the amplified verifier (Definition 2.18) has at most $2^{O(\log n)} = n^{O(1)} =: N$ vertices, and by Theorem 2.19

$$x \in \text{3SAT} \Rightarrow \omega(H_x) = 2^r, \qquad x \notin \text{3SAT} \Rightarrow \omega(H_x) \le 2^r \cdot 2^{-\Omega(\log^{1/2-\varepsilon_1} n)}.$$

The gap is $2^{\Omega(\log^{1/2-\varepsilon_1} N)}$, which exceeds $2^{c \log^{1/2-\varepsilon} N}$ for every constant c and large N because $\varepsilon_1 < \varepsilon$. An approximation algorithm within $2^{c \log^{1/2-\varepsilon} N}$ would therefore decide 3SAT in polynomial time. □

Compare with Corollary 2.23, which reached $2^{\Omega((\log N)^{1/(1+\varepsilon)})}$ from a constant-query PCP and quasi-polynomial time. Here the PCP has $\log^{1/2+\varepsilon_1} n$ queries, but the amplification is randomness-efficient, so the reduction takes *polynomial* time: *this* is the improvement that earned the paper its title.

3.11 Why it stops at $\log^{1/2+\varepsilon} n$, and how to reach $O(1)$

The chase of Table 3.2 has two bottlenecks.

Queries are inherited, never reduced. By Lemma 3.23 the new number of queries is $O(Q + q(\tau)) \geq Q$. Composition shrinks the *symbols* but never the number of symbols queried, and V_{poly} has $Q = m = \log n / \log(h+1)$. To have few queries one needs h large, hence symbols of $\Omega(h)$ bits, hence *many* compositions to make them short.

Constants compound. With $h+1 = n^{1/m}$ and m constant, the alphabet has $n^{\Omega(1/m)}$ bits and shrinking it to $\operatorname{polylog} n$ takes $\Theta(\log\log n)$ compositions, each multiplying the hidden constants in $O(Q+q(\tau))$ by a factor $c > 1$. After $\Theta(\log\log n)$ rounds the product is $c^{\Theta(\log\log n)} = \operatorname{polylog} n$ queries. This is exactly the caution in the paper: "We do not wish to invoke the Composition Lemma more than $O(1)$ times because it hid constant factors in its $O(\cdot)$ notation and those constant factors could grow very fast." The $\log^{1/2+\varepsilon} n$ of Theorem 3.1 is what survives when one limits oneself to $O(1)$ compositions.

The remedy of ALMSS. According to Section 6 of Arora and Safra (1998), Arora et al. (1998) proceed in three steps.

1. They construct a new outer verifier that reads only $O(1)$ entries of the polynomial tables (not $O(m)$) and has polylogarithmic decision time. The two ingredients are a more efficient low-degree test, based on the main lemma of Arora–Safra (the bivariate Lemma 3.17's base case) and on the work of Rubinfeld and Sudan (1992), and a *parallelization* step in the spirit of Lapidot and Shamir (1991) and Feige and Lovász (1992) (a close relative of the constraint–variable game of Lecture 2, in which the answers to all queries of a random string are bundled into one symbol), which collapses several queries into one.
2. They compose this verifier with itself. The result reads $O(1)$ symbols of size $O(\log\log n)$.
3. They compose that with a verifier, built from ideas of Blum et al. (1990), which is *wasteful in randomness* (polynomial in the proof size) but makes only $O(1)$ *bit* queries. Since by now the input to the last verifier has size $\operatorname{polylog} n$, its wastefulness is harmless. The result is a $(\log n, 1)$-restricted verifier.

This is Lecture 4. The last of the three verifiers is based on the *Hadamard code* and linearity testing (Blum et al., 1990); the *long code* of Bellare et al. (1998), the workhorse of Lecture 7, is a more redundant relative of the Hadamard code.

The shape of the argument, in one paragraph

A satisfying assignment is encoded by a low-degree polynomial, a code in which a 1%-corrupted word still determines the codeword. Satisfiability becomes one vanishing sum, which the sum-check verifies at a single random point of a polynomial that is itself computed

from three random points of the table. A low-degree test certifies that the table is close to a polynomial. Because the verifier only ever asks for *uniformly distributed* points, corruption of a small fraction of the table is invisible to it. This yields a verifier with $O(\log n)$ random bits and few, but long, queries. Composition then re-checks each long symbol *in encoded form* with a second such verifier, trading symbol length for proof length. Iterating, one reaches sublogarithmic query bits; getting to $O(1)$ needs a better outer verifier and a Hadamard-code inner verifier (Lecture 4), or Dinur's entirely different amplification (Lecture 6).

Exercises

Exercise 3.1 ([⋆⋆]). Show that Lemma 3.3 is nearly tight: for $d < q$ and every m there is a nonzero $p \in \mathcal{P}_d^m$ vanishing on exactly a $1-(1-d/q)^m$ fraction of $\mathbb{F}_q^m$. Compare $1-(1-d/q)^m$ with dm/q.

Exercise 3.2 ([⋆]). Compute the multilinear extension $\hat{s}$ of the majority function $s\colon \{0,1\}^3 \to \{0,1\}$ and evaluate it at $(2,2,2)$ over $\mathbb{F}_7$. Verify that $\hat{s}$ has degree 1 in each variable but total degree 3.

Exercise 3.3 ([⋆]). In Example 3.8 take instead $a = (1,1,1,1)$. Which clause is violated? Find all nonzero points of $g_{\hat{a}}$ on $\mathbb{K}^8$ and their values, and compute $\Pr_t[F(t) = 0]$ exactly as in Example 3.10.

Exercise 3.4 ([⋆⋆]). Let $B \equiv 0$ be the zero polynomial in ℓ variables, $H \subseteq \mathbb{F}_q$, $c \neq 0$. Show that the best table convinces the sum-check with probability *exactly* $1-(1-d/q)^\ell$ provided that there is a polynomial of degree $\leq d$ with d distinct roots in $\mathbb{F}_q$ whose sum over H is c. Exhibit one for $q = 7$, $d = 2$, $H = \{0,1\}$, $c = 1$. (*Hint: scaling a polynomial by a nonzero constant keeps its roots and scales its sum over H.*)

Exercise 3.5 ([⋆]). Show that Lemma 3.15 fails if lines of one direction are omitted: for $m = 2$ let $f(x_1,x_2) = \varphi(x_1)$ where $\varphi\colon \mathbb{F}_q \to \mathbb{F}_q$ is a function of degree greater than h. Every row $f(x_1,\cdot)$ is constant, but $f \notin \mathcal{P}_h^2$. Prove that $\Delta(f,\mathcal{P}_h^2) = \Delta(\varphi,\mathcal{P}_h^1)$, so f can be as far from $\mathcal{P}_h^2$ as a univariate function can be from $\mathcal{P}_h^1$. *Hint: for a fixed polynomial p average over x_2.*

Exercise 3.6 ([⋆⋆]). Suppose an inner verifier has $s(N) = t(N) = N^{1/3}$, $q(N) = O(1)$, $r(N) = \log N$ and the outer verifier has $(R,S,Q,T) = (O(\log n), n^{1/3}, c_0, n^{1/3})$. Compute the parameters after k compositions and find the least k for which the alphabet is $\operatorname{polylog} n$. If each composition multiplies the number of queries by a constant $c > 1$, what is the final number of queries? Compare with Step 4 of Section 3.9.

Exercise 3.7 ([⋆⋆]). Design the Booleanity check mentioned in the pitfalls of Sections 3.3 and 3.7: give a polynomial in $\mathcal{P}_{2h}^m$ that vanishes on $\mathbb{K}^m$ iff A is $\{0,1\}$-valued there, and show how a second sum-check (with its own zero-tester, $k = m$ variables) detects a violation with probability at least $1-3hm/q$. How many extra queries does the verifier make?

Exercise 3.8 ([⋆⋆⋆]). *Reading.* Compare the low-degree test of Section 4.6 of Arora and Safra (1998) with the one analyzed in Section 7 of Arora et al. (1998): what is tested (axis-parallel lines or random lines), how many queries are made, how large the field must be, and which of the two lemmas is the bivariate base case. Which of these differences is responsible for the drop from $O(m)$ to $O(1)$ queries?

Notes and further reading

Where this lecture comes from. Everything is from Arora and Safra (1998): Sections 3 and 4 give the verifier and the composition lemma, Section 5 the algebraic lemmas that we quoted, and Section 6 the account of what followed. The algebraic characterization of satisfying polynomials (Lemma 3.7) is credited there to Babai et al. (1991a), in a form similar in spirit to Babai et al. (1991b), and the zero-testers (Lemma 3.9) to Babai et al. (1991a) and Feige et al. (1996); the sum-check is due to Lund et al. (1992). The names "zero-tester" and "table of partial sums", the form of the composition lemma and our parameter table follow Arora and Safra (1998); the numerical examples (Examples 3.8, 3.10, 3.14) were computed for these notes.

On attributions. We distinguish three statements that are often confused: Arora–Safra proved sub-logarithmic queries, Arora–Lund–Motwani–Sudan–Szegedy constant queries. Section 6 of Arora and Safra (1998) is generous in crediting the second paper and the earlier observation of Szegedy and Sudan that a verifier with small decision time yields a gap for Max-3SAT. The modular formulation of composition (PCPs of proximity, assignment testers) is due to Ben-Sasson et al. (2006) and Dinur and Reingold (2006); Arora and Safra's normal form verifier is the special case used here. The analysis of the low-degree test was subsequently improved by Rubinfeld and Sudan (1992), Polishchuk and Spielman (1994) and Arora and Sudan (1997); none of these is needed for what we do.

A caveat on the text. Two small points in the presentation of Arora and Safra (1998) that we were careful with: the arithmetization certifies rounded satisfiability, so Booleanity must be checked separately when decoding (pitfall after Lemma 3.7); and in the remark after Lemma 4.6.4 the paper writes $\gamma^{2(l-1)} \geq \frac{1}{5}$, which is true but too weak for the conclusion drawn from it for $l > 5$; the correct bound is $\geq \frac{4}{5}$ (Bernoulli's inequality), and it is this that makes the choice of $8m/\delta$ checkpoints in the test sufficient.

What is not covered. We have not proved the robust lemmas, nor the analysis of V_{savior} at small input size, and we have not treated the $\mathsf{PCP}[\log n, (\log \log n)^2]$ refinement mentioned in the paper. The proof of the constant-query theorem is Lecture 4, and the combinatorial alternative is Lecture 6.

LECTURE 4

Constant Queries: Hadamard Codes, Parallelization and the ALMSS Proof

Reorder the compositions, and a wasteful verifier finishes the job

Lecture 3 ended with a verifier that reads $\log^{1/2+\varepsilon} n$ bits. Its bottleneck was not the algebra but the bookkeeping of the composition lemma: composition shrinks *symbols* but never the *number* of symbols read, and the starting verifier reads $\Omega(m) = \Omega(\log n / \log h)$ of them. Arora et al. (1998), writing at the same time as Arora and Safra (both papers appeared at FOCS 1992), found a way around this by changing the *order* of the operations. Their theorem is the one that is usually meant by the PCP theorem.

Theorem 4.1 (Arora–Lund–Motwani–Sudan–Szegedy)**.** *There is a constant q such that* $\mathsf{NP} = \bigcup_{c>0} \mathsf{PCP}[c \log n,\ q]$.

The proof has three ideas, and each gets a section of this lecture.

1. *A wasteful verifier with $O(1)$ bit queries.* Encode the assignment with the *Hadamard code* (all parities) and its tensor square. The proof has length $2^{\Theta(n^2)}$, but a verifier can check it with $O(1)$ queries using the linearity test of Blum, Luby and Rubinfeld, self-correction, and a Freivalds-type test (Sections 4.2–4.3). Used directly this is useless: the proof is exponential. Used as the *innermost* verifier of a composition, on a circuit of size $\operatorname{polylog} \log n$, the exponential is harmless.
2. *Parallelization.* Starting from the Arora–Safra verifier, which makes $\operatorname{polylog} n$ queries, collapse all its queries into $O(1)$ queries by asking for the restriction of the encoded proof to a *curve* through the $\operatorname{polylog} n$ points, at the price of larger answers (Sections 4.4–4.6).
3. *Reorder the composition.* Compose *the parallelized verifier with itself* once (answer size $\operatorname{polylog} n \to \operatorname{polylog} \log n$), then with the Hadamard verifier (answer size $\to 1$). Only two compositions are needed, so the constants do not compound (Section 4.7).

After this lecture you should be able to...

(1) state the outer/inner verifier formalism of ALMSS and prove its composition theorem, and say how it relates to the normal form verifiers of Lecture 3 (Section 4.1);

(2) prove the linearity test of Blum–Luby–Rubinfeld by Fourier analysis, with the sharp bound "rejection probability $\geq$ distance to the nearest linear function" (Section 4.2);

(3) prove the soundness of the Freivalds-type tensor test, and assemble the Hadamard verifier, computing its number of queries and acceptance probabilities on a 5-gate circuit (Section 4.3);

(4) prove the properties of self-correction on lines and of the curve-check, and explain how a curve turns $\operatorname{polylog} n$ queries into one (Sections 4.4–4.5);

(5) carry out the three-step parameter chase of Theorem 4.1 and say which ingredients we quote rather than prove (Section 4.7).

4.1 Outer verifiers, inner verifiers, and composition again

4.1.1 The formalism

Lecture 3 proved the composition lemma for *normal form verifiers*. ALMSS use a formalism that separates the two roles more cleanly and states soundness in terms of decoding. Both are the same idea, and we shall see precisely how they correspond.

Definition 4.2 (Outer verifier)**.** An (r, p, c, a) *outer verifier* V for inputs of length n expects as proof an oracle π whose entries are strings of $a(n)$ bits. On input x it (i) draws $R \in \{0,1\}^{r(n)}$; (ii) computes in time $\mathrm{poly}(n)$ a circuit C_R of size $c(n)$ and $p(n)$ positions $q_1, \dots, q_{p(n)}$; (iii) reads $a_i = \pi(q_i)$; (iv) accepts iff $a_1 \circ \cdots \circ a_p$ satisfies C_R. A language L is in $\mathrm{RPCP}(r, p, c, a, e)$ if such a verifier accepts some oracle with probability 1 when $x \in L$ and every oracle with probability at most e when $x \notin L$.

Since the verifier reads $p(n)\,a(n)$ bits, $\mathrm{RPCP}(r, p, c, a, \frac{1}{2}) \subseteq \mathsf{PCP}[r,\, pa]$. What distinguishes an outer verifier from an arbitrary verifier is *succinctness*: its decision is a small circuit evaluated on the concatenated answers. This is the interface through which an inner verifier is plugged in.

An inner verifier must check "the concatenation of the queried answers satisfies C" *without reading the answers*, so these must be given in an encoded form.

Definition 4.3 (Encoding scheme)**.** An (ℓ, a) *encoding scheme* is a family $E = \{E_m\}$ of maps $E_m \colon \{0,1\}^m \to \{f \colon [\ell(m)] \to \{0,1\}^{a(m)}\}$ together with a *decoding* E_m^{-1} in the other direction with $E_m^{-1}(E_m(x)) = x$ for all x. (Decoding is arbitrary on words that are not codewords.)

Definition 4.4 (Inner verifier system)**.** A $(k; r, p, c, a; e)$ *inner verifier system* is a triple (V, E, E^{-1}). The input is a circuit C of size n with km input nodes; the verifier expects a proof consisting of $k+1$ oracles $X_1, \dots, X_k$ and π, each with answer size $a(n)$. It draws $r(n)$ random bits, computes a circuit C' of size $c(n)$ and $p(n)$ positions in the oracles, reads them, and accepts iff the concatenated answers satisfy C'.

- *Completeness.* If $x_1 \circ \cdots \circ x_k$ ($x_i \in \{0,1\}^m$) satisfies C, then there is π such that V accepts $(E(x_1), \dots, E(x_k), \pi)$ with probability 1.
- *Soundness.* If for some oracles $X_1, \dots, X_k, \pi$ the verifier accepts with probability $> e$, then $E^{-1}(X_1) \circ \cdots \circ E^{-1}(X_k)$ satisfies C.

Compare with Definition 3.21: the X_i are the pieces z_i of the split-encoded assignment, π is the auxiliary proof, and soundness here is the conjunction of the two soundness clauses there ("some piece is far from a codeword" and "the decoded assignment does not satisfy") phrased through the *decoding* map, whose behaviour on non-codewords is part of the scheme. The number of queries is p, in place of $O(p + q)$.

Proposition 4.5 (An inner verifier is an outer verifier)**.** *If a* $(k; r, p, c, a; e)$ *inner verifier system exists then* CKT-SAT $\in \mathrm{RPCP}(r, p, c, a, e)$.

Proof. Concatenate $X_1, \dots, X_k, \pi$ into one oracle. If C is satisfiable, completeness gives a proof accepted with probability 1. If a proof is accepted with probability $> e$, soundness shows that C is satisfiable. Hence for unsatisfiable C every proof is accepted with probability $\le e$. □

Proposition 4.6 (Repetition)**.** *From a $(k; r, p, c, a; e)$ inner verifier system one obtains, for every $l \ge 1$, a $(k; lr, lp, lc, a; e^l)$ inner verifier system.*

Proof. Run V l times with independent random strings and accept iff all runs accept. For fixed oracles the runs are independent, so the acceptance probability is the l-th power of that of a single run: if it is $\ge e^l$ then a single run accepts with probability $\ge e$, and soundness of V applies. Completeness is unchanged. (ALMSS write la for the answer size; a suffices, since the same oracles are queried.) □

4.1.2 Composition

Theorem 4.7 (Composition; ALMSS Theorem 13, after Arora–Safra)**.** *Suppose a $(p; r_1, p_1, c_1, a_1; e_1)$ inner verifier system exists. Then for all functions r, c, a and $0 < e < 1$,*

$$\mathrm{RPCP}(r,\ p,\ c,\ a,\ e) \ \subseteq\ \mathrm{RPCP}(r + r_1(c),\ p_1(c),\ c_1(c),\ a_1(c),\ e + e_1 - ee_1).$$

Proof. Let V_1 be an (r, p, c, a) outer verifier for L and (V_2, E, E^{-1}) the inner system. Fix x of length n and let Y be the number of entries of V_1's oracle. The new verifier V expects an oracle π that consists of $Y + 2^{r(n)}$ *sub-oracles*: $\pi[1], \dots, \pi[Y]$ (supposedly $E(\pi'(1)), \dots, E(\pi'(Y))$ for a proof π' of V_1) and $\pi[Y + R]$ for every $R \in \{0,1\}^{r(n)}$ (an auxiliary proof for the run R).

Program. Draw $R \in \{0,1\}^{r(n)}$ and simulate steps (i)–(ii) of V_1, obtaining a circuit C_R of size $c(n)$ and positions $Q_1, \dots, Q_p$. Draw $R_2 \in \{0,1\}^{r_1(c(n))}$ and run V_2 on input C_R with random string R_2 and oracles $X_i = \pi[Q_i]$ $(i \le p)$ and $\pi_{\mathrm{aux}} = \pi[Y + R]$. Accept iff V_2 accepts. The number of random bits is $r(n) + r_1(c(n))$, and all other parameters are those of V_2 on an input of size $c(n)$; in particular the verdict is a circuit of size $c_1(c(n))$ of the $p_1(c(n))$ answers.

Completeness. Let π' be accepted by V_1 with probability 1 and set $\pi[j] = E(\pi'(j))$. For each R the answers $\pi'(Q_1) \circ \cdots \circ \pi'(Q_p)$ satisfy C_R, so by completeness of V_2 there is an auxiliary proof for R which we place in $\pi[Y + R]$; then V accepts with probability 1.

Soundness. Suppose $x \notin L$ and V accepts some π with probability $> e + e_1 - ee_1$. Define $\pi'(j) = E^{-1}(\pi[j])$ for $j \le Y$. Let $\mathcal{R} = \{R : \Pr_{R_2}[V \text{ accepts on } (R, R_2)] > e_1\}$ and $\rho = |\mathcal{R}|/2^{r(n)}$. Then $\Pr[V \text{ accepts}] \le \rho + (1 - \rho)e_1$, which is increasing in ρ; if $\rho \le e$ this is at most $e + (1 - e)e_1 = e + e_1 - ee_1$, a contradiction. So $\rho > e$. For $R \in \mathcal{R}$, the inner verifier V_2 accepts the oracles $\pi[Q_1], \dots, \pi[Q_p], \pi[Y + R]$ with probability $> e_1$, so by its soundness $E^{-1}(\pi[Q_1]) \circ \cdots \circ E^{-1}(\pi[Q_p]) = \pi'(Q_1) \circ \cdots \circ \pi'(Q_p)$ satisfies C_R, i.e. V_1 accepts π' on R. Hence V_1 accepts π' with probability $> e$, contradicting soundness of V_1. □

Two observations. First, the soundness arithmetic is slightly different from Lemma 3.23 ($e + e_1 - ee_1$ versus $p_1 + (1 - p_1)\frac{1}{4}$); both say "the cheater either fools the outer verifier or fools the inner verifier on the runs where the outer verifier rejects". Second, the roles of the parameters: the number of queries of the composed verifier is $p_1(c)$, that of the *inner* verifier on the *small* circuit. This is what changes the game: if the *inner* verifier

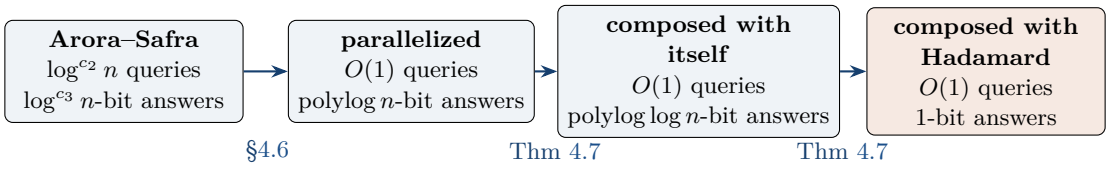


Figure 4.1: The architecture of the ALMSS proof. Random bits stay $O(\log n)$ throughout; only the *answer size* and the *decision circuit* are shrunk by the compositions, and the number of queries is made constant once and for all in the first arrow.

has $O(1)$ queries, so does the composition, whatever the outer verifier's query count p was. In Lecture 3 the inner verifiers were algebraic verifiers whose query count $q(\tau)$ grew with τ (as $\sqrt{\log \tau}$, and as $\log \tau / \log\log \tau$ for V_{savior}).

4.2 The Hadamard code and its local tests

Throughout this section all arithmetic is over $\mathbb{F}_2$ and $z \cdot y = \sum_i z_i y_i$.

Definition 4.8 (Hadamard / parity encoding)**.** For $z \in \mathbb{F}_2^n$ let $L_z \colon \mathbb{F}_2^n \to \mathbb{F}_2$, $L_z(y) = z \cdot y$. The *Hadamard encoding* of z is the table of all 2^n values of L_z. Decoding maps a function $f \colon \mathbb{F}_2^n \to \mathbb{F}_2$ to the string z minimizing $\Delta(f, L_z)$.

Every linear function $\mathbb{F}_2^n \to \mathbb{F}_2$ is some L_z. For $z \neq z'$ the function $L_z + L_{z'} = L_{z+z'}$ is a nonzero linear function, hence equals 1 on exactly half of $\mathbb{F}_2^n$: the code has relative distance $\frac{1}{2}$, so a function within distance $< \frac{1}{4}$ of a codeword decodes to it. This is the code with the *worst rate* one could imagine (2^n bits for n) and the *best local structure*.

4.2.1 Fourier analysis over $\mathbb{F}_2^n$ in one page

Write a function $F \colon \mathbb{F}_2^n \to \mathbb{F}_2$ as $f = (-1)^F \colon \mathbb{F}_2^n \to \{\pm 1\}$, and for $S \in \mathbb{F}_2^n$ let $\chi_S(x) = (-1)^{S \cdot x}$, so that $\chi_S = (-1)^{L_S}$. Define

$$\widehat{f}(S) \;=\; \mathop{\mathbb{E}}_{x}[f(x)\chi_S(x)] \;=\; 1 - 2\Delta(F, L_S).$$

(The second equality: $f\chi_S = +1$ where $F = L_S$ and -1 elsewhere.)

Lemma 4.9 (Orthonormality and Parseval)**.** *(a)* $\mathbb{E}_x[\chi_S \chi_T] = \mathbf{1}[S = T]$. *(b)* $f = \sum_S \widehat{f}(S)\chi_S$. *(c)* $\sum_S \widehat{f}(S)^2 = \mathbb{E}[f^2] = 1$.

Proof. (a) $\chi_S \chi_T = \chi_{S+T}$, and for $U \neq 0$ the function $x \mapsto U \cdot x$ takes each value on half of $\mathbb{F}_2^n$, so $\mathbb{E}[\chi_U] = 0$. (b) The 2^n functions χ_S are orthonormal for the inner product $\langle g, h \rangle = \mathbb{E}[gh]$ on the 2^n-dimensional space of real functions, hence a basis, and the coefficient of χ_S in f is $\langle f, \chi_S \rangle$. (c) Expand $\mathbb{E}[f^2]$ using (a) and (b). □

4.2.2 Linearity testing

Definition 4.10 (Linearity test; Blum–Luby–Rubinfeld)**.** Given $F \colon \mathbb{F}_2^n \to \mathbb{F}_2$: choose $x, y \in \mathbb{F}_2^n$ uniformly and independently; accept iff $F(x) + F(y) = F(x + y)$. (Three queries, $2n$ random bits.)

Theorem 4.11 (Linearity test, sharp form)**.** *If F is linear the test accepts with probability* 1*. In general*

$$\Pr[\mathit{accept}] \;=\; \frac{1}{2} + \frac{1}{2}\sum_{S} \widehat{f}(S)^3,$$

$$\min_{S} \Delta(F, L_S) \;\le\; \Pr[\mathit{reject}] \;\le\; 3 \min_{S} \Delta(F, L_S).$$

Proof. Since $f(x)f(y)f(x+y) = 1$ iff the test accepts, $\Pr[\text{accept}] = \frac{1}{2} + \frac{1}{2}\,\mathbb{E}[f(x)f(y)f(x+y)]$. Expanding each factor by Lemma 4.9(b),

$$\mathop{\mathbb{E}}_{x,y}[f(x)f(y)f(x+y)] = \sum_{S,T,U} \widehat{f}(S)\widehat{f}(T)\widehat{f}(U)\; \mathop{\mathbb{E}}_{x}[\chi_S\chi_U(x)]\; \mathop{\mathbb{E}}_{y}[\chi_T\chi_U(y)] = \sum_{S} \widehat{f}(S)^3,$$

using $\chi_U(x+y) = \chi_U(x)\chi_U(y)$ and Lemma 4.9(a). For the lower bound on rejection: $\widehat{f}(S)^3 = \widehat{f}(S)\cdot\widehat{f}(S)^2 \le \max_T \widehat{f}(T)\cdot\widehat{f}(S)^2$, so by Parseval $\sum_S \widehat{f}(S)^3 \le \max_T \widehat{f}(T)$ and

$$\Pr[\text{reject}] = \frac{1}{2} - \frac{1}{2}\sum_{S} \widehat{f}(S)^3 \;\ge\; \frac{1 - \max_T \widehat{f}(T)}{2} = \min_{T} \Delta(F, L_T).$$

For the upper bound, let L be a nearest linear function; if $F(x)+F(y) \neq F(x+y)$ then one of $F(x), F(y), F(x+y)$ differs from L, and each of these three points is uniform, so $\Pr[\text{reject}] \le 3\Delta(F, L)$. □

Remark. Arora et al. (1998) quote the theorem of Blum et al. (1990) in the weaker form "rejection probability $\delta < 2/9$ implies 2δ-close to linear" and remark that the exact constant may differ from the original; the Fourier-analytic argument above, from the later literature on linearity testing (Bellare et al., 1996), gives δ-closeness for *every* δ, which improves the soundness constants below. Exhaustively, over all 2^8 functions on $\mathbb{F}_2^3$ and all 2^{16} on $\mathbb{F}_2^4$ the identity above holds exactly and the largest ratio $\min_S \Delta/\Pr[\text{reject}]$ is $2/3$ and $8/9$ respectively.

4.2.3 Self-correction and concatenation

A function that is ε-close to linear is correct on most points but the verifier needs its value at *specific* points (for instance a coefficient vector of a circuit). Linearity lets us turn an arbitrary point into two uniform ones.

Proposition 4.12 (Self-correction; BLR)**.** *If F is ε-close to the linear function L then for* every *$x \in \mathbb{F}_2^n$, $\Pr_y[F(x+y) + F(y) \neq L(x)] \le 2\varepsilon$.*

Proof. Both $x+y$ and y are uniformly distributed, so $F(x+y) = L(x+y)$ and $F(y) = L(y)$ each fail with probability $\le \varepsilon$, and when both hold $F(x+y)+F(y) = L(x)+L(y)+L(y) = L(x)$. □

Self-correction is why the verifier may be given a *wrong* table: it never queries the table at a point of its own choosing, only at points that are individually uniform. This is the same principle as in Section 3.2.

A linear function on $\mathbb{F}_2^{n_1+\cdots+n_k}$ is the same thing as a k-tuple of linear functions: $L(x_1,\dots,x_k) = \sum_i L_i(x_i)$ with $L_i(u) = L(0,\dots,u,\dots,0)$ (u in the i-th block). This is the *concatenation* we need to compare an oracle A for the whole assignment with oracles X_i for its pieces.

Proposition 4.13 (Concatenation test)**.** *Let $f\colon \mathbb{F}_2^{n_1+\cdots+n_k} \to \mathbb{F}_2$ and $f'\colon \mathbb{F}_2^{n_i} \to \mathbb{F}_2$ be ε-close to linear g, g' respectively. Test: choose $u \in \mathbb{F}_2^{n_i}$ and $y \in \mathbb{F}_2^{n_1+\cdots+n_k}$ uniformly, let $\iota_i(u)$ be u placed in block i, and accept iff $f(\iota_i(u)+y)+f(y) = f'(u)$. Then (a) if $g_i = g'$ the test accepts with probability 1 when $f = g$, $f' = g'$; (b) if $g_i \neq g'$ it accepts with probability at most $\frac{1}{2} + 3\varepsilon$.*

Proof. (b) Since g_i and g' are distinct linear functions, $g(\iota_i(u)) = g_i(u) \neq g'(u)$ for half of all u. Accepting requires that this event fails, or that self-correction of f at $\iota_i(u)$ returns a wrong value (probability $\leq 2\varepsilon$ by Proposition 4.12), or that $f'(u) \neq g'(u)$ (probability $\leq \varepsilon$). □

4.3 Verifying a circuit with Hadamard codes

4.3.1 A circuit as a system of quadratic equations

Let C be an algebraic circuit (gates $+$ and $\cdot$ over $\mathbb{F}_2$, fan-in 2) with n gates numbered $1, \ldots, n$ and km input nodes $1, \ldots, km$; gate n is the output. For an input $x \in \mathbb{F}_2^{km}$ let $z = C\text{-aug}(x) \in \mathbb{F}_2^n$ be the vector of all gate values (so the first km coordinates are x). Associate to every non-input gate j with input gates j_1, j_2 the equation

$$P_j(z) = z_{j_1} + z_{j_2} + z_j = 0 \text{ (addition)}, \qquad P_j(z) = z_{j_1} z_{j_2} + z_j = 0 \text{ (multiplication)},$$

and add $P_{\text{out}}(z) = z_n + 1 = 0$. These are $n - km + 1$ polynomials of degree ≤ 2.

Lemma 4.14. *$C(x) = 1$ iff there is $z \in \mathbb{F}_2^n$ with $z_{1..km} = x$ and $P_j(z) = 0$ for all j. Moreover then $z = C\text{-aug}(x)$.*

(Immediate: the gate equations force z to be the gate-value vector, and P_{out} says the output is 1.)

Because $z_a^2 = z_a$ over $\mathbb{F}_2$, every polynomial of degree ≤ 2 is $P(z) = \sum_{a,b} c_{ab} z_a z_b + c_0$, i.e.

$$P(z) = \langle z \otimes z,\ c \rangle + c_0, \qquad (z \otimes z)_{ab} = z_a z_b, \quad c \in \mathbb{F}_2^{n^2}. \tag{4.1}$$

This is the key observation: *evaluating every quadratic function of z is a single linear query to the Hadamard encoding $L_{z\otimes z}$ of $z \otimes z \in \mathbb{F}_2^{n^2}$.* The proof therefore consists of L_z (to compare with the input pieces) and $L_{z\otimes z}$ (to evaluate the equations); what must be tested is that the second really is the tensor square of the first.

Lemma 4.15 (Tensor test; Freivalds)**.** *Let $M \in \mathbb{F}_2^{n\times n}$ be nonzero. For uniform $u, v \in \mathbb{F}_2^n$, $\Pr[u^\top M v = 1] \geq \frac{1}{4}$. (In fact it equals $\frac{1}{2} - 2^{-\operatorname{rank}(M)-1}$; Exercise 4.2.)*

Proof. The map $v \mapsto Mv$ is linear and nonzero, so $Mv \neq 0$ with probability $\geq \frac{1}{2}$. For a fixed nonzero vector w, $u^\top w$ is a uniform bit. Multiply. □

4.3.2 The verifier $V^{\oplus}$

Definition 4.16 (Hadamard verifier)**.** Input: a circuit C as above. Proof: functions $X_1, \ldots, X_k\colon \mathbb{F}_2^m \to \mathbb{F}_2$, $A\colon \mathbb{F}_2^n \to \mathbb{F}_2$ and $B\colon \mathbb{F}_2^{n^2} \to \mathbb{F}_2$. (Expected: $X_i = L_{x_i}$, $A = L_z$, $B = L_{z\otimes z}$ for $z = C\text{-aug}(x_1 \circ \cdots \circ x_k)$.) The verifier performs *all* of the following tests with independent randomness, and accepts iff all pass.

1. *Linearity:* the test of Definition 4.10 on A, on B and on each X_i.

2. *Concatenation:* for $i = 1, \dots, k$ the test of Proposition 4.13 on (A, X_i), with block i the i-th chunk of m input coordinates.
3. *Tensor:* draw $z_1, z_2 \in \mathbb{F}_2^n$ and $w \in \mathbb{F}_2^{n^2}$ uniformly; check $A(z_1)A(z_2) = B(z_1 \otimes z_2 + w) + B(w)$.
4. *Circuit:* draw $r \in \mathbb{F}_2^{n-km+1}$ and $w \in \mathbb{F}_2^{n^2}$ uniformly; let $c = \sum_j r_j c^{(j)}$ and $c_0 = \sum_j r_j c_0^{(j)}$ be the coefficients, in the sense of (4.1), of $\sum_j r_j P_j$; check $B(c + w) + B(w) = c_0$.

Theorem 4.17 (The Hadamard inner verifier; ALMSS Theorem 15)**.** $(V^{\oplus}, E^{\oplus}, (E^{\oplus})^{-1})$ *is a* $(k;\ r,\ 6k+12,\ O(k),\ 1;\ \frac{19}{20})$ *inner verifier system with* $r = 4n^2 + (k+5)n + 2km + 1$ *random bits.*

Proof. Resources. Linearity: $3(k+2)$ queries; concatenation: $3k$; tensor: 4; circuit: 2; total $6k+12$. Random bits: $(2n+2n^2+2km)+k(m+n)+(2n+n^2)+(n-km+1+n^2) = 4n^2 + (k+5)n + 2km + 1$. The verdict is the AND of $O(k)$ parity checks of the answer bits, a circuit of size $O(k)$.

Completeness. If $x_1 \circ \cdots \circ x_k$ satisfies C, put $z = C\text{-aug}(\cdot)$, $X_i = L_{x_i}$, $A = L_z$, $B = L_{z \otimes z}$. Linearity and (since $\iota_i(u) \cdot z = u \cdot x_i$) concatenation tests hold; the tensor test holds as $L_z(z_1)L_z(z_2) = (z \cdot z_1)(z \cdot z_2) = \langle z \otimes z, z_1 \otimes z_2 \rangle$; the circuit test holds as $\langle z \otimes z, c \rangle = \sum_j r_j P_j(z) + c_0 = c_0$ by Lemma 4.14.

Soundness. Suppose the proof is accepted with probability $> \frac{19}{20}$. The tests use independent randomness, so each individual test accepts with probability $> \frac{19}{20}$.

(i) By Theorem 4.11 each of $X_1, \dots, X_k, A, B$ is ε-close, for some $\varepsilon < \frac{1}{20}$, to a linear function, say L_{x_i}, L_z, L_b, with $b \in \mathbb{F}_2^{n^2}$ regarded as an $n \times n$ matrix M.

(ii) *Tensor.* Suppose $M \neq zz^{\top}$. By Lemma 4.15 applied to $M + zz^{\top}$, with probability $\geq \frac{1}{4}$ over z_1, z_2 we have $L_z(z_1)L_z(z_2) \neq L_b(z_1 \otimes z_2)$. The test accepts only if this event fails, or $A(z_1)$ or $A(z_2)$ is wrong (probability $\leq \varepsilon$ each, both points uniform), or self-correction of B at $z_1 \otimes z_2$ is wrong ($\leq 2\varepsilon$). So it accepts with probability $\leq \frac{3}{4} + 4\varepsilon < \frac{19}{20}$, a contradiction. Hence $L_b = L_{z \otimes z}$.

(iii) *Circuit.* Let $z_1, \dots, z_k$ be the consecutive m-bit blocks of the first km coordinates of z. If some $P_j(z) \neq 0$ then for uniform r we have $\sum_j r_j P_j(z) = 1$ with probability exactly $\frac{1}{2}$, i.e. $\langle z \otimes z, c \rangle \neq c_0$ with probability $\frac{1}{2}$; the test accepts only if this event fails or the self-correction of B at c errs ($\leq 2\varepsilon$): at most $\frac{1}{2} + 2\varepsilon < \frac{19}{20}$, a contradiction. So $P_j(z) = 0$ for all j, and by Lemma 4.14 C accepts $z_1 \circ \cdots \circ z_k$ and $z = C\text{-aug}(z_1 \circ \cdots \circ z_k)$.

(iv) *Concatenation.* By Proposition 4.13(b) and $\frac{1}{2} + 3\varepsilon < \frac{19}{20}$, the i-th block function of L_z equals L_{x_i}, i.e. $z_i = x_i$. Since $\varepsilon < \frac{1}{4}$, $(E^{\oplus})^{-1}(X_i) = x_i$.

Therefore $(E^{\oplus})^{-1}(X_1) \circ \cdots \circ (E^{\oplus})^{-1}(X_k) = z_1 \circ \cdots \circ z_k$ satisfies C. □

Remark. Arora et al. (1998) obtain soundness $\frac{35}{36}$ with the weaker linearity theorem (closeness 2δ for rejection $\delta < \frac{2}{9}$); with Theorem 4.11 the same argument gives $\frac{19}{20}$, as above. Both are constants less than 1, which is all that is used.

Example 4.18 (Two circuits on five gates)**.** Take $k = 1$, $m = 2$ (inputs z_1, z_2), $n = 5$. The proof has $|X_1| = 4$, $|A| = 2^5 = 32$ and $|B| = 2^{25} = 33{,}554{,}432$ bits, the verifier reads 18 of them and uses $4 \cdot 25 + 6 \cdot 5 + 2 \cdot 2 + 1 = 135$ random bits.

(a) A satisfiable circuit. Gates $3 = 1 \cdot 2$, $4 = 1 + 2$, $5 = 3 + 4$, output 5: $C(x) = x_1 \vee x_2$. Its equations are $z_3 + z_1 z_2$, $z_4 + z_1 + z_2$, $z_5 + z_3 + z_4$, $z_5 + 1$. For $x = (1, 0)$ the augmented vector is $z = (1, 0, 0, 1, 1)$, and the honest proof is accepted with probability 1.

(b) An unsatisfiable circuit. Gates $3 = 1 + 2$, $4 = 1 \cdot 2$, $5 = 3 \cdot 4$, output 5. Over $\mathbb{F}_2$, $(x_1 + x_2)x_1x_2 = x_1x_2 + x_1x_2 = 0$, so $C \equiv 0$; the equations are $z_3 + z_1 + z_2$, $z_4 + z_1z_2$, $z_5 + z_3z_4$, $z_5 + 1$. No z satisfies all four, and some z violates exactly one. We now find the best cheating proof *among linear oracles* ($X_1 = L_x$, $A = L_z$, $B = L_b$), for which every test can be computed exactly. Take X_1 the prefix of z (so the concatenation test passes) and let M be the matrix of b. With $N = M + zz^\top$, the tensor test accepts with probability $\frac{1}{2} + 2^{-\operatorname{rank} N - 1}$ (Exercise 4.2), and the circuit test accepts with probability 1 if $\langle M, c^{(j)} \rangle = c_0^{(j)}$ for all four equations, and with probability exactly $\frac{1}{2}$ otherwise.

- $\operatorname{rank} N = 0$ (B is honest): tensor 1; but for every z some equation fails, so circuit $\frac{1}{2}$. Total $\frac{1}{2}$.
- $\operatorname{rank} N = 1$: tensor $\frac{3}{4}$. An exhaustive search over all $32 \cdot 31 \cdot 31$ choices of (z, u, v) with $N = uv^\top$ finds the circuit test passing with probability 1 for, e.g. , $z = 0$, $u = e_3 + e_5$, $v = e_4 + e_5$: then $u^\top C^{(j)} v = (0, 0, 0, 1) = c_0^{(j)}$. Total $\frac{3}{4}$.
- $\operatorname{rank} N \geq 2$: tensor $\leq \frac{5}{8}$.

So the best linear cheating proof is accepted with probability exactly $\frac{3}{4}$, well below the guarantee $\frac{19}{20}$ of Theorem 4.17. Note *which* test it cheats on: B is not the tensor square of A at all (here $A \equiv 0$), and passes the circuit test by being a rank-one tensor that happens to solve the four linear conditions. Without the tensor test the verifier would accept the unsatisfiable circuit with probability 1.

Why this verifier is a good *inner* verifier and a useless outer one

The proof has length 2^{n^2} and the verifier uses $\Theta(n^2)$ random bits, where n is the size of the circuit. For $n = \operatorname{poly}(N)$ this is exponential, so $V^\oplus$ can not be used as a PCP for 3SAT of size N. It reads *six bits per piece* regardless of n, and is exact about what it needs: the input is a *split-encoded* assignment $(X_1, \ldots, X_k)$, exactly the interface demanded by Definition 4.4. So $V^\oplus$ is the ideal last verifier of a recursion, applied to circuits that are tiny: if the previous verifier has a decision circuit of size $\operatorname{polylog} \log N$, then 2^{n^2} is $2^{(\log \log N)^{O(1)}} = N^{o(1)}$.

4.4 Lines: low-degree testing and correction with $O(1)$ queries

For the rest of this lecture $\mathbb{F}$ is a finite field, and $\mathcal{Q}_d^m$ denotes the polynomials $\mathbb{F}^m \to \mathbb{F}$ of *total* degree at most d. (In Lecture 3 we used the per-variable degree h, for which total degree is $\leq hm$. A line sees the total degree, so that is the natural notion here.)

Lemma 4.19 (Schwartz–Zippel, total degree)**.** *Two distinct $p, p' \in \mathcal{Q}_d^m$ agree on at most a $d/|\mathbb{F}|$ fraction of $\mathbb{F}^m$.*

Proof. It suffices to show that a nonzero $p \in \mathcal{Q}_d^m$ has at most a $d/|\mathbb{F}|$ fraction of zeros. For $m = 1$ there are at most d roots. For $m > 1$ write $p = \sum_{i \leq k} x_m^i p_i(x_1, \ldots, x_{m-1})$ with $p_k \neq 0$, so $\deg p_k \leq d - k$. By induction p_k vanishes on at most a $(d-k)/|\mathbb{F}|$ fraction; for every other $(x_1..x_{m-1})$ the polynomial in x_m has degree k and at most k roots. Total: $\leq (d-k)/|\mathbb{F}| + k/|\mathbb{F}| = d/|\mathbb{F}|$. □

In particular, if $d/|\mathbb{F}| \leq \frac{1}{2}$ and $\delta < \frac{1}{4}$ then f is δ-close to at most one $p \in \mathcal{Q}_d^m$ (Remark 22 of ALMSS).

The encoding. Let $H \subseteq \mathbb{F}$ with $|H| = h+1$ and let $n \le |H|^m$. The proof of Lemma 3.5 works verbatim for any $H \subseteq \mathbb{F}$ with $|H| = h+1$: every function $s\colon H^m \to \mathbb{F}$, in particular every bit string $s \in \{0,1\}^n$ padded to length $|H|^m$, has an extension $\hat{s}$ of total degree $\le mh$. The polynomial encoding $P_{n,m,H,\mathbb{F}}(s) = \hat{s}$ is the table of $\hat{s}$ on $\mathbb{F}^m$; the decoder returns, for $f\colon \mathbb{F}^m \to \mathbb{F}$, the values on H^m of the nearest polynomial of $\mathcal{Q}_d^m$ (d a degree bound specified by the verifier; $d \ge mh$), truncated to n bits. We need $|\mathbb{F}| \ge 2d$ so that nearest polynomials are unique.

4.4.1 The low-degree test with a table of lines

The Arora–Safra test of Lecture 3 reads $O(m)$ entries. The test of Rubinfeld and Sudan (1996), as strengthened by Arora–Safra's bivariate lemma, reads $O(1)$. Its trick is to use *all* lines, not only axis-parallel ones. Let $\ell_{x,h}(t) = x + th$ be the line with base x and direction $h \in \mathbb{F}^m$. For $p \in \mathcal{Q}_d^m$ the restriction $t \mapsto p(x+th)$ is a univariate polynomial of degree $\le d$.

Definition 4.20 (Line table, Poly-test)**.** A *line table* is a function $B\colon \mathbb{F}^{2m} \to \mathcal{Q}_d^1$ assigning to every pair (x,h) a univariate polynomial $B(x,h)$ of degree $\le d$. The honest line table of p is $\mathrm{lines}_p(x,h) = p(x + t\,h)$ (as a polynomial in t). *Poly-test*(f,B): choose $x,h \in \mathbb{F}^m$ and $t \in \mathbb{F}$ uniformly; accept iff $f(x+th) = B(x,h)(t)$. It makes two queries.

Theorem 4.21 (Low-degree test; ALMSS Theorem 45)**.** *There are constants $\delta_0 > 0$ and $\alpha < \infty$ such that for every d and field with $|\mathbb{F}| \ge \alpha(d+1)^3$ and every $\delta \le \delta_0$: (1)* Poly-test(p, lines_p) *accepts with probability* 1 *for $p \in \mathcal{Q}_d^m$; (2) if* Poly-test(f,B) *accepts with probability $\ge 1-\delta$ for some B, then f is 2δ-close to $\mathcal{Q}_d^m$.*

We quote it. Part (1) is trivial; part (2) is proved in Section A of ALMSS by combining the analysis of Rubinfeld and Sudan (1996) (whose number of queries is independent of m but whose field size depends on more than d) with the bivariate lemma of Arora–Safra (the base case of Lemma 3.17). The point to retain is the quantifiers: *two* queries, independent of m, d and δ, in exchange for a field of size $\Omega(d^3)$.

4.4.2 Correcting a polynomial at a point

Definition 4.22 (Poly-self-corr)**.** *Poly-self-corr*(A,B,x) with $A\colon \mathbb{F}^m \to \mathbb{F}$ and a line table B: choose $h \in \mathbb{F}^m$ and $t \in \mathbb{F} \setminus \{0\}$ uniformly; let $q = B(x,h)$; if $q(t) \ne A(x+th)$ reject, else output $q(0)$. Two queries.

Proposition 4.23 (ALMSS Proposition 46)**.** *Let $p \in \mathcal{Q}_d^m$. (1)* Poly-self-corr(p, lines_p, x) *outputs $p(x)$ with probability* 1*. (2) If $\Delta(A,p) = \varepsilon$ then for every $x \in \mathbb{F}^m$ and* every *line table B,*

$$\Pr_{h,t}\big[\textit{the procedure does not reject and its output is} \ne p(x)\big] \;\le\; 2\sqrt{\varepsilon} + \frac{d}{|\mathbb{F}|-1}.$$

Proof. (1) is clear. (2) For fixed x and $t \ne 0$, the point $x+th$ is uniform when h is uniform, so $\Pr_{h,t}[A(x+th) \ne p(x+th)] = \varepsilon$. Call h *bad* if $\Pr_t[A(x+th) \ne p(x+th)] > \sqrt{\varepsilon}$; by Markov $\Pr_h[\text{bad}] < \sqrt{\varepsilon}$. Fix a good h and write $q = B(x,h)$. If $q = p(x+\cdot\,h)$ then the output is $p(x)$ or the procedure rejects. If $q \ne p(x+\cdot\,h)$ then the two degree-$\le d$ polynomials agree in at most d values of t (so for $d/(|\mathbb{F}|-1)$ of $t \ne 0$), and a wrong output without rejection needs $q(t) = A(x+th)$, which happens when $A(x+th) = p(x+th) = q(t)$ (probability $\le d/(|\mathbb{F}|-1)$) or when $A(x+th) \ne p(x+th)$ (probability $\le \sqrt{\varepsilon}$ since h is good). Summing, $\sqrt{\varepsilon} + \sqrt{\varepsilon} + d/(|\mathbb{F}|-1)$. □

4.4.3 Concatenating polynomials

The analogue of Proposition 4.13 for polynomial encodings: from $p_1, \dots, p_k \in \mathcal{Q}_d^m$ build one polynomial in $m+1$ variables whose slices are the p_i. Fix an enumeration $\zeta_1, \zeta_2, \dots$ of the elements of $\mathbb{F}$; the ζ_i serve as indices for the k pieces.

Definition 4.24 (Concatenated polynomial)**.** For $k \le |\mathbb{F}|$ let $\mathrm{concat}_{p_1,\dots,p_k}(x, y) = \sum_{i=1}^{k} \lambda_i(y) p_i(x) \in \mathcal{Q}_{d+k-1}^{m+1}$, where λ_i is the Lagrange polynomial of degree $k-1$ with $\lambda_i(\zeta_j) = \mathbf{1}[i = j]$. It satisfies $\mathrm{concat}(x, \zeta_i) = p_i(x)$.

Lemma 4.25 (Concatenation test; ALMSS Lemma 49)**.** *Let $A \colon \mathbb{F}^{m+1} \to \mathbb{F}$ be ε-close to $p \in \mathcal{Q}_d^{m+1}$, let $p_i = p(\cdot, \zeta_i)$, and let $C \colon \mathbb{F}^m \to \mathbb{F}$.* Poly-corr-concat-test$(\zeta_i, A, B, C)$*: choose $x \in \mathbb{F}^m$; accept iff* Poly-self-corr$(A, B, (x, \zeta_i))$ *does not reject and outputs $C(x)$. (Three queries.) (1) If $A = p$, $B = \mathrm{lines}_p$, $C = p_i$ it accepts with probability 1. (2) If for some B it accepts with probability ρ, then $\Delta(C, p_i) < 1 - \rho + 2\sqrt{\varepsilon} + d/(|\mathbb{F}| - 1)$.*

Proof. (2) Let $\Delta = \Delta(C, p_i)$. The test accepts only if $C(x) = p(x, \zeta_i)$ (probability $1 - \Delta$ over x) or if the self-correction outputs something other than $p(x, \zeta_i)$ without rejecting; the latter has probability $\le 2\sqrt{\varepsilon} + d/(|\mathbb{F}| - 1)$ for *every* point, in particular for (x, ζ_i) (Proposition 4.23). □

Note that the correction is performed at the *non-uniform* points (x, ζ_i); this is why the line $\ell_{(x,\zeta_i),h}$ with random h is used: the point actually queried in A is uniform.

4.5 Curves: collapsing l queries into one

The Arora–Safra verifier asks $l = \mathrm{polylog}\, n$ queries. The l query points $z_1, \dots, z_l \in \mathbb{F}^{w+1}$ are determined by the random string R, so we may pre-compute them, and ask for the values of the polynomial p at all of them *at once*. A line through two points is easy; through l points in general position we need a curve of degree $l - 1$.

Definition 4.26 (Curves)**.** A *curve of degree s* in $\mathbb{F}^m$ is a map $\mathcal{C} \colon \mathbb{F} \to \mathbb{F}^m$ whose coordinates are univariate polynomials of degree $\le s$. Given points $z_1, \dots, z_l \in \mathbb{F}^m$ and the first l elements $\zeta_1, \dots, \zeta_l$ of $\mathbb{F}$, let $\mathrm{curve}_{z_1..z_l}$ be the curve of degree $l - 1$ obtained by Lagrange interpolation in every coordinate, so that $\mathrm{curve}(\zeta_j) = z_j$.

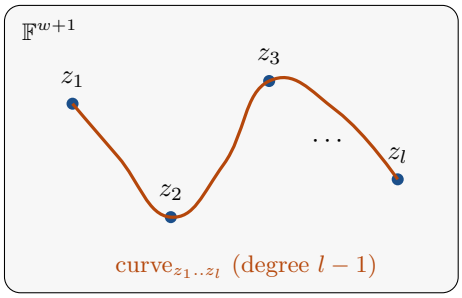
Proposition 4.27. *If $p \in \mathcal{Q}_d^m$ and $\mathcal{C}$ is a curve of degree s then $t \mapsto p(\mathcal{C}(t))$ is a univariate polynomial of degree $\le ds$. In particular for $\mathcal{C} = \mathrm{curve}_{z_1..z_l}$ the degree is $\le d(l-1)$.*

Proof. Each monomial of p has total degree $\le d$, and substituting polynomials of degree $\le s$ gives a polynomial of degree $\le ds$. □

Definition 4.28 (Curve-corr-check)**.** Given $A \colon \mathbb{F}^m \to \mathbb{F}$, a line table B, an oracle $O \colon \mathbb{F}^{ml} \to \mathcal{Q}_{d(l-1)}^1$ and points $z_1, \dots, z_l \in \mathbb{F}^m$: let $q = O(z_1, \dots, z_l)$; choose $t \in \mathbb{F}$ uniformly; if $q(t) \ne$ Poly-self-corr$(A, B, \mathrm{curve}_{z_1..z_l}(t))$ (including the case that the latter rejects) then reject; else output $(q(\zeta_1), \dots, q(\zeta_l))$. It makes three queries (one to O, two inside Poly-self-corr) independently of l.

Lemma 4.29 (ALMSS Lemma 57)**.** *Let $p \in \mathcal{Q}_d^m$ and A be ε-close to p. (1) If $B = \mathrm{lines}_p$ and $O(z_1..z_l) = p|_{\mathrm{curve}_{z_1..z_l}}$ then the output is $(p(z_1), \dots, p(z_l))$ with probability 1. (2) For all B, O and $z_1, \dots, z_l$,*

$$\Pr\big[\textit{no rejection and output} \ne (p(z_1), \dots, p(z_l))\big] \;\le\; 2\sqrt{\varepsilon} + \frac{d}{|\mathbb{F}| - 1} + \frac{d(l-1)}{|\mathbb{F}|}.$$



Without curves: l queries $A(z_1), \ldots, A(z_l)$, each of which must itself be self-corrected.

With curves: one query $q = O(z_1, \ldots, z_l)$, a polynomial of degree $d(l-1)$, then one random t and one self-corrected value at $\text{curve}(t)$. Output: $q(\zeta_1), \ldots, q(\zeta_l)$.

Figure 4.2: Aggregating queries. The oracle O lists, for every tuple of points, the restriction of the (encoded) proof to the curve through them. The verifier *checks* this restriction at one random point of the curve, and *reads off* the l values it wanted from the coefficients.

Proof. (1) is clear since $q(\zeta_j) = p(\text{curve}(\zeta_j)) = p(z_j)$. (2) Let $\mathcal{C} = \text{curve}_{z_1..z_l}$ and $q = O(z_1..z_l)$. If $q = p \circ \mathcal{C}$ then the output, if any, is $(p(z_j))_j$. Otherwise q and $p \circ \mathcal{C}$ are distinct polynomials of degree $\leq d(l-1)$ and agree for at most $d(l-1)/|\mathbb{F}|$ of the t. Not rejecting requires $q(t)$ to equal the self-corrected value at $\mathcal{C}(t)$. That value is $p(\mathcal{C}(t))$ except with probability $\leq 2\sqrt{\varepsilon} + d/(|\mathbb{F}| - 1)$ (Proposition 4.23, which is valid at every point, in particular at $\mathcal{C}(t)$ for each fixed t). So non-rejection requires $q(t) = p(\mathcal{C}(t))$ (probability $\leq d(l-1)/|\mathbb{F}|$) or a wrong self-correction. □

Pitfall: a curve is not a random point

A point $\text{curve}(t)$ on a curve through *given* points is not uniformly distributed in $\mathbb{F}^m$; it is determined by $z_1, \ldots, z_l$. An adversary who knows the verifier's program may corrupt A exactly on the points that the verifier will want to read, and querying A directly at $\text{curve}(t)$ would give no protection from the ε-fraction where A is wrong. This is why the check goes through Poly-self-corr, which replaces the point by $\text{curve}(t) + t'h$ for a random direction h: a uniformly distributed point.

4.6 The parallelized verifier

The starting point is the Arora–Safra verifier in the inner-verifier formalism.

Theorem 4.30 (Arora–Safra, in the form of ALMSS Theorem 39; quoted)**.** *For every constant k there are constants c_1, a, b and $e_{\text{seq}} < 1$ and an inner verifier system V^{seq} with parameters*

$$(k;\ r, p, c, a;\ e) \ =\ (k;\ c_1 \log n,\ \log^a n,\ \log^b n,\ 1;\ e_{\text{seq}}),$$

i.e. $c_1 \log n$ random bits, $\log^a n$ queries, a decision circuit of size $\log^b n$ and bit answers.

This is the verifier of Section 3.7 with $h = \Theta(\log n)$ and $m = \log n / \log\log n$ (the verifier V_{savior} of Section 3.9), read as an inner verifier system: its encoding is the polynomial extension, it makes $O(k+m) = \text{polylog}\, n$ queries, and its decision is a circuit of size $\text{poly}(h) = \text{polylog}\, n$. (ALMSS write the exponents of the number of queries and of the circuit size as c_2 and c_3 in the statement but use c_3 for the number of queries in Section 7.5; we call them a and b.) Its oracles are bit strings of length $N \leq \log^a n \cdot 2^{c_1 \log n} = \text{poly}(n)$.

Parameters of V^{par}. Let $w = \lceil \log N / \log\log N \rceil$, $H \subseteq \mathbb{F}$ with $|H| = \log N$ (so $|H|^w \geq N$ and the polynomial encodings have total degree at most $w(|H|-1)$), $d = w|H|+k$, and $|\mathbb{F}| = \log^s N$ with $s = \max\{7, a+3\}$. Then $|\mathbb{F}| \geq \alpha(d+1)^3$ for large n (as $d = O(\log^2 N)$), and $dl/|\mathbb{F}| = o(1)$ where $l = \log^a n$ is the number of queries of V^{seq}. The encoding of V^{par} is $E^{\mathrm{par}} = P_{N,w,H,\mathbb{F}} \circ E^{\mathrm{seq}}$ (first encode with V^{seq}'s scheme, then as a polynomial). Its decoder first finds the nearest polynomial of degree $\leq d$ (the degree bound against which the test certifies closeness; for codewords of degree $\leq w|H|$ this changes nothing) and then applies $(E^{\mathrm{seq}})^{-1}$.

Definition 4.31 (The verifier V^{par})**.** The input is a circuit C with km input nodes. The proof consists of $X_1, \ldots, X_k \colon \mathbb{F}^w \to \mathbb{F}$ and of X_{k+1} with four suboracles:

oracle	type	expected content
X_i $(i \leq k)$	$\mathbb{F}^w \to \mathbb{F}$	$P(E^{\mathrm{seq}}(x_i))$: encoding of the i-th piece
Z	$\mathbb{F}^w \to \mathbb{F}$	$P(\pi)$: encoding of the auxiliary proof π of V^{seq}
A	$\mathbb{F}^{w+1} \to \mathbb{F}$	$\mathrm{concat}_{X_1,\ldots,X_k,Z}$
B	$\mathbb{F}^{2(w+1)} \to \mathcal{Q}^1_d$	lines_A (to test and correct A)
O	$\mathbb{F}^{(w+1)l} \to \mathcal{Q}^1_{d(l-1)}$	$A\vert_{\mathrm{curve}(\cdot)}$ (to aggregate queries)

1. *Low degree:* run Poly-test(A, B).
2. *Concatenation:* for $i = 1, \ldots, k$ run the test of Lemma 4.25 for (ζ_i, A, B, X_i), and for (ζ_{k+1}, A, B, Z).
3. *Simulation of V^{seq}:* draw $R \in \{0,1\}^{c_1 \log n}$; let $(i_1, q_1), \ldots, (i_l, q_l)$ be the queries of V^{seq} on (C, R) (oracle i_j, position $q_j \in [N]$); let $\# \colon [N] \to H^w$ be the canonical map and $z_j = (\#(q_j), \zeta_{i_j}) \in \mathbb{F}^{w+1}$. Run Curve-corr-check$(A, B, O, z_1, \ldots, z_l)$ to obtain $(a_1, \ldots, a_l)$; reject if some $a_j \notin \{0,1\}$; reject if V^{seq}'s decision circuit on (C, R) rejects the answers $(a_1, \ldots, a_l)$.

Accept iff nothing rejects.

Theorem 4.32 (Parallelization; ALMSS Theorem 14)**.** *V^{par} is a $(k;\ O(\log n),\ 3k+8,\ \mathrm{polylog}\, n,\ \mathrm{polylog}\, n;\ e)$ inner verifier system for some constant $e < 1$ depending on k.*

Proof. Resources. Queries: 2 (Poly-test) $+3(k+1)$ (concatenation tests) $+3$ (curve check) $= 3k+8$. (ALMSS count $3k+5$, omitting the test for Z; this is immaterial.) Random bits: each test draws $O(w)$ field elements, i.e. $O(w \log|\mathbb{F}|) = O(\log N)$ bits since $w \log|\mathbb{F}| = s \log N$, so the total is $O(k \log N) + r = O(\log n)$ for fixed k. Answer sizes are those of the entries of B and O: $O(dl \log|\mathbb{F}|) = \mathrm{polylog}\, n$ bits. Poly-test, the correction and the curve-check involve $\mathrm{poly}(d, l, m)$ field operations; the decision of V^{seq} is a circuit of size $\log^b n$; so the decision is a circuit of size $\mathrm{polylog}\, n$ of the answers.

Completeness. Given $x_1 \circ \cdots \circ x_k$ satisfying C, let π be the proof of V^{seq} accepted with probability 1 and set the oracles to the expected contents of the table. Every test is then an identity between polynomials, and in Step 3 the answers a_j are exactly the bits V^{seq} would read, which it accepts.

Soundness. Let $\eta = 2\sqrt{0.002} + \frac{d}{|\mathbb{F}|-1} + \frac{dl}{|\mathbb{F}|} < 0.1$ (for large n) and $e = \max\{0.999,\ 1-\delta_0,\ 1-(1-\eta)(1-e_{\mathrm{seq}})\} < 1$. Suppose V^{par} accepts $(X_1..X_k, Z, A, B, O)$ with probability $> e$.

(i) Step 1 accepts with probability $> \max\{0.999, 1-\delta_0\}$, so by Theorem 4.21 there is $p \in \mathcal{Q}_d^{w+1}$ with $\varepsilon := \Delta(A,p) < 2\min\{0.001, \delta_0\} \le 0.002$.

(ii) Each concatenation test accepts with probability > 0.999, so by Lemma 4.25 $\Delta(X_i, p_i) < 0.001 + 2\sqrt{0.002} + o(1) < \frac{1}{4}$ for $p_i = p(\cdot, \zeta_i)$ $(i \le k)$ and likewise for Z and p_{k+1}. Since $d/|\mathbb{F}| \le \frac{1}{2}$, p_i is the unique polynomial of $\mathcal{Q}_d^w$ within $\frac{1}{4}$ of X_i, so the decoder returns, from X_i, the string Y_i of values of p_i on H^w (after $(E^{\mathrm{seq}})^{-1}$). Let Y_{k+1} be the values of p_{k+1} on H^w.

(iii) Let S be the set of R for which the tuple $(p(z_j))_{j \le l}$ is *not* a tuple of bits accepted by the decision circuit of V^{seq} on (C, R). Since $p(\#(q_j), \zeta_{i_j}) = p_{i_j}(\#(q_j))$ is the q_j-th value of Y_{i_j}, this tuple is what V^{seq} would read from $Y_1, \dots, Y_{k+1}$ on R (replace non-bit entries of the Y_i by 0 to obtain bit strings; this does not change the behaviour on $R \notin S$). Fix $R \in S$. By Lemma 4.29, Step 3 returns $(p(z_j))_j$ or rejects, except with probability $\le \eta$; and on $(p(z_j))_j$ Step 3(f) rejects since $R \in S$. So Step 3 accepts with probability $\le \eta$ given $R \in S$.

(iv) Hence $\Pr[V^{\mathrm{par}} \text{ accepts}] \le \Pr[S]\,\eta + (1 - \Pr[S]) = 1 - (1-\eta)\Pr[S]$. Since this exceeds $e \ge 1 - (1-\eta)(1-e_{\mathrm{seq}})$, we get $\Pr[S] < 1 - e_{\mathrm{seq}}$: V^{seq} accepts $Y_1, \dots, Y_{k+1}$ (as bit strings) with probability $> e_{\mathrm{seq}}$. By its soundness $(E^{\mathrm{seq}})^{-1}(Y_1) \circ \cdots \circ (E^{\mathrm{seq}})^{-1}(Y_k)$ satisfies C, and this string is the decoding of $X_1, \dots, X_k$ under E^{par}. □

Remark (Where each technique is used). *Low-degree test and concatenation tests* certify that the (arbitrary) tables X_i, Z are close to polynomials that are the slices of one polynomial p: they play the role that linearity and concatenation play in Section 4.3. *Self-correction* is used so that every read of A is at a uniform point. *Curves* are the new ingredient, absent from Lecture 3: they are the reason the number of queries is independent of l.

4.7 Assembling the proof: three steps, constant parameters

Theorem 4.33 (ALMSS Theorem 17)**.** *There is a constant C such that for every $L \in \mathsf{NP}$ there is a constant c_L with $L \in \mathrm{RPCP}(c_L \log n,\ C,\ 2C,\ 1,\ \frac{1}{2})$.*

Since an (r, p, c, a) outer verifier reads pa bits, Theorem 4.33 gives $L \in \mathsf{PCP}[c_L \log n, C]$, which is Theorem 4.1.

Proof. It suffices to treat CKT-SAT, which is NP-complete. All inner verifiers below are brought to error $\frac{1}{16}$ by Proposition 4.6; this multiplies each parameter by a constant.

Step 1: constant queries, large answers. Apply Theorem 4.32 with $k = 1$ (its proof starts from the verifier of Theorem 4.30). By Proposition 4.5,

$$\text{CKT-SAT} \in \mathrm{RPCP}(c_1 \log n,\ p_1,\ \mathrm{polylog}\, n,\ \mathrm{polylog}\, n,\ \tfrac{1}{16}) \qquad (p_1 \text{ a constant}). \quad (1)$$

Step 2: shrink the answers by composing with the same kind of verifier. Apply Theorem 4.32 again, now with $k := p_1$ *the constant of Step 1.* This gives a $(p_1;\ r_2, p_2, c_2, a_2;\ \frac{1}{16})$ inner system with $r_2 = O(\log n)$, $p_2 = O(1)$ and $c_2, a_2 = \mathrm{polylog}\, n$. Compose with (1) (Theorem 4.7); the circuit size to feed it is $c(n) = \mathrm{polylog}\, n$, so $r_2(c(n)) = O(\log \log n)$ and $c_2(c(n)), a_2(c(n)) = \mathrm{polylog}(\mathrm{polylog}\, n) = (\log \log n)^{O(1)}$:

$$\text{CKT-SAT} \in \mathrm{RPCP}(c_1 \log n + O(\log \log n),\ p_2,\ (\log \log n)^{O(1)},\ (\log \log n)^{O(1)},\ \tfrac{1}{8}). \quad (2)$$

stage	random bits	queries	answer bits	decision circuit
Thm 4.30 ($k=1$)	$c_1 \log n$	$\log^a n$	1	$\log^b n$
(1) parallelized	$O(\log n)$	$p_1 = O(1)$	$\operatorname{polylog} n$	$\operatorname{polylog} n$
(2) composed with $V^{\mathrm{par}}_{k=p_1}$	$O(\log n) + O(\log\log n)$	$p_2 = O(1)$	$(\log\log n)^{O(1)}$	$(\log\log n)^{O(1)}$
(3) composed with $V^{\oplus}_{k=p_2}$	$O(\log n) + (\log\log n)^{O(1)}$	$h = O(1)$	1	$2h$

Table 4.1: The parameter chase of Theorem 4.33. Compare with Table 3.2: there, *every* composition kept $\sqrt{\log n}$ queries and shrank the exponent of the alphabet by half; here the queries are made constant before any composition, and two compositions suffice.

Step 3: shrink the answers to one bit with the Hadamard verifier. Apply Theorem 4.17 with $k := p_2$. This gives a $(p_2;\, gn^2, h, 2h, 1;\, \frac{1}{16})$ inner system, with g, h constants. Compose with (2); its circuit size is $c(n) = (\log\log n)^{O(1)}$, so the random bits become $c_1 \log n + O(\log\log n) + g\, c(n)^2 = c_1 \log n + (\log\log n)^{O(1)} = O(\log n)$:

$$\text{CKT-SAT} \in \mathrm{RPCP}\big(O(\log n),\ h,\ 2h,\ 1,\ \tfrac{3}{16}\big).$$

The errors compose by $e \mapsto e + \frac{1}{16} - \frac{e}{16}$: from $\frac{1}{16}$ to $\frac{31}{256} < \frac{1}{8}$ in step 2 and to $\frac{721}{4096} < \frac{3}{16}$ in step 3 (Exercise 4.5). This is below $\frac{1}{2}$; take $C = h$. □

Why the order matters

In Lecture 3 the number of queries after a composition is $O(Q + q(\tau))$: it never goes below the outer verifier's Q. So the *first* verifier must already have $O(1)$ queries, and parallelization achieves that at the price of larger answers. The second observation is that the *size* of the answer is irrelevant to the number of queries of the composed verifier, so large answers can be cut down later by inner verifiers of *any* speed. Finally, because each inner verifier is fed an outer verifier with a constant number p_i of queries, its parameter $k = p_i$ is a constant, so the constants p_2 and h are *independent of* n: nothing compounds. The Hadamard verifier closes the recursion because its exponential size is paid on a circuit of size $(\log\log n)^{O(1)}$.

Remark (Constructivity). Following the proofs one sees that, given a satisfying assignment, the accepted proof is computable in polynomial time, and conversely a satisfying assignment can be recovered in polynomial time from any proof accepted with probability $\geq \frac{1}{2}$ (Remark 18 of ALMSS: decoding polynomial and Hadamard codes is efficient). This constructive form is what lets a PCP reduction be *a Karp reduction with a witness map*, used in the Label Cover reductions of Lectures 5–7.

Remark (How large is q?). The constant is not made explicit in the paper: it depends on δ_0 and α of Theorem 4.21, which are not computed. Section 8 of ALMSS reports that the smallest q that can be obtained went, in subsequent work, from 29 (Bellare–Goldwasser–Lund–Russell) to 22 (Feige–Kilian) to 16 (Bellare–Goldreich–Sudan) to at most 9 (the best that can be inferred directly from Håstad's work); all of these are for soundness $\frac{1}{2}$. Håstad's optimal results (Lecture 7) concern *three*-bit verifiers with soundness close to the trivial value: completeness $1-\varepsilon$ and soundness $\frac{1}{2}+\varepsilon$ (accept iff a parity of the three bits equals a prescribed value), and perfect completeness with soundness $\frac{7}{8}+\varepsilon$ (accept iff a 3-clause is satisfied). They are tight because a random proof achieves $\frac{1}{2}$, respectively

$\frac{7}{8}$. The Hadamard verifier of this lecture on its own reads $6k + 12$ bits per run, and 55 repetitions bring $\frac{19}{20}$ down to $\frac{1}{16}$ (Exercise 4.5); this is a first indication of where the constant comes from.

4.8 Consequences, and the link to games

Max-3SAT. By Proposition 1.9 (hardness direction) the theorem gives a constant $\varepsilon_0 > 0$ such that $\textsc{Gap}_{1,1-\varepsilon_0}$-3SAT is NP-hard; with q queries and soundness $\frac{1}{2}$ one may take $\varepsilon_0 = 1/(2(q-2)2^q)$, and ALMSS's own computation gives $1/(q\,2^{q+1})$ in their Theorem 3 (where they credit the idea to Arora–Motwani–Safra–Sudan–Szegedy). By the L-reductions of Papadimitriou and Yannakakis this transfers to every Max-SNP-hard problem: none has a PTAS unless P = NP.

Max-Clique. Let V be the verifier of Theorem 4.1 ($r = c\log n$ random bits, q queries, soundness $\frac{1}{2}$). Replace it by the verifier V_k that uses a k-step random walk on a constant-degree expander on $\{0,1\}^r$ (Exercise 2.6): $r + O(k)$ random bits, kq queries, soundness $\le (\frac{3}{4})^{k-1}$. Take $k = \lceil \log n \rceil$. Then the FGLSS graph H_x has $N = 2^{r+O(k)+kq} = n^{O(1)}$ vertices and, by Theorem 2.19, clique number $2^{r'}$ if $x \in$ 3SAT and $\le 2^{r'}(3/4)^{k-1}$ otherwise: a gap of $(4/3)^{k-1} = n^{\Omega(1)} = N^{\varepsilon_1}$. So approximating the clique number within N^{ε_1} is NP-hard (Arora et al., 1998, Theorem 6). This is exactly the route indicated by their Theorem 2 (FGLSS plus "recycled" randomness), and it supersedes Corollary 3.2: the exponent is now polynomial. Håstad's $N^{1-\varepsilon}$ comes in Lecture 7.

Label Cover. Apply Corollary 2.14 to the verifier of Theorem 4.1: the constraint–variable game is a projection game with $|\Sigma_U| \le 2^q$, $|\Sigma_W| = 2$ and

$$x \in \text{3SAT} \Rightarrow \text{val} = 1, \qquad x \notin \text{3SAT} \Rightarrow \text{val} \le 1 - \tfrac{1}{2q}.$$

This is the starting point of Theorem 1.17: ALMSS point out (Section 1.2) that their Theorem 4 "provides the first known construction of a 2-prover 1-round proof system that uses logarithmic random bits and constant number of communication bits". Lecture 5 will show how Raz's theorem drives $1-\frac{1}{2q}$ to δ at the cost of an alphabet of size $\text{poly}(1/\delta)$.

Transparent proofs of mathematics. The set of pairs $(\varphi, 1^n)$ such that φ has a proof of length n in ZFC is in NP. By Theorem 4.1 every theorem therefore has a "proof" that a verifier using $O(\log n)$ random bits checks by reading $O(1)$ bits, and by the constructive form of the remark above the transformation from an ordinary proof to such a proof is polynomial. This is the sense of the pitch in Lecture 1 ("proofs that can be checked by reading only a few bits").

4.9 What the proof does not do, and what replaces it

Three features of the proof are worth noticing before moving on.

1. *There is no mechanism that makes a gap grow, other than repetition.* Each module—linearity, tensor, low-degree, concatenation, curve-check—detects a corrupted proof with some *constant* probability, and soundness $\frac{1}{2}$ is reached by independent repetition (Proposition 4.6). The final constants are those of the

weakest module (here δ_0 of Theorem 4.21). A *gap-amplifying* step, the heart of Lecture 6, is a different idea.

2. *The only part with an explicit constant is the Hadamard verifier.* The rest passes through the low-degree test with the field-size condition $|\mathbb{F}| \geq \alpha(d+1)^3$ and the non-explicit δ_0.
3. *The algebra is used twice, for two different reasons.* Polynomials give *local* testability with few queries (low-degree test, self-correction, curves). Hadamard codes give *constant-size* testability at the price of exponential length.

Dinur (2007) (Lecture 6; see Figure 1.5) replaces all of the polynomial machinery by a combinatorial operation on constraint graphs that amplifies the gap by a constant factor at each step, using expander graphs and random walks, so that after $O(\log n)$ steps the gap is constant. The *composition* step survives, but only with a verifier of *constant* input size. Dinur remarks that this one verifier can be found by exhaustive search, provided its existence has been established independently, and that one may use "Long-code based or Hadamard-code based" constructions for it; the Hadamard-type verifier of Section 4.3 is the prototype. (Dinur composes with an *assignment tester*, i.e. a PCP of proximity, which is a slightly stronger notion than our inner verifier; Section 6 of her paper gives the long-code instance.) In short: Lecture 4 builds the theorem from algebra, Lecture 6 from combinatorics plus one small, constant-size verifier of the kind constructed in Section 4.3.

Exercises

Exercise 4.1 ([⋆]). Let $F\colon \mathbb{F}_2^3 \to \mathbb{F}_2$ be the majority function. Compute all Fourier coefficients of $f = (-1)^F$, the acceptance probability of the linearity test via Theorem 4.11, and the distance to the nearest linear function. Check against a direct count of the 64 pairs (x, y). Compare Pr[reject] with the two bounds Δ and 3Δ of the theorem.

Exercise 4.2 ([⋆]). Prove that for $M \in \mathbb{F}_2^{n\times n}$ of rank $r \geq 1$, $\Pr_{u,v}[u^\top M v = 1] = \frac{1}{2} - 2^{-r-1}$. *Hint: change bases so that* $M = \operatorname{diag}(1,\dots,1,0,\dots,0)$*; then* $u^\top M v = \sum_{i\leq r} u_i v_i$*, and the bias of an inner product of* r*-bit strings is* 2^{-r}. Conclude that Lemma 4.15 is tight for $r = 1$.

Exercise 4.3 ([⋆]). Show that every 3-CNF with n variables and m clauses can be written as a system of $2m$ quadratic equations over $\mathbb{F}_2$ in $n+m$ variables, each equation of the form $P(z) = 0$ with $\deg P \leq 2$, which is satisfiable iff the formula is. *Hint: the clause* $\ell_1 \vee \ell_2 \vee \ell_3$ *is violated iff* $(1+\ell_1)(1+\ell_2)(1+\ell_3) = 1$*; introduce* $y = (1+\ell_1)(1+\ell_2)$. Why does the verifier of Section 4.3 work for this input directly, without a circuit?

Exercise 4.4 ([⋆⋆]). Return to Example 4.18(b). (a) List all $z \in \mathbb{F}_2^5$ that violate exactly one of the four equations. (b) Verify that $z = 0$, $u = e_3 + e_5$, $v = e_4 + e_5$ and $B = L_{uv^\top}$ pass the circuit test with probability 1. (c) Prove that no linear cheating proof (all of X_1, A, B linear) is accepted with probability more than $\frac{3}{4}$. (d) Find a *non-linear* A that is accepted with higher probability than your best linear cheating strategy, or explain why Theorem 4.17 still guarantees at most $\frac{19}{20}$.

Exercise 4.5 ([⋆⋆]). (a) How many independent repetitions bring the error $\frac{19}{20}$ of $V^{\oplus}$ to at most $\frac{1}{16}$? What is the resulting number of queries as a function of k? (b) By Theorem 4.7 the error after composing three verifiers each with error $\frac{1}{16}$ is $e + e_1 - e e_1$ applied twice; compute it exactly.

Exercise 4.6 ([⋆⋆]). The oracle O of V^{par} is defined on $\mathbb{F}^{(w+1)l}$, a superpolynomial set. Explain why the *proof* is nevertheless of polynomial size. How many entries of O can be read, as a function of n? Why is it legitimate for the soundness argument that O is arbitrary on all other entries?

Exercise 4.7 ([⋆⋆]). Suppose we try to aggregate l queries with a *line* instead of a curve: ask for p restricted to a line through z_1 and z_2 and so on. (a) Show that for l points in general position no line contains them. (b) What do curves of degree s cost in the degree of the restriction and in the soundness error $ds/|\mathbb{F}|$? (c) Why is it essential that $|\mathbb{F}| \gg dl$?

Exercise 4.8 ([⋆ ⋆ ⋆]). *Reading.* Compare Lemma 3.23 (Lecture 3) with Theorem 4.7. Which soundness clause of Definition 3.21 corresponds to "E^{-1} of the oracles satisfies C" in Definition 4.4? Where does the inner system of this lecture use *decoding* while the normal-form verifier of Lecture 3 used *closeness*? Then identify, in the proof of Theorem 4.32, the single place where the unique-decoding radius is used, and what would break if $\Delta(X_i, p_i)$ could only be bounded by $\frac{1}{2} + \varepsilon$.

Notes and further reading

Where this lecture comes from. Everything is from Arora et al. (1998): the outer/inner formalism and composition are Sections 2.1; the Hadamard verifier is Sections 5–6; parallelization is Section 7; the Max-3SAT consequence is Section 3. The linearity test and self-correction are due to Blum et al. (1990); the tensor test is a Freivalds-type matrix-product check (Freivalds, 1979); the Fourier-analytic form of the linearity analysis follows the standard modern treatment, for which see the detailed study of the test in characteristic two by Bellare et al. (1996). The low-degree test with a table of lines and its analysis combine Rubinfeld and Sudan (1996) and the bivariate lemma of Arora and Safra (1998); the correction procedure goes back to Beaver–Feigenbaum and Lipton and, in the form with constant queries, to Gemmell–Lipton–Rubinfeld–Sudan–Wigderson, as cited in ALMSS. The numerical Example 4.18 was computed for these notes.

On attributions. The conference versions of Arora and Safra (1998) and Arora et al. (1998) both appeared at FOCS 1992 (pp. 2–12 and 13–22). The constant-query theorem is rightly credited to ALMSS, but its proof rests on the composition technique and the polylogarithmic verifier of Arora and Safra; ALMSS say so in their Section 1.2. The *linearity-testing / Hadamard* half of the proof was in turn inspired, as they note, by work on program checking.

Small points in the text of Arora et al. (1998) that we handled differently. Five points, as we read the paper; none affects the theorem, and we list them so that a reader comparing with the original is not surprised. (i) Step 4(c) of $V^{\oplus}$ compares the self-corrected value of B at coeff_P with 0, while coeff_p is defined only for $p(0) = 0$ and the output polynomial (zero iff the output gate accepts, i.e. $z_n + 1$ over $\mathbb{F}_2$) has a constant term; we compare with the constant term of $\sum_j r_j P_j$ (Definition 4.16), which repairs this. (ii) The number of queries of V^{par} is stated as $3k + 5$; counting the concatenation test for Z gives $3k + 8$. (iii) The decoder of E^{par} is defined with the nearest polynomial of degree

mh, whereas the soundness proof produces the nearest polynomial of the degree d that Poly-test certifies; we define the decoder with degree d, which agrees with the original on all honest codewords. (iv) The linearity theorem is quoted with closeness 2δ; the Fourier argument gives δ, which we used to get soundness $\frac{19}{20}$ rather than $\frac{35}{36}$. (v) In Section 7.5 the exponent c_3 is used both for the circuit size (as in the statement of Theorem 39) and for the number of queries l; the field size $\log^{\max\{2+c_3,6\}} N$ must dominate dl, so it is the exponent of the number of queries that matters.

What is not covered. We have not proved Theorem 4.21 (the low-degree test with lines) or Theorem 4.30 beyond what Lecture 3 gave, and we have not discussed the $\mathsf{PCP}[\log n, (\log\log n)^2]$ intermediate result. Lecture 5 turns to the *soundness* side: Raz's parallel repetition theorem, which the constant-query PCP of this lecture feeds with a projection game of value $1 - \frac{1}{2q}$.

Bibliography

M. Ajtai, J. Komlós, and E. Szemerédi. Deterministic simulation in LOGSPACE. In *Proceedings of the 19th Annual ACM Symposium on Theory of Computing (STOC)*, pages 132–140, 1987.

S. Arora and B. Barak. *Computational Complexity: A Modern Approach*. Cambridge University Press, 2009.

S. Arora and S. Safra. Probabilistic checking of proofs: a new characterization of NP. *Journal of the ACM*, 45(1):70–122, 1998.

S. Arora and M. Sudan. Improved low degree testing and its applications. In *Proceedings of the 29th Annual ACM Symposium on Theory of Computing (STOC)*, pages 496–505, 1997.

S. Arora, L. Babai, J. Stern, and Z. Sweedyk. The hardness of approximate optima in lattices, codes, and systems of linear equations. *Journal of Computer and System Sciences*, 54(2):317–331, 1997.

S. Arora, C. Lund, R. Motwani, M. Sudan, and M. Szegedy. Proof verification and the hardness of approximation problems. *Journal of the ACM*, 45(3):501–555, 1998.

S. Arora, B. Barak, and D. Steurer. Subexponential algorithms for unique games and related problems. *Journal of the ACM*, 62(5):Article 42, 2015.

L. Babai, L. Fortnow, L. A. Levin, and M. Szegedy. Checking computations in polylogarithmic time. In *Proceedings of the 23rd Annual ACM Symposium on Theory of Computing (STOC)*, pages 21–31, 1991a.

L. Babai, L. Fortnow, and C. Lund. Non-deterministic exponential time has two-prover interactive protocols. *Computational Complexity*, 1(1):3–40, 1991b.

M. Bellare, D. Coppersmith, J. Håstad, M. Kiwi, and M. Sudan. Linearity testing in characteristic two. *IEEE Transactions on Information Theory*, 42(6):1781–1795, 1996.

M. Bellare, O. Goldreich, and M. Sudan. Free bits, PCPs, and nonapproximability—towards tight results. *SIAM Journal on Computing*, 27(3):804–915, 1998.

M. Ben-Or, S. Goldwasser, J. Kilian, and A. Wigderson. Multi-prover interactive proofs: how to remove intractability assumptions. In *Proceedings of the 20th Annual ACM Symposium on Theory of Computing (STOC)*, pages 113–131, 1988.

E. Ben-Sasson, O. Goldreich, P. Harsha, M. Sudan, and S. Vadhan. Robust PCPs of proximity, shorter PCPs, and applications to coding. *SIAM Journal on Computing*, 36(4):889–974, 2006.

M. Blum, M. Luby, and R. Rubinfeld. Self-testing/correcting with applications to numerical problems. In *Proceedings of the 22nd Annual ACM Symposium on Theory of Computing (STOC)*, pages 73–83, 1990.

M. Charikar, K. Makarychev, and Y. Makarychev. Near-optimal algorithms for unique games. In *Proceedings of the 38th Annual ACM Symposium on Theory of Computing (STOC)*, pages 205–214, 2006.

S. A. Cook. The complexity of theorem-proving procedures. In *Proceedings of the 3rd*

Annual ACM Symposium on Theory of Computing (STOC), pages 151–158, 1971.

I. Dinur. The PCP theorem by gap amplification. *Journal of the ACM*, 54(3):Article 12, 2007.

I. Dinur and O. Reingold. Assignment testers: towards a combinatorial proof of the PCP theorem. *SIAM Journal on Computing*, 36(4):975–1024, 2006.

I. Dinur and D. Steurer. Analytical approach to parallel repetition. In *Proceedings of the 46th Annual ACM Symposium on Theory of Computing (STOC)*, 2014. arXiv:1305.1979.

I. Dinur, S. Khot, G. Kindler, D. Minzer, and M. Safra. Towards a proof of the 2-to-1 games conjecture? In *Proceedings of the 50th Annual ACM Symposium on Theory of Computing (STOC)*, pages 376–389, 2018.

U. Feige. A threshold of $\ln n$ for approximating set cover. *Journal of the ACM*, 45(4): 634–652, 1998.

U. Feige and L. Lovász. Two-prover one-round proof systems: their power and their problems. In *Proceedings of the 24th Annual ACM Symposium on Theory of Computing (STOC)*, pages 733–744, 1992.

U. Feige, S. Goldwasser, L. Lovász, S. Safra, and M. Szegedy. Interactive proofs and the hardness of approximating cliques. *Journal of the ACM*, 43(2):268–292, 1996.

U. Feige, G. Kindler, and R. O'Donnell. Understanding parallel repetition requires understanding foams. In *Proceedings of the 22nd IEEE Conference on Computational Complexity (CCC)*, 2007.

R. Freivalds. Fast probabilistic algorithms. In *Mathematical Foundations of Computer Science (MFCS 1979)*, volume 74 of *Lecture Notes in Computer Science*, pages 57–69. Springer, 1979.

M. X. Goemans and D. P. Williamson. Improved approximation algorithms for maximum cut and satisfiability problems using semidefinite programming. *Journal of the ACM*, 42(6):1115–1145, 1995.

P. Harsha. CMSC 39600: PCPs, codes and inapproximability, lecture 1. Lecture notes, scribe J. A. Grochow, University of Chicago, 2007.

P. Harsha and M. Charikar. Limits of approximation algorithms: PCPs and unique games (DIMACS tutorial lecture notes). arXiv:1002.3864, 2010.

J. Håstad. Clique is hard to approximate within $n^{1-\epsilon}$. *Acta Mathematica*, 182(1):105–142, 1999.

J. Håstad. Some optimal inapproximability results. *Journal of the ACM*, 48(4):798–859, 2001.

T. Holenstein. Parallel repetition: simplification and the no-signaling case. *Theory of Computing*, 5:141–172, 2009.

D. S. Johnson. Approximation algorithms for combinatorial problems. *Journal of Computer and System Sciences*, 9(3):256–278, 1974.

R. M. Karp. Reducibility among combinatorial problems. In R. E. Miller and J. W. Thatcher, editors, *Complexity of Computer Computations*, pages 85–103. Plenum Press, 1972.

S. Khot. On the power of unique 2-prover 1-round games. In *Proceedings of the 34th*

Annual ACM Symposium on Theory of Computing (STOC), pages 767–775, 2002.

S. Khot. On the unique games conjecture (invited survey). In *Proceedings of the 25th IEEE Conference on Computational Complexity (CCC)*, pages 99–121, 2010.

S. Khot. On the proof of the 2-to-2 games conjecture. Expository article, 2019.

S. Khot and O. Regev. Vertex cover might be hard to approximate to within $2-\epsilon$. *Journal of Computer and System Sciences*, 74(3):335–349, 2008.

S. Khot and N. K. Vishnoi. The unique games conjecture, integrality gap for cut problems and embeddability of negative-type metrics into ℓ_1. *Journal of the ACM*, 62(1):Article 8, 2015.

S. Khot, G. Kindler, E. Mossel, and R. O'Donnell. Optimal inapproximability results for MAX-CUT and other 2-variable CSPs? *SIAM Journal on Computing*, 37(1):319–357, 2007.

S. Khot, D. Minzer, and M. Safra. On independent sets, 2-to-2 games, and Grassmann graphs. In *Proceedings of the 49th Annual ACM Symposium on Theory of Computing (STOC)*, pages 576–589, 2017. Journal version: Theory of Computing 21(10), 2025.

S. Khot, D. Minzer, and M. Safra. Pseudorandom sets in Grassmann graph have near-perfect expansion. In *Proceedings of the 59th IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 592–601, 2018.

D. Lapidot and A. Shamir. Fully parallelized multi prover protocols for NEXPTIME. In *Proceedings of the 32nd IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 13–18, 1991.

L. A. Levin. Universal sequential search problems. *Problems of Information Transmission*, 9(3):115–116, 1973.

C. Lund and M. Yannakakis. On the hardness of approximating minimization problems. *Journal of the ACM*, 41(5):960–981, 1994.

C. Lund, L. Fortnow, H. Karloff, and N. Nisan. Algebraic methods for interactive proof systems. *Journal of the ACM*, 39(4):859–868, 1992.

E. Mossel, R. O'Donnell, and K. Oleszkiewicz. Noise stability of functions with low influences: invariance and optimality. *Annals of Mathematics*, 171(1):295–341, 2010.

OpenAI. The unique games theorem. Manuscript, dated September 23, 2026; not independently verified at the time of writing, 2026.

A. Polishchuk and D. A. Spielman. Nearly-linear size holographic proofs. In *Proceedings of the 26th Annual ACM Symposium on Theory of Computing (STOC)*, pages 194–203, 1994.

P. Raghavendra. Optimal algorithms and inapproximability results for every CSP? In *Proceedings of the 40th Annual ACM Symposium on Theory of Computing (STOC)*, pages 245–254, 2008.

R. Raz. A parallel repetition theorem. *SIAM Journal on Computing*, 27(3):763–803, 1998.

R. Raz. A counterexample to strong parallel repetition. *SIAM Journal on Computing*, 40(3):771–777, 2011.

R. Rubinfeld and M. Sudan. Self-testing polynomial functions efficiently and over rational domains. In *Proceedings of the 3rd Annual ACM-SIAM Symposium on Discrete*

Algorithms (SODA), pages 23–32, 1992.

R. Rubinfeld and M. Sudan. Robust characterizations of polynomials with applications to program testing. *SIAM Journal on Computing*, 25(2):252–271, 1996.

L. Trevisan. CS294: PCP and hardness of approximation, lecture notes (lectures 1 and 3). University of California, Berkeley, 2006.

L. Trevisan. On Khot's unique games conjecture. *Bulletin of the American Mathematical Society*, 49(1):91–111, 2012.

L. Trevisan, G. B. Sorkin, M. Sudan, and D. P. Williamson. Gadgets, approximation, and linear programming. *SIAM Journal on Computing*, 29(6):2074–2097, 2000.

D. Zuckerman. Linear degree extractors and the inapproximability of max clique and chromatic number. *Theory of Computing*, 3:103–128, 2007.